Project report (M.Tech level)

Localization, Mapping and Motion Planning for a Self-Driving Car

Visual-inertial odometry · LiDAR SLAM · HD-map localization
Behaviour trees · Hybrid A* and state lattices

vishwasnarayan2345@gmail.com

November 2026

Status. Nine simulation experiments are complete (E1–E5 core, E6–E9 extended) and reported with measured numbers (Section 8). Experiments planned for the coming weeks, and a longer list of research ideas, are in Section 13. All code and raw results are in the accompanying folder (Section 14); `python run_all.py` regenerates every number, figure and table in this report.

Abstract

Safe automated driving needs a vehicle that knows where it is, decides sensibly and moves along paths it can physically drive. This report studies five building blocks of such a stack in a reproducible 2-D urban simulation, implemented from scratch in NumPy/SciPy: (i) a visual-inertial odometry (VIO) filter that bridges GPS outages in a tunnel and an underpass, (ii) LiDAR SLAM with NDT registration and pose-graph loop closure, (iii) localization against a pre-built HD map with NDT and a particle filter, (iv) behaviour trees (BT) for overtaking, roundabout and pedestrian yielding compared with finite-state machines (FSM), and (v) Hybrid A* and a Frenet state lattice for kinematically feasible paths. Measured results: VIO tunnel-exit error 1.33 m against 2.83 m without vision (20 seeds); NDT+loop-closure SLAM ATE 0.15 m against 28.2 m for scan-to-scan ICP; HD-map NDT localization at 1.4/0.8 cm (lon/lat RMS); BT 296/300 scenarios against 298/300 for a complete FSM and 142/300 for a basic one; Hybrid A* feasible in 6/6 perpendicular, 4/6 parallel and 6/6 K-turn trials where grid A* produced no feasible path. Extended runs add a Hybrid A* slot-length sweep, a 20-seed statistical study of SLAM and HD-map localization, a lattice failure analysis and a closed-loop run in which NDT localization drives the whole stack (10/10 successes) while VO dead reckoning fails (0/10). The study is simulation-only, uses simulated visual odometry and, for most experiments, a small number of seeds; Section 11 states what the results do *not* show, and Section 13 lists the follow-up experiments.

Keywords: visual-inertial odometry, LiDAR SLAM, normal distributions transform, particle filter, HD map, behaviour trees, Hybrid A*, state lattice, autonomous driving.

Contents

1 Introduction **5**
- 1.1 Motivation and problem statement . . . 5
- 1.2 Objectives . . . 5
- 1.3 Contributions . . . 5

2 Related work **5**

3 Problem formulation and evaluation metrics **6**

4 System architecture and code layout **6**

5 Simulation environment **7**

6 Methods and implementation **7**
- 6.1 Visual-inertial odometry (`vio.py`) . . . 7
- 6.2 LiDAR SLAM (`ndt.py`, `lidar_slam.py`, `pose_graph.py`) . . . 8
- 6.3 HD-map localization (`hd_map.py`, `particle_filter.py`) . . . 8
- 6.4 Behaviour tree (`bt.py`, `scenario.py`) . . . 8
- 6.5 Hybrid A* and state lattice (`hybrid_astar.py`, `dubins.py`, `lattice.py`) . . . 9
- 6.6 Closed-loop stack (`exp9_closed_loop.py`) . . . 10

7 Experimental methodology **11**

8 Results (completed experiments) **12**
- 8.1 E1: VIO through GPS-denied sections . . . 12
- 8.2 E2: LiDAR SLAM . . . 13
- 8.3 E3: HD-map localization . . . 14

8.4 E4: Behaviour tree vs. finite-state machine . . . 15
8.5 E5: Motion planning . . . 17
8.6 E6: Hybrid A* slot length and search speed . . . 18
8.7 E7: lattice failure analysis . . . 19
8.8 E8: 20-seed statistics and LiDAR degradation . . . 20
8.9 E9: closed-loop integration . . . 21

9 Discussion **21**

10 Real-world applications **22**

11 Limitations and threats to validity **22**

12 Conclusions **23**

13 Future work **24**
13.1 Planned experiments and status . . . 24
13.2 Experiment log (extend this table) . . . 24
13.3 Research ideas beyond this week . . . 25

14 Reproducibility and folder structure **26**

A Source code **28**

List of Figures

1 Pipeline. Each solid block is implemented and evaluated on its own; integration is planned (P4). . . . 6
2 Implemented behaviour tree, priority from top to bottom (? = fallback, → = sequence, ellipse = condition, red box = action). . . . 9
3 E1, representative run. Left: route (tunnel in grey). Right: position error (log scale); shaded = tunnel and underpass. . . . 12
4 E1: tunnel-exit error across the Monte-Carlo runs. . . . 12
5 E2, seed 0. Left: trajectories. Centre: SLAM map and loop-closure edges. Right: absolute trajectory error against distance driven (log scale). . . . 13
6 E2: NDT Hessian conditioning along the route (seed 0); lower means weaker constraint. 14
7 E3. Left: error along the drive (seed 0, 15 unmapped vehicles). Centre: position RMS vs. number of unmapped vehicles. Right: global localization from a uniform prior. . . . 14
8 E4: one BT run (labels = active leaf). The ego follows a slow lead through the roundabout (yield, traverse), starts an overtake at about 59 s, aborts it shortly afterwards (`ABORT FALL BACK`) as an oncoming vehicle approaches, and later yields at the crosswalk. . . . 16
9 E4: share of scenarios completed without collision. . . . 16
10 E5: Hybrid A* (blue, footprints every few poses; dashed black = goal pose) and grid A* (orange) from the nominal start poses. . . . 17
11 E5: executed lateral profile and curvature on the route with 7 obstacles (5 vehicles, 2 small objects); dotted = vehicle curvature limit. . . . 17

12 E6. Left: feasible starts vs. slot length. Centre: solve time of feasible runs. Right: search variants on the 6.2 m slot (run time capped at 40 s). 18
13 E7: the layouts in which the quintic lattice fails. Dark = cells blocked for the 3-disc body at road heading, light = free; blue = executed path; dashed orange = first collision. 19
14 E8. Left: SLAM ATE RMSE per seed (dots) with mean and 95% bootstrap CI. Right: HD-map position RMS under LiDAR degradation, mean and 95% CI. 20
15 E9, seed 0. Left: true path in Frenet coordinates for the three pose sources (grey = obstacles, stretched by the unequal axes). Right: localization error in the loop. . . . 21

List of Tables

1 Modules and source files (full listings in Appendix A). 7
2 Sensor models (all noise parameters are in `sim/drive.py`). 7
3 Completed experiments. Scripts are in `experiments/`; raw results in `results/*.json`. 11
4 E1: position error [m] over a 1.25-lap drive (20 Monte-Carlo runs, mean±std). . . . 13
5 E1: VIO tunnel-exit error vs. VO noise scale (6 runs each). 13
6 E2: LiDAR SLAM, 1.25 laps, 3 seeds (mean±std). Map error = distance of map points to noise-free scene geometry [m]. 13
7 E3: localisation against the HD map (3 seeds; mean). Errors in the vehicle frame [cm]; "unmapped" = vehicles absent from the map. 14
8 E3: particle-count sweep (seed 0, 15 unmapped vehicles). 15
9 E3: global localisation from a uniform prior (20 000 particles, 45 s). 15
10 E4: 300 randomised scenarios (lead vehicle, oncoming traffic, roundabout, pedestrian). Success = route completed without collision. 15
11 E4: structural size of the decision logic. 15
12 E5: tight-space planning, 6 start poses per scenario. Feasible = collision-free (exact footprint), curvature $\leq 0.212\,\mathrm{m}^{-1}$, goal pose reached. Means over found paths. . . . 17
13 E5: receding-horizon lateral planning on 20 random obstacle layouts. OK = collision-free and curvature within the vehicle limit. 18
14 E6: Hybrid A* parallel parking vs. slot length (car 4.5 m, 6 start poses per slot, 40 s / 60 000-expansion budget). Time, expansions, cusps and length are means over feasible runs. 18
15 E6: search-speed variants on the nominal 6.2 m slot (same 6 start poses, run back to back on the same machine). 18
16 E7: the layouts in which the quintic lattice fails. "Corridor path" = a Frenet-grid search finds any path from the start to $s = 360$ m inside the 6.2 m corridor (lateral slope ≤ 0.4, 3-disc body at road heading), with zero clearance and with the planner's 0.35 m margin; the last three columns re-run the planner with a denser lattice, 2 m replanning, or both. 19
17 E8: SLAM ATE RMSE [m] over 20 seeds: mean with 95% bootstrap CI of the mean, median and standard deviation. 20
18 E8: HD-map localization under LiDAR degradation (20 seeds, 15 unmapped vehicles). Position RMS [cm], mean with 95% bootstrap CI; "fail" = maximum error above 1 m. 20

19 E9: closed loop (pose source → behaviour tree → lattice → pure pursuit → bicycle model), 10 runs per pose source, 7 obstacles on a 330 m route. Clearance is the minimum body-to-obstacle gap; cross-track is the distance of the true position to the current plan. 21
20 Planned experiments and status. Hypotheses are expectations, not findings; results of the experiments that were run are in the cited sections. 24
21 Experiment log. 24

1 Introduction

1.1 Motivation and problem statement

An automated vehicle has to answer three questions continuously: *where am I?* (localization and mapping), *what should I do?* (behaviour decision) and *how do I get there?* (motion planning). Each has a well-known failure mode: GNSS is unavailable in tunnels and urban canyons; odometry and scan-matching drift; maps become stale; hand-wired decision logic becomes brittle as behaviours are added; and grid-based paths ignore the vehicle's minimum turning radius and body shape. This project builds one working implementation of a remedy for each and measures how well it works, on a common simulator, with fixed seeds.

1.2 Objectives

- **O1** Keep a metre-level position estimate through GPS-denied tunnel and underpass sections (VIO).
- **O2** Build a consistent map online and quantify drift with and without loop closure (LiDAR SLAM).
- **O3** Localize to centimetre level against a pre-built HD map and test robustness to unmapped objects (NDT, particle filter).
- **O4** Replace a rigid FSM by a BT for overtaking, roundabout yielding and pedestrian yielding, and measure the trade-off.
- **O5** Generate smooth, curvature-bounded paths for parking and tight turns (Hybrid A*) and for on-road obstacle avoidance (state lattice), against grid-based baselines.

1.3 Contributions

C1 A compact code base (about 3 700 lines of Python for library, experiments and tests) implementing a stochastic-cloning VIO EKF, multi-resolution NDT SLAM with an SE(2) pose graph, HD-map NDT and particle-filter localization, a behaviour-tree library with FSM baselines, and Hybrid A* with Dubins shots and a quintic Frenet lattice.

C2 A reproducible simulation benchmark with GPS-denied sections, repetitive geometry, unmapped objects, scripted traffic scenarios and tight parking maps; every experiment is seeded and regenerates its figures and LaTeX tables.

C3 Nine experiments with measured results, including the cases that did *not* work: global localization converged in 1 of 3 seeds, parallel parking was solved in 4/6 trials, search speed-ups lost solutions, and a complete FSM matched the BT in safety (the BT advantage found is structural).

C4 An explicit validity analysis and a list of planned experiments and research ideas (Section 13).

Scope. Planar (SE(2)) models, one synthetic map, modules evaluated independently (no closed loop yet). The "visual" front end is *not* implemented: VO measurements are simulated from ground truth with a parametric error model.

2 Related work

Visual-inertial odometry. Filter-based VIO such as the MSCKF [24] keeps a window of camera poses in the state; stochastic cloning [30] is the underlying device for processing relative-pose measurements without double counting. Optimization-based systems (OKVIS [21], VINS-Mono [28], ORB-SLAM3 [5]) use IMU preintegration [12]. The filter here follows the first line in a loosely coupled, planar form.

LiDAR odometry and SLAM. Scan-to-scan ICP [1, 31] drifts; LOAM [39] and its descendants [32, 33, 38] register edge and planar features to a local map. The normal distributions

transform (NDT) [2, 23] registers a scan to a grid of Gaussians and is the basis of HDL-graph-SLAM [19]. Back ends are pose graphs [14, 20]; loop edges are commonly robustified (Huber [15], switchable constraints [35]). Degeneracy of the registration problem (e.g. corridors) can be read from the Hessian spectrum [40], which motivates the indicator used here.

Map-based localization. Monte Carlo localization [7] and NDT matching against a prior point-cloud map are the two standard approaches in open-source driving stacks [17]; HD-map formats such as Lanelet2 [27] add semantics. Probabilistic foundations are in [36].

Behaviour architectures. FSMs are the traditional choice; behaviour trees [6, 16] provide hierarchy, reactivity and modularity and are used in robot navigation stacks [22]. Quantitative FSM-versus-BT comparisons on driving scenarios are rare, which motivates E4.

Motion planning. Hybrid A* [8] searches continuous vehicle states on a discretized grid; Dubins [10] and Reeds–Shepp [29] curves give analytic connections and heuristics; state lattices [26] and Frenet-frame polynomial sampling [37] are the on-road counterparts. Optimization-based refinement [42] and production planners [11] build on these. Surveys: [25].

3 Problem formulation and evaluation metrics

Notation. A planar pose is $\mathbf{x} = (\mathbf{p}, \theta) \in SE(2)$, $\mathbf{p} \in \mathbb{R}^2$, with rotation $R(\theta)$; $\oplus$ composes and $\ominus$ gives the relative pose. Inputs are IMU samples $(g_z, \mathbf{a}_m)$, relative visual odometry $\mathbf{z}_{vo}$, intermittent GPS positions $\mathbf{z}_{gps}$, 2-D LiDAR scans $\mathcal{S}_k$ and, for E3, a map $\mathcal{M}$. The estimation task is $\hat{\mathbf{x}}_k = f(\text{inputs}_{1:k})$.

Localization errors. With ground truth $\mathbf{x}^{gt}$ and $\mathbf{d} = \hat{\mathbf{p}} - \mathbf{p}^{gt}$,

$$\begin{bmatrix} e_{lon} \\ e_{lat} \end{bmatrix} = R(\theta^{gt})^\top \mathbf{d}, \qquad e_{pos} = \|\mathbf{d}\|, \qquad \text{ATE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\mathbf{p}}_i - \mathbf{p}_i^{gt}\|^2}. \tag{1}$$

All estimators start from the known initial pose, so no trajectory alignment is applied before computing ATE [34, 41]. SLAM ATE is evaluated at keyframes; map error is the nearest-neighbour distance from map points to noise-free simulator geometry.

Decision metric. A scenario is a *success* if the ego vehicle reaches the end of the route without collision. Hard-brake events and structural size (nodes, transitions) are reported as secondary metrics.

Planning feasibility. A path is *feasible* if it (a) is collision-free for the exact rectangular footprint, (b) has peak curvature $\leq 1.05\,\kappa_{\max}$, and (c) ends within 0.35 m and 6° of the goal pose (parking) — for the lattice, (a) and (b) only. The search itself accepts a goal within 0.30 m and 0.09 rad.

Reporting. Means ± sample standard deviations are over seeds; no significance tests are run because sample sizes are small (3–20 seeds).

4 System architecture and code layout

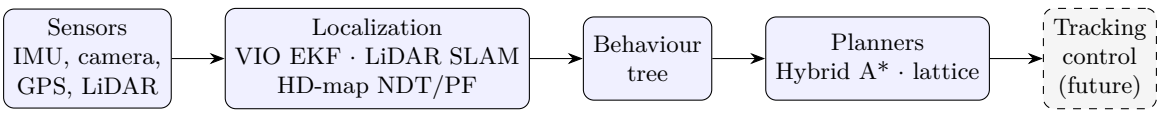


Figure 1: Pipeline. Each solid block is implemented and evaluated on its own; integration is planned (P4).

Table 1: Modules and source files (full listings in Appendix A).

Topic	Files	Experiment
Simulator, sensors	`sim/world.py`, `sim/drive.py`, `geometry.py`	all
VIO (EKF, stochastic cloning)	`localization/vio.py`	E1
LiDAR SLAM (NDT, ICP, pose graph)	`ndt.py`, `icp.py`, `pose_graph.py`, `lidar_slam.py`	E2
HD map, NDT/PF localization	`hd_map.py`, `particle_filter.py`	E3
Behaviour tree, FSM baselines	`behavior/bt.py`, `behavior/scenario.py`	E4
Hybrid A*, Dubins, lattice	`planning/*.py`	E5
Unit tests	`tests/test_core.py` (6 tests, pass)	–

5 Simulation environment

The road is a closed rounded rectangle (220×140 m, corner radius 25 m, length 677 m) with buildings, poles and trees, a 130 m curved *tunnel* (route position $s \in [200, 330]$ m, low texture) and a 25 m *underpass* ($s \in [500, 525]$ m); GPS is unavailable in both. The vehicle drives 1.25 laps (846 m) with a curvature-limited speed profile (≤ 11 m/s).

Table 2: Sensor models (all noise parameters are in `sim/drive.py`).

Sensor	Rate	Model
IMU	100 Hz	gyro-z and body-frame (a_x, a_y); white noise 0.002 rad/s, 0.05 m/s^2; random-walk biases
Visual odometry	10 Hz	relative pose; $\sigma_t = 1\,\text{cm} + 1\% \cdot d$, $\sigma_r = 1.5\,\text{mrad} + 0.2\% \cdot \lvert\Delta\theta\rvert$, 0.4% scale error, $\times 2.5$ noise in the tunnel
GPS	1 Hz	position, $\sigma = 0.6$ m, absent in tunnel/underpass
LiDAR	10 Hz	360 beams, 40 m range, $\sigma = 2\,\text{cm} + 0.1\% \cdot r$, 2% dropout, ray-cast against walls and poles

The HD map (30 498 points) is built from a separate survey drive using LiDAR scans placed with ground-truth poses perturbed by (2 cm, 2 cm, 1.5 mrad), a stand-in for an RTK-INS mapping vehicle.

6 Methods and implementation

6.1 Visual-inertial odometry (`vio.py`)

A loosely coupled EKF with state $\mathbf{x} = [\mathbf{p}, \theta, \mathbf{v}, b_g, \mathbf{b}_a] \in \mathbb{R}^8$ and a *cloned* previous camera pose $\mathbf{c} = (\mathbf{p}_c, \theta_c)$ (stochastic cloning), so that relative VO measurements do not reuse information. IMU propagation:

$$\omega = g_z - b_g, \quad \mathbf{a}_w = R(\theta)(\mathbf{a}_m - \mathbf{b}_a), \quad \mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\Delta t + \tfrac{1}{2}\mathbf{a}_w \Delta t^2, \quad \mathbf{v} \leftarrow \mathbf{v} + \mathbf{a}_w \Delta t, \quad \theta \leftarrow \theta + \omega \Delta t. \tag{2}$$

The VO update compares the measured relative pose with the one implied by the state and the clone, then re-clones:

$$\mathbf{z}_{vo} = \begin{bmatrix} R(\theta_c)^\top (\mathbf{p} - \mathbf{p}_c) \\ \theta - \theta_c \end{bmatrix} + \mathbf{n}, \qquad \mathbf{n} \sim \mathcal{N}(0, \operatorname{diag}(\sigma_t^2, \sigma_t^2, \sigma_r^2)) \cdot 1.3^2. \tag{3}$$

GPS updates ($\mathbf{z} = \mathbf{p} + \mathbf{n}$) and VO updates are χ^2-gated (99.9%) [45]. The covariance uses the Joseph form and is symmetrized after every step (needed for low VO noise, where round-off otherwise made the position variance negative).

6.2 LiDAR SLAM (`ndt.py`, `lidar_slam.py`, `pose_graph.py`)

Front end (HDL-graph-SLAM style [19], NDT [2]): keyframe-based NDT scan-to-submap registration (last 20 keyframes, cells 3 m then 1.5 m, four half-cell-shifted grids). With $\mathbf{x}_i' = T(\mathbf{p})\mathbf{x}_i$ and cell mean/covariance $(\boldsymbol{\mu}_i, \Sigma_i)$ the cost minimised by damped Newton is

$$F(\mathbf{p}) = \sum_i d_1 \exp\Big(-\tfrac{d_2}{2} (\mathbf{x}_i' - \boldsymbol{\mu}_i)^\top \Sigma_i^{-1} (\mathbf{x}_i' - \boldsymbol{\mu}_i) \Big), \qquad d_1 < 0, \tag{4}$$

with d_1, d_2 from the Gaussian-plus-uniform approximation. The conditioning of the translational Hessian block, $\rho = \lambda_{\min}/\lambda_{\max}$, is logged per scan as a degeneracy indicator. A new keyframe is added every 1.5 m or 0.15 rad. **Loops** are proposed when a keyframe is within 20 m of one that is > 80 m of travel away, verified by multi-start NDT with a KD-tree inlier ratio ≥ 0.65. **Back end:** SE(2) pose graph

$$\min_{\mathbf{x}} \sum_{(i,j)} \varrho(\mathbf{e}_{ij}^\top \Omega_{ij} \mathbf{e}_{ij}), \quad \mathbf{e}_{ij} = \begin{bmatrix} R(z_\theta)^\top \big(R(\theta_i)^\top (\mathbf{t}_j - \mathbf{t}_i) - \mathbf{z}_t \big) \\ \operatorname{wrap}(\theta_j - \theta_i - z_\theta) \end{bmatrix}, \tag{5}$$

solved by sparse Gauss–Newton with a Huber weight $3/\sqrt{\chi^2}$ on loop edges. Baseline: scan-to-scan point-to-line ICP odometry. A LOAM-style curvature feature extractor (`extract_loam_features`) is implemented but *not* part of E2 (see P1).

6.3 HD-map localization (`hd_map.py`, `particle_filter.py`)

NDT localizer: the same multi-resolution NDT, registering each scan against the HD-map NDT grid from an odometry prediction (VO relative pose). **Particle filter:** 500 particles, motion noise on the VO increment, likelihood from a Euclidean distance field $d(\cdot)$ of the map with 72 beams per scan:

$$w^{[m]} \propto \prod_k \Big[(1 - z_r)\, e^{-d_k^2/2\sigma^2} + z_r \Big]^\beta, \quad \sigma = 0.2\,\text{m},\ z_r = 0.08,\ \beta = 0.35, \tag{6}$$

with systematic resampling when $N_{\text{eff}} < N/2$.

6.4 Behaviour tree (`bt.py`, `scenario.py`)

A tick-based BT library (Sequence →, Fallback ?, Condition, Action) with blackboard; a higher-priority branch *halts* running lower branches (pre-emption). The decision tree (Fig. 2) is re-evaluated every 0.1 s: emergency brake > pedestrian yield > roundabout (enter if the gap is safe, else hold at the yield line) > abort overtake > overtake > follow > cruise. Longitudinal control is an intelligent-driver model (IDM) [43] towards the target speed or a virtual obstacle (stop line, lead vehicle); lane changes are quintic. Two hand-written FSMs use the same predicates and controller: *FSM-basic* (transitions for the nominal sequence plus a global emergency brake) and *FSM-full* (adds abort and yield edges from every relevant state).

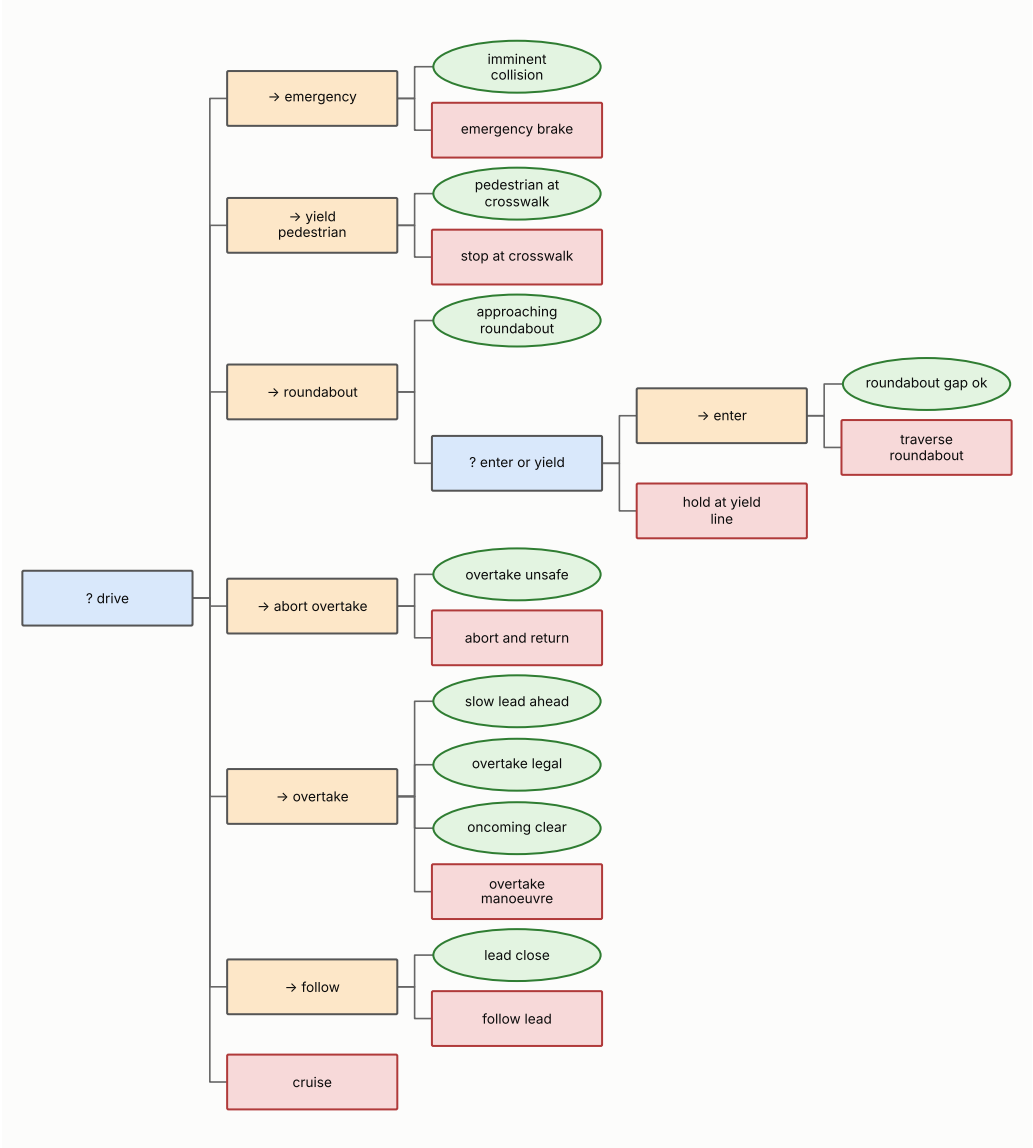


Figure 2: Implemented behaviour tree, priority from top to bottom (? = fallback, → = sequence, ellipse = condition, red box = action).

6.5 Hybrid A* and state lattice (`hybrid_astar.py`, `dubins.py`, `lattice.py`)

Hybrid A* [8] searches continuous poses of a kinematic bicycle ($\dot{\theta} = \frac{v}{L}\tan\delta$, $L = 2.7\,\mathrm{m}$, $|\delta| \leq 0.52\,\mathrm{rad}$, hence $\rho_{\min} = 4.72\,\mathrm{m}$, $\kappa_{\max} = 0.212\,\mathrm{m}^{-1}$) with five steering angles × {forward, reverse} × arc lengths $\{0.6, 1.5\}\,\mathrm{m}$, de-duplicated on a $0.5\,\mathrm{m}/5^\circ$ grid. Arc cost

$$\Delta g = s\,(1 + 2\cdot\mathbb{1}_{rev}) + 0.4|\delta| + 1.5|\delta - \delta_{prev}| + 6\cdot\mathbb{1}_{gear\ switch}, \tag{7}$$

$$h = \max\left(h_{hol},\ \min(L^{fwd}_{Dubins},\ 3L^{rev}_{Dubins})\right), \tag{8}$$

where h_{hol} is a holonomic Dijkstra distance on the obstacle grid. A collision-checked Dubins "shot" (forward or time-reversed for reverse driving) to the goal is attempted during the search. The 0.6 m arcs are needed to move inside slots that leave $\approx 0.85\,\mathrm{m}$ of slack at each end. Collision checking samples the oriented car rectangle (0.3 m lattice) on a distance field; final paths are re-verified against raw occupancy with a 4 cm raster. **Baseline:** 8-connected grid A* for a point robot inflated by half the car width (optimistic), raw and Chaikin-smoothed [46]; both use A* [44].

State lattice (Frenet frame of the route): quintic lateral profiles $d(s)$ with $(d_0, d_0', d_0'') \to (d_f, 0, 0)$ over horizons $S_f \in \{18, 26, 36\}\,\mathrm{m}$ and 13 terminal offsets, so the stitched receding-horizon

path is C^2. Curvature on a road of curvature κ_r (κ_r' neglected):

$$\kappa = \frac{\kappa_r + \dfrac{d''a + d'^2\kappa_r}{a^2 + d'^2}}{\sqrt{a^2 + d'^2}}, \quad a = 1 - \kappa_r d, \tag{9}$$

$$J = 40\int d'''^2 ds + \left[(d_f - d_{ref})^2 + \overline{(d - d_{ref})^2}\right] + 200\max(|\kappa|_{\max} - 0.6\kappa_{\lim}, 0)^2. \tag{10}$$

Candidates leaving the road, exceeding $\kappa_{\lim}$ or closer than 0.35 m to an obstacle (disc model) are rejected. The baseline uses the same candidates and checks but piecewise-linear 12 m ramps.

6.6 Closed-loop stack (`exp9_closed_loop.py`)

Every 0.1 s a pose estimate is produced by one of three sources: ground truth, NDT against the HD map (with the simulated VO increment as prior), or VO dead reckoning. Every 0.5 s the estimate is projected to a Frenet state (s, d, d'), a three-behaviour BT (*stop* if the last plan had no feasible candidate, *slow* at 5 m/s if an obstacle is within 45 m, else *cruise* at 8 m/s) sets the speed, and the quintic lattice plans around the 7 obstacles. A pure-pursuit tracker steers a kinematic bicycle (50 Hz, steering rate ≤ 0.6 rad/s) using the *estimated* pose:

$$\delta = \operatorname{atan2}\left(2L\sin\alpha,\ L_d\right), \qquad L_d = \operatorname{clip}(v + 3,\ 4,\ 10)\ \text{m}, \tag{11}$$

with α the bearing of the look-ahead point relative to the estimated heading. Obstacle positions are known exactly (perfect perception) but are absent from the HD map, and are rendered as octagons in the simulated LiDAR scans, so NDT sees them as unmapped objects. A run is a success if the true body reaches $s = 360$ m without touching an obstacle.

7 Experimental methodology

Table 3: Completed experiments. Scripts are in `experiments/`; raw results in `results/*.json`.

ID	Script	Setup	Metrics
E1	`exp1_vio.py`	20 seeds of sensor noise; 4 estimators + VIO with GPS only for 10 s; VO-noise sweep (×0.5–4)	position RMSE, tunnel exit/max, underpass max, yaw
E2	`exp2_slam.py`	3 seeds (drive and noise); ICP vs NDT odometry vs NDT + loops; map vs noise-free geometry	ATE (keyframes), map error, ms/scan, ρ
E3	`exp3_hdmap.py`	3 seeds × 0,15,40 unmapped vehicles; PF sweep; global localization (20k particles, 3 seeds)	lon/lat/yaw RMS, P95, convergence
E4	`exp4_behavior.py`	300 random scenarios (seed 123); BT, FSM-basic, FSM-full, FSM-full minus one edge	success, collisions, hard brakes, structure
E5	`exp5_planning.py`	3 tight-space maps × 6 start poses; 20 random obstacle layouts for the lattice	feasibility, time, cusps, $\kappa_{\max}$
E6	`exp6_hybrid_sweep.py`	parallel slot 5.5–7.5 m × 6 starts; 4 search variants at 6.2 m	feasible starts, time, expansions
E7	`exp7_lattice_failures.py`	the 4 failing E5 layouts: Frenet-grid solvability, denser lattice, 2 m replanning	solvable?, recovered?
E8	`exp8_sensitivity.py`	E2 with 20 seeds; E3 with 20 seeds × {nominal, noise ×4, 40% dropout}	mean, 95% bootstrap CI, failures
E9	`exp9_closed_loop.py`	3 pose sources × 10 runs through the full stack	success, clearance, cross-track

Errors are measured against simulator ground truth. Means ± standard deviations are over seeds; E8 reports means with 95% percentile-bootstrap confidence intervals (5 000 resamples). Seeds are fixed in the scripts, so every number can be regenerated with `python run_all.py`.

8 Results (completed experiments)

8.1 E1: VIO through GPS-denied sections

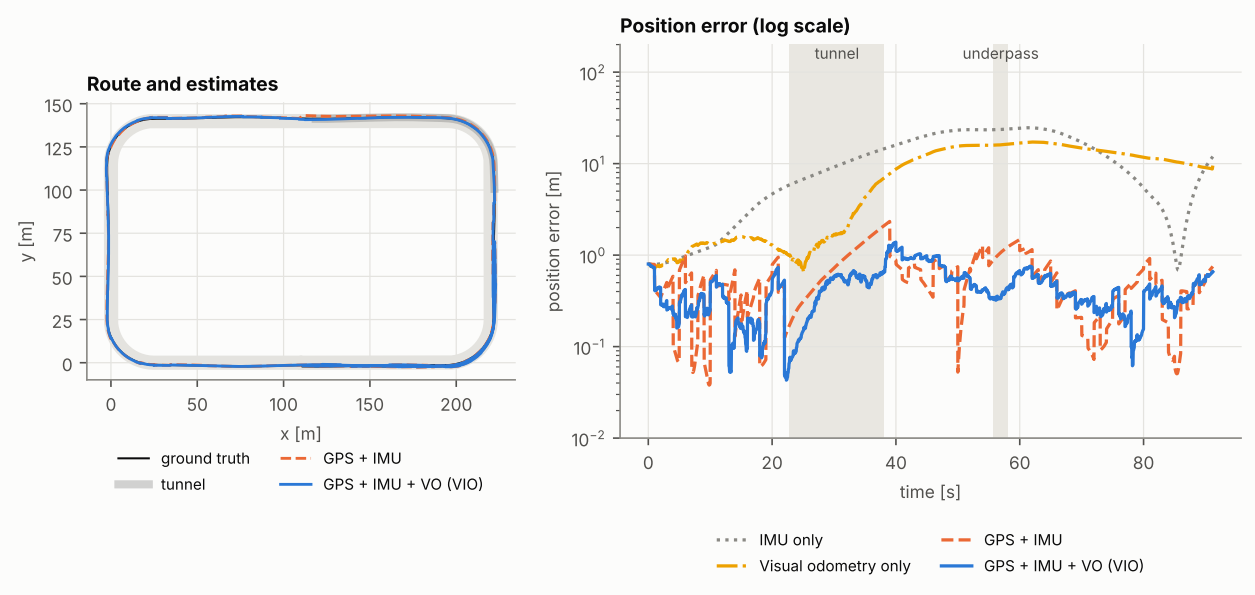


Figure 3: E1, representative run. Left: route (tunnel in grey). Right: position error (log scale); shaded = tunnel and underpass.

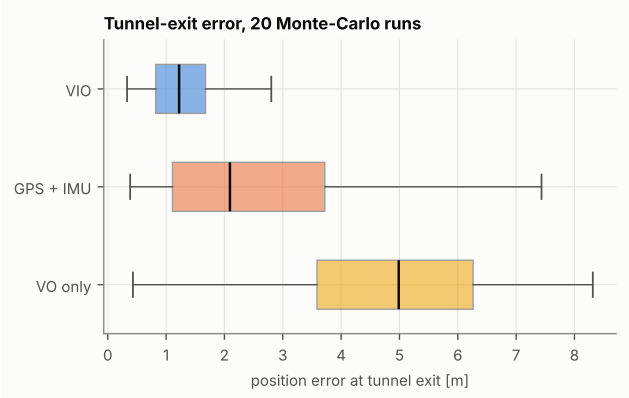


Figure 4: E1: tunnel-exit error across the Monte-Carlo runs.

Tunnel-exit error is 1.33 m for VIO, against 2.83 m for the same EKF without VO, 4.76 m for VO alone and 35 m for the IMU alone (Table 4); VIO whole-run RMSE is 0.59 m. If GPS is removed after 10 s, the final error is 0.47% of the distance driven. In the tunnel the filter reports $\sigma \approx 1.01$ m (radial) against a measured RMSE of 0.86 m, and errors stay inside 3σ for 100% of tunnel samples, so the covariance is conservative rather than over-confident. Exit error grows monotonically with VO noise (Table 5). **Observation:** VIO gives *metre-level* continuity during outages; centimetre level comes only from map-based localization (E3). The VO input is simulated, so these numbers bound the filter, not a real camera pipeline.

Table 4: E1: position error [m] over a 1.25-lap drive (20 Monte-Carlo runs, mean±std).

Estimator	RMSE	Max in tunnel	Tunnel exit	Max underpass	Yaw RMSE [deg]
IMU only	43.48±23.65	35.40±19.96	35.40±19.96	58.92±34.34	12.13
Visual odometry only	6.66±2.84	5.26±2.09	4.76±2.25	8.71±4.81	2.31
GPS + IMU	0.97±0.40	2.92±2.03	2.83±2.11	0.75±0.33	1.30
GPS + IMU + VO (VIO)	0.59±0.17	1.42±0.72	1.33±0.74	0.62±0.27	0.54
VIO, GPS only first 10 s	3.12±1.56	2.77±1.16	2.57±1.19	4.41±2.38	1.45

Table 5: E1: VIO tunnel-exit error vs. VO noise scale (6 runs each).

VO noise scale	×0.5	×1.0	×2.0	×4.0
Tunnel-exit error [m]	0.84	1.49	2.34	3.42
Max tunnel error [m]	0.88	1.54	2.39	3.44

8.2 E2: LiDAR SLAM

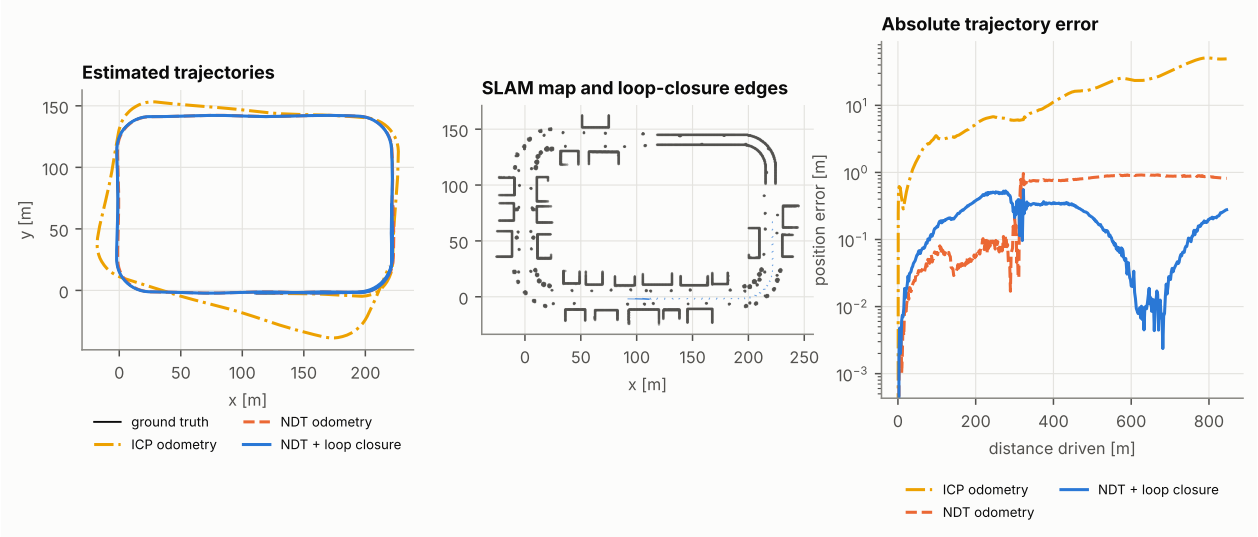


Figure 5: E2, seed 0. Left: trajectories. Centre: SLAM map and loop-closure edges. Right: absolute trajectory error against distance driven (log scale).

Table 6: E2: LiDAR SLAM, 1.25 laps, 3 seeds (mean±std). Map error = distance of map points to noise-free scene geometry [m].

Method	ATE RMSE [m]	ATE max [m]	Final [m]	Map mean / P95 [m]	ms/scan	Loops
ICP odometry (scan-to-scan)	28.22±4.61	53.17±2.05	52.13±2.44	–	–	–
NDT scan-to-submap odometry	0.46±0.27	0.72±0.37	0.52±0.28	0.249 / 0.634	14.8	0
NDT + loop closure (pose graph)	0.15±0.10	0.35±0.19	0.14±0.12	0.077 / 0.227	30.1	32

Scan-to-scan ICP drifts to 28.2 m ATE RMSE (final heading error ≈ 24°); NDT scan-to-submap odometry reaches 0.46 m; adding loop closure gives 0.15 m with 32 loops closed and lowers the mean map error from 0.249 to 0.077 m (Table 6). In the representative run (Fig. 5, right) the NDT-odometry error steps from ≈ 0.1 m to ≈ 0.8 m around the tunnel exit and then stays roughly constant until loop closure removes it. The median NDT Hessian ratio ρ is 0.026 in the tunnel and 0.225 elsewhere (9× lower; Fig. 6), which is consistent with weak longitudinal constraint in a featureless corridor; the causal link was not isolated (P2). **ICP caveat:** in interactive diagnostics

(not stored in `results/`) the per-scan ICP error was small and not tunnel-specific (RMS 4.4 cm in the tunnel, 2.8 cm elsewhere), with a mean heading bias of about -5×10^{-4} rad per scan in corners that accumulates into the heading drift above. The ICP baseline is untuned (360-beam 2-D scans, ≈217 points), so the result says little about ICP in general. The loop count is identical across seeds because the route geometry is identical.

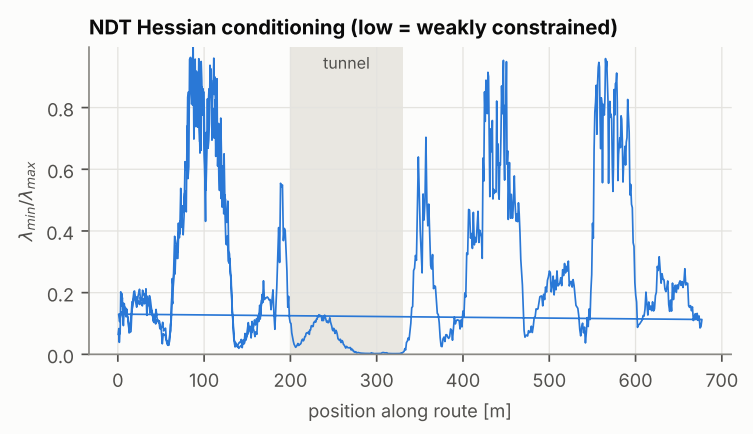


Figure 6: E2: NDT Hessian conditioning along the route (seed 0); lower means weaker constraint.

8.3 E3: HD-map localization

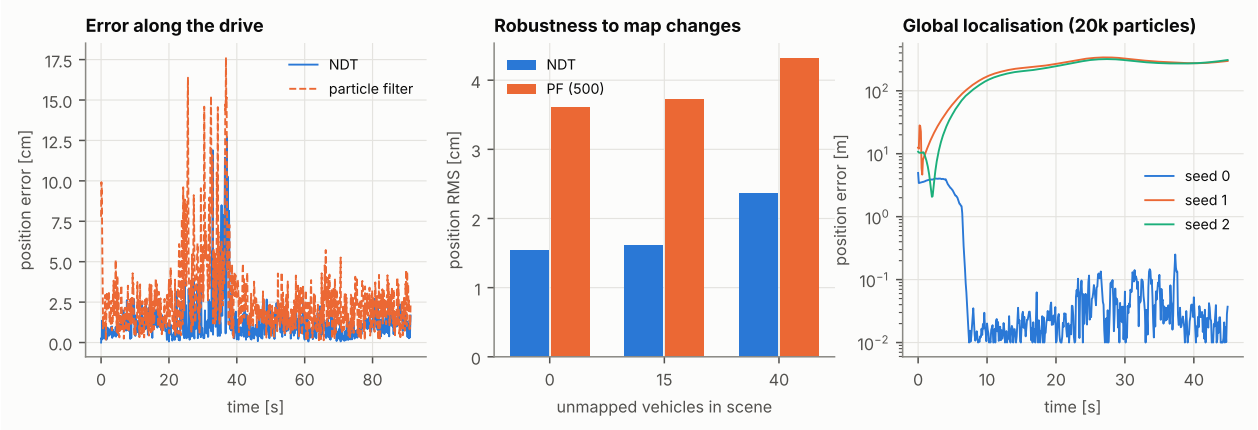


Figure 7: E3. Left: error along the drive (seed 0, 15 unmapped vehicles). Centre: position RMS vs. number of unmapped vehicles. Right: global localization from a uniform prior.

Table 7: E3: localisation against the HD map (3 seeds; mean). Errors in the vehicle frame [cm]; "unmapped" = vehicles absent from the map.

Unmapped	Method	Lon RMS	Lat RMS	Yaw RMS [deg]	P95	Max	ms/scan
0	NDT	1.3	0.7	0.016	2.6	12.1	11.9
	PF (500)	3.3	1.5	0.063	7.7	19.2	1.7
	VO dead reckoning	454.0	320.9	2.636	1010.5	1079.4	–
15	NDT	1.4	0.8	0.018	2.7	12.5	11.7
	PF (500)	3.4	1.6	0.068	7.7	18.5	1.6
	VO dead reckoning	454.0	320.9	2.636	1010.5	1079.4	–
40	NDT	1.9	1.4	0.044	2.8	45.2	11.5
	PF (500)	3.9	1.9	0.081	8.9	22.4	1.5
	VO dead reckoning	454.0	320.9	2.636	1010.5	1079.4	–

With 15 unmapped vehicles NDT reaches 1.4 cm longitudinal and 0.8 cm lateral RMS (max 12 cm), the particle filter 3.4 / 1.6 cm; dead reckoning on VO alone is metres off (Table 7). Both degrade gradually as more vehicles are absent from the map, and both peak while driving through the tunnel (Fig. 7, left, $t \approx 23$–38 s). The PF error saturates beyond about 1 000 particles (Table 8). **Global localization is not reliable:** only 1 of 3 seeds converged from the uniform prior (Table 9); the other two settled roughly 300 m from the true pose in this repetitive road geometry.

Table 8: E3: particle-count sweep (seed 0, 15 unmapped vehicles).

Particles	Lon RMS [cm]	Lat RMS [cm]	ms/scan
50	4.0	2.1	0.3
200	3.4	1.7	0.6
1000	3.2	1.6	2.7
3000	3.1	1.6	8.2

Table 9: E3: global localisation from a uniform prior (20 000 particles, 45 s).

Seed	Converged at [s]	Final error [m]
0	6.6	0.04
1	not converged	297.30
2	not converged	308.21

8.4 E4: Behaviour tree vs. finite-state machine

Table 10: E4: 300 randomised scenarios (lead vehicle, oncoming traffic, roundabout, pedestrian). Success = route completed without collision.

Controller	Success	of which surprise	Collisions	Time [s]	Hard brakes/run	Emerg.	Aborts
Behaviour tree	296/300	146/150	lead 2, oncoming 2	84.4	1.00	7	20
FSM-basic (nominal transitions)	142/300	67/150	roundabout 147, lead 11	78.6	0.53	22	0
FSM-full (all pre-emption edges)	298/300	148/150	oncoming 2	86.6	0.97	7	20
FSM-full, re-check edge removed	149/300	74/150	roundabout 149, oncoming 2	79.6	0.54	7	22

Surprise = an oncoming vehicle outside the 90 m perception range appears mid-overtake (150 of 300 scenarios).

Table 11: E4: structural size of the decision logic.

Quantity	BT	FSM (nominal)	FSM (full)
Nodes / states	26	10	10
Transitions	–	24	38
Edits to add one global-priority behaviour	3 nodes	–	11 transitions

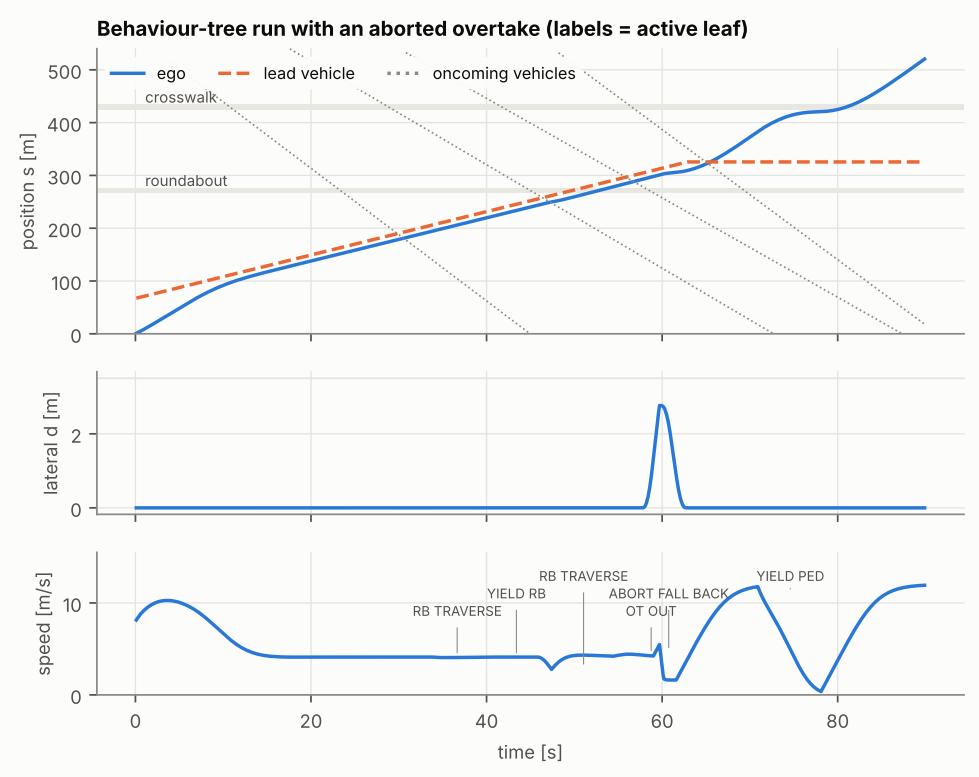


Figure 8: E4: one BT run (labels = active leaf). The ego follows a slow lead through the roundabout (yield, traverse), starts an overtake at about 59 s, aborts it shortly afterwards (`ABORT FALL BACK`) as an oncoming vehicle approaches, and later yields at the crosswalk.

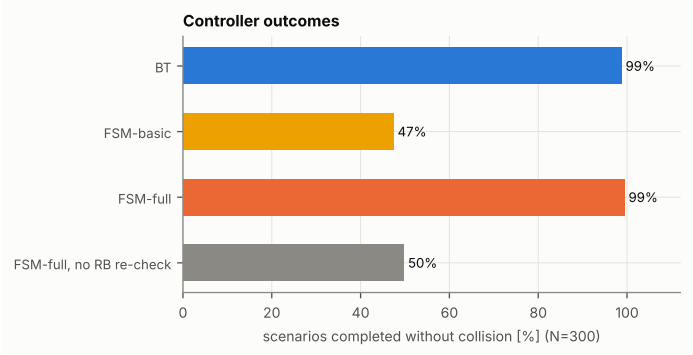


Figure 9: E4: share of scenarios completed without collision.

The BT completes 296/300 scenarios, FSM-full 298/300 and FSM-basic 142/300 (Table 10). FSM-basic fails almost entirely at the roundabout: once the gap check first succeeds (evaluated from 60 m before the yield line on vehicles expected within a 9 s window) it switches to the traversal state, which has no edge back to re-check. Removing that single re-check edge from FSM-full gives 149/300, so the gap is caused by one missing transition, not by FSMs as such; this omission was in my first "full" FSM and was found through the collisions. The BT needed no extra wiring because the gap condition is re-evaluated on every tick. **With all edges added, the FSM is as safe as the BT** (298 vs. 296 successes here), so the BT advantage measured is structural: 26 nodes against 38 transitions, and 3 nodes against 11 transitions to add one global-priority behaviour (Table 11). The BT aborted an overtake in 20 scenarios. Its 4 failures (2 lead, 2 oncoming) are not yet root-caused (P2).

8.5 E5: Motion planning

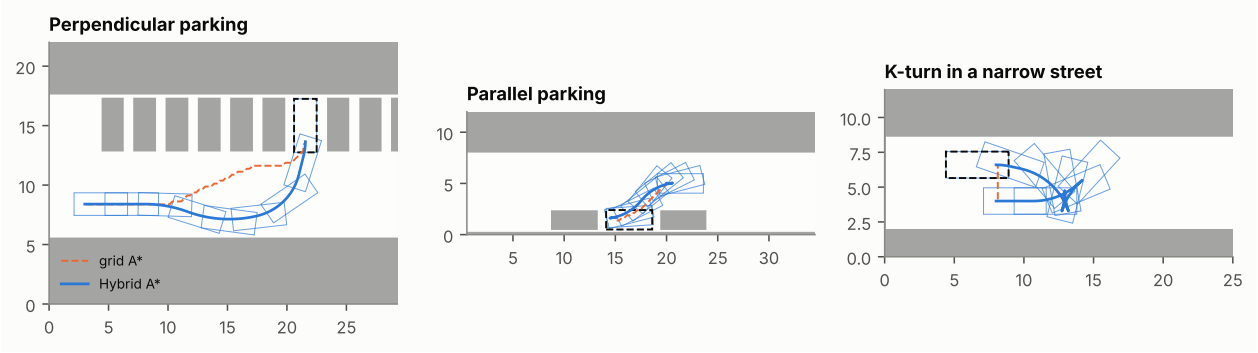


Figure 10: E5: Hybrid A* (blue, footprints every few poses; dashed black = goal pose) and grid A* (orange) from the nominal start poses.

Table 12: E5: tight-space planning, 6 start poses per scenario. Feasible = collision-free (exact footprint), curvature $\leq 0.212\,\mathrm{m}^{-1}$, goal pose reached. Means over found paths.

Scenario	Planner	Feasible	Time [s]	Cusps	Max $\lvert\kappa\rvert$ [1/m]	Collision frac.	Goal head. err [deg]
Perpendicular parking	Hybrid A*	6/6	0.9	0.0	0.21	0	<6
	Grid A*	0/6	–	–	2.60	0.21	45
	Grid A* + Chaikin	0/6	–	–	3.96	0.23	45
Parallel parking	Hybrid A*	4/6	13.7	3.0	0.21	0	<6
	Grid A*	0/6	–	–	2.17	0.66	172
	Grid A* + Chaikin	0/6	–	–	3.30	0.67	172
K-turn in a narrow street	Hybrid A*	6/6	9.3	4.0	0.21	0	<6
	Grid A*	0/6	–	–	1.74	0.57	98
	Grid A* + Chaikin	0/6	–	–	2.64	0.58	98

Hybrid A* found a feasible path in 6/6 (perpendicular), 4/6 (parallel) and 6/6 (K-turn) trials. In parallel parking 2 of 6 starts were not solved within the 40 s / 60 000-expansion budget; solved cases took 13.7 s on average (pure Python, not real-time) and ended about 15 cm from the nominal goal (accepted tolerance 30 cm, 5°; an exact Dubins connection is used when one exists). Grid A* produced paths in every trial but none was feasible: curvature of 1.7–4.0 m^{-1} against the 0.212 m^{-1} limit, 21–67% of poses colliding when checked with the true footprint, and goal-heading errors of 45–172°. This baseline has no heading in its state, so these failures are expected by construction; the comparison shows what the extra state buys.

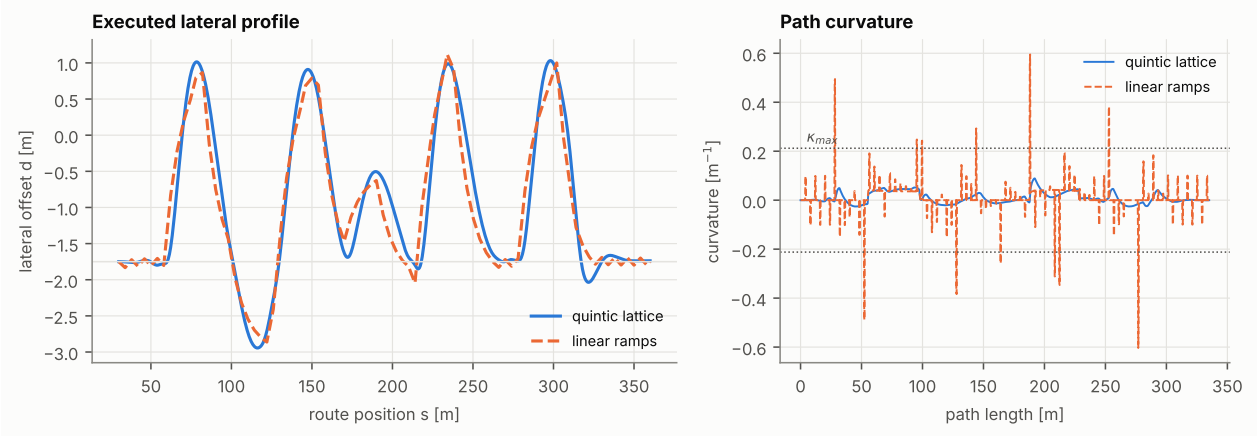


Figure 11: E5: executed lateral profile and curvature on the route with 7 obstacles (5 vehicles, 2 small objects); dotted = vehicle curvature limit.

Table 13: E5: receding-horizon lateral planning on 20 random obstacle layouts. OK = collision-free and curvature within the vehicle limit.

Profile	OK	Collision-free	Max $\lvert\kappa\rvert$ [1/m]	Max $\lvert d\kappa/dl\rvert$ [1/m^2]	Replans w/o candidate
quintic	16/20	16/20	0.075±0.015	0.07±0.00	45
linear	0/20	16/20	0.674±0.251	2.61±0.97	66

Curvature rate depends on the 0.25 m sampling of the executed path for the kinked baseline.

Both lateral profiles use the same candidates and collision test and are collision-free in the same layouts, but only the quintic lattice stays within the curvature limit: 16/20 OK layouts against 0/20, peak curvature 0.075 vs. 0.67 m^{-1} (limit 0.212; Fig. 11). In the 4 layouts that fail for both, some replans had no feasible candidate; whether those layouts admit any collision-free path was not checked (P7).

8.6 E6: Hybrid A* slot length and search speed

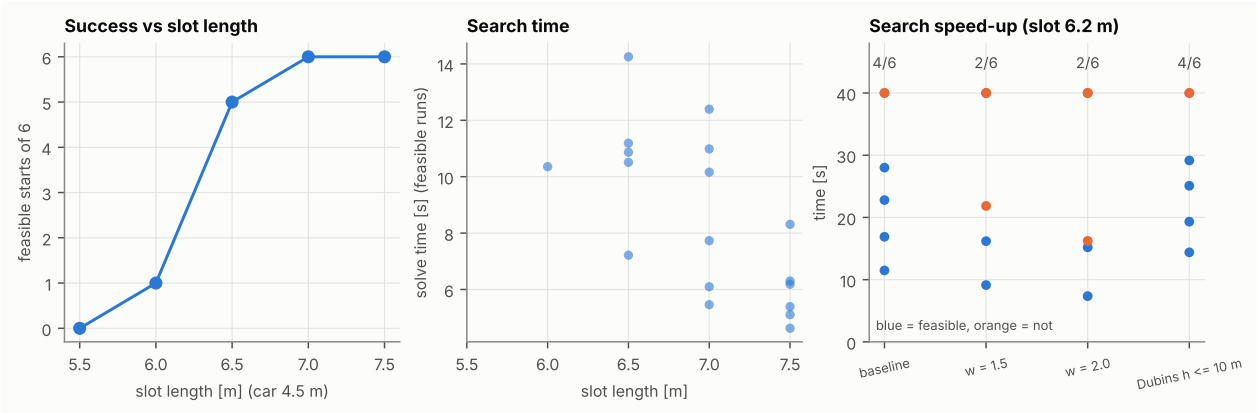


Figure 12: E6. Left: feasible starts vs. slot length. Centre: solve time of feasible runs. Right: search variants on the 6.2 m slot (run time capped at 40 s).

Table 14: E6: Hybrid A* parallel parking vs. slot length (car 4.5 m, 6 start poses per slot, 40 s / 60 000-expansion budget). Time, expansions, cusps and length are means over feasible runs.

Slot [m]	Feasible	Median time [s]	Expansions	Cusps	Length [m]
5.5	0/6	–	–	–	–
6.0	1/6	10.4	12372	2.0	8.4
6.5	5/6	10.9	12637	3.4	10.6
7.0	6/6	8.9	8743	2.2	10.0
7.5	6/6	5.8	6836	2.0	10.3

Table 15: E6: search-speed variants on the nominal 6.2 m slot (same 6 start poses, run back to back on the same machine).

Variant	Feasible	Median time [s]	Expansions	Length [m]
Baseline ($w = 1$)	4/6	19.8	14376	9.6
Weighted A*, $w = 1.5$	2/6	12.7	8520	8.9
Weighted A*, $w = 2$	2/6	11.3	7504	9.1
Dubins term only within 10 m	4/6	22.2	17949	9.4

Feasible starts are 0/6, 1/6, 5/6, 6/6 and 6/6 for slots of 5.5, 6.0, 6.5, 7.0 and 7.5 m (Table 14), and solve time falls as the slot widens. The threshold, between 6.0 and 7.0 m for a 4.5 m car, belongs to this planner (primitive set, 0.5 m/5° grid, 8 cm margin, 40 s/60 000-expansion budget); whether the 5.5–6.0 m failures are geometric limits or budget limits was not tested. Weighted A* ($w = 1.5$ and 2) removes about 40–50% of the expansions and lowers the median time from 19.8 to 12.7 and 11.3 s, but solves only 2/6 and 2/6 starts against 4/6 (Table 15). Restricting the Dubins heuristic term to 10 m made the search *slower* (17949 against 14376 expansions) at equal success. **No variant gave a speed-up without losing solutions**; the compiled port (P5) is still open. Wall-clock times vary between runs on this shared machine (an earlier run gave a 12 s median for the same, deterministic baseline searches), so expansions are the reliable comparison.

8.7 E7: lattice failure analysis

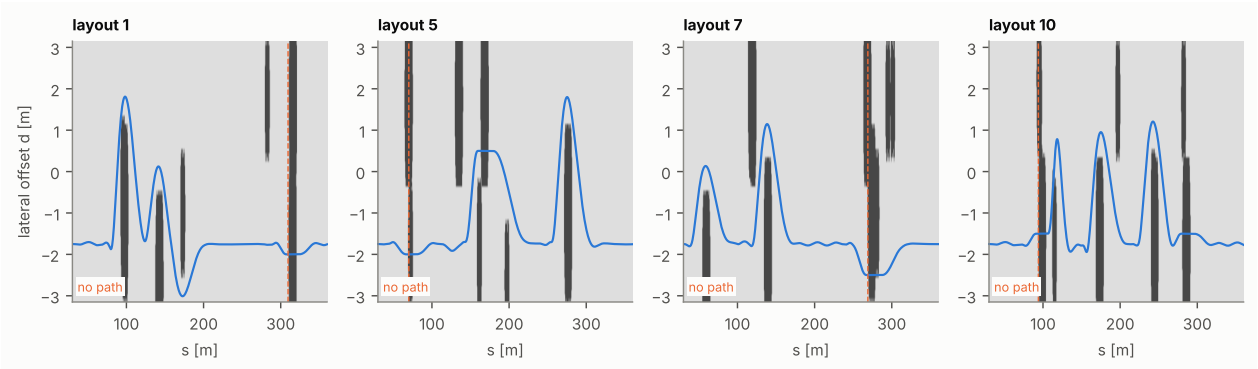


Figure 13: E7: the layouts in which the quintic lattice fails. Dark = cells blocked for the 3-disc body at road heading, light = free; blue = executed path; dashed orange = first collision.

Table 16: E7: the layouts in which the quintic lattice fails. "Corridor path" = a Frenet-grid search finds any path from the start to $s = 360$ m inside the 6.2 m corridor (lateral slope ≤ 0.4, 3-disc body at road heading), with zero clearance and with the planner's 0.35 m margin; the last three columns re-run the planner with a denser lattice, 2 m replanning, or both.

Layout	Obstacles	Corridor path, 0 m	Corridor path, 0.35 m	Dense	2 m replan	Both
1	14	no	no	no	no	no
5	15	no	no	no	no	no
7	18	no	no	no	no	no
10	16	no	no	no	no	no

In 0 of the 4 failing layouts a Frenet-grid search finds any path inside the corridor, even with zero clearance, and a denser lattice (25 offsets, five horizons), 2 m replanning or both recover 0 of them (Table 16). Fig. 13 shows why: each contains a column of obstacles covering the whole 6.2 m corridor. Under the stated assumptions these are infeasible layouts, not planner faults, which supports the P7 hypothesis for this set. **Caveats:** the search is approximate (road-heading body, lateral slope ≤ 0.4, corridor limits); with the planner's 0.35 m margin it also marks 1 of the 16 passing layouts as unsolvable, because the planner's fallback (keep the previous plan) can pass with less margin; and a wider corridor (oncoming lane) would change the answer.

8.8 E8: 20-seed statistics and LiDAR degradation

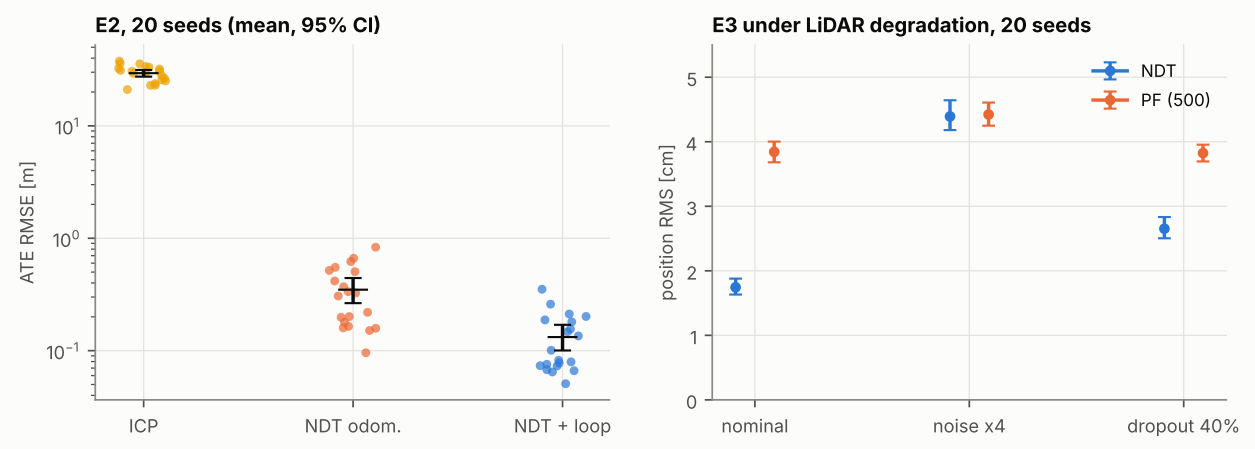


Figure 14: E8. Left: SLAM ATE RMSE per seed (dots) with mean and 95% bootstrap CI. Right: HD-map position RMS under LiDAR degradation, mean and 95% CI.

Table 17: E8: SLAM ATE RMSE [m] over 20 seeds: mean with 95% bootstrap CI of the mean, median and standard deviation.

Method	Mean [95% CI]	Median	Std
ICP odometry	29.4 [27.4, 31.4]	30.14	4.78
NDT odometry	0.35 [0.26, 0.44]	0.32	0.21
NDT + loop closure	0.13 [0.10, 0.17]	0.09	0.08

Table 18: E8: HD-map localization under LiDAR degradation (20 seeds, 15 unmapped vehicles). Position RMS [cm], mean with 95% bootstrap CI; "fail" = maximum error above 1 m.

LiDAR	NDT RMS [cm]	NDT fail	PF RMS [cm]	PF fail
nominal	1.7 [1.6, 1.9]	0/20	3.8 [3.7, 4.0]	0/20
noise ×4	4.4 [4.2, 4.6]	1/20	4.4 [4.2, 4.6]	0/20
dropout 40%	2.7 [2.5, 2.8]	0/20	3.8 [3.7, 4.0]	0/20

With 20 seeds the SLAM ATE (mean [95% CI], m) is 29.4 [27.4, 31.4] for ICP, 0.35 [0.26, 0.44] for NDT odometry and 0.13 [0.10, 0.17] with loop closure; the intervals do not overlap, so the ordering of E2 holds. The 3-seed means of E2 were slightly higher for NDT odometry (0.46 m) than the 20-seed mean. For HD-map localization (15 unmapped vehicles) the nominal position RMS is 1.7 [1.6, 1.9] cm for NDT and 3.8 [3.7, 4.0] cm for the particle filter. Four times the LiDAR noise raises NDT to 4.4 [4.2, 4.6] cm (1/20 runs with a maximum error above 1 m) and the PF to 4.4 [4.2, 4.6] cm; 40% dropout gives 2.7 [2.5, 2.8] and 3.8 [3.7, 4.0] cm. NDT is the more sensitive of the two and loses its accuracy advantage at 4× noise; the PF is nearly insensitive but never better than NDT. Only LiDAR noise and dropout were varied; VO/IMU noise (partly covered in E1) and map age remain open (P3).

8.9 E9: closed-loop integration

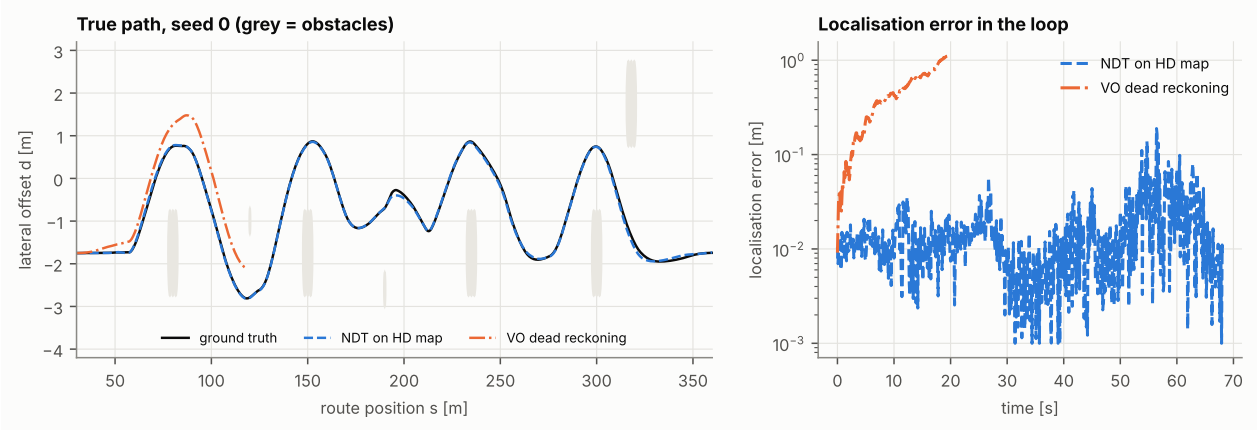


Figure 15: E9, seed 0. Left: true path in Frenet coordinates for the three pose sources (grey = obstacles, stretched by the unequal axes). Right: localization error in the loop.

Table 19: E9: closed loop (pose source → behaviour tree → lattice → pure pursuit → bicycle model), 10 runs per pose source, 7 obstacles on a 330 m route. Clearance is the minimum body-to-obstacle gap; cross-track is the distance of the true position to the current plan.

Pose source	Success	Collisions	Min clearance [m]	Loc. RMS [cm]	Cross-track RMS [m]	Replans w/o candidate
Ground-truth pose	10/10	0	0.50±0.00	0.0±0.0	0.16±0.00	7.0±0.0
NDT on HD map	10/10	0	0.48±0.02	3.1±0.7	0.15±0.00	5.5±1.7
VO dead reckoning	0/10	10	-0.08±0.05	61.5±30.6	0.52±0.25	1.5±1.5

With ground-truth pose the stack completes 10/10 runs. NDT on the HD map matches it (10/10) at 3.1 cm RMS localization error, with essentially the same cross-track error (0.15 m RMS) and nearly the same clearance (0.48 vs. 0.50 m): at this accuracy localization is not the limiting factor, the tracker is. VO dead reckoning succeeds in 0/10: its error reaches about 1 m (RMS 0.62 m) before the first obstacle and every run ends in a collision (Fig. 15), consistent with the P4 hypothesis. **Caveats:** perfect obstacle perception, one layout and route, simulated VO, a kinematic plant without tyre slip or actuator delay, a three-behaviour BT, and no VIO EKF in the loop (the E1 filter runs offline); the dead-reckoning baseline stands in for a VIO outage and is not the E1 filter.

9 Discussion

Two accuracy layers. VIO keeps the vehicle within about a metre through a 130 m outage (E1), whereas map-based localization reaches centimetres (E3). The first provides *continuity* when external references are missing, the second *accuracy* when a map is available; in a full stack the VIO output is the prior that NDT or the particle filter refines. The tunnel is the hard case for every module, for different reasons: the dead-reckoning error accumulates (E1; in the simulator VO noise is deliberately inflated there), the NDT Hessian is poorly conditioned (E2), and the map-based error peaks (E3). The Hessian ratio ρ is cheap to compute and could gate or down-weight scan-matching updates in the filter; this was not tested.

Loop closure buys consistency, not local accuracy. NDT odometry is already sub-metre over 846 m; the pose graph reduces ATE from 0.46 to 0.15 m and, more usefully, removes the step introduced near the tunnel exit, giving a map error of 0.077 m. The scan-to-scan ICP baseline, in contrast, is unusable without a map-based correction.

BT versus FSM. The measured difference between a BT and a *complete* FSM is not safety but specification effort. The first FSM-full omitted one edge (roundabout re-check) and lost about

half of the scenarios; the BT does not need that edge because every condition is re-evaluated on each tick. Complete FSMs close the gap but need 38 transitions instead of 26 nodes, and adding a global-priority behaviour costs 11 transitions against 3 nodes. A different FSM designer could narrow or widen this gap, so the claim is about how failure arises (a missing edge) rather than a universal ranking. That the BT still failed 4 of 300 scenarios shows that tree structure alone does not guarantee safety.

Feasibility is a property of the state space. Grid A* optimizes length over positions only, so it can neither respect $\kappa_{\max}$ nor the footprint; Hybrid A* pays seconds of search time in Python (and failed on two of six parallel-parking starts within budget) for paths a car can drive. The quintic lattice trades a little lateral agility for a curvature rate about 35 times lower than the linear-ramp baseline (0.073 vs. $2.6\,\mathrm{m}^{-2}$ in E5).

Error budget (E8, E9). NDT's centimetre-level error is about an order of magnitude below the planner's 0.35 m margin and leaves the closed loop unchanged, whereas VO dead reckoning exceeds the margin within tens of metres. The 20-seed intervals show that the method ranking of E2 is not a small-sample artefact, and that NDT, not the particle filter, is the component whose accuracy degrades with LiDAR quality.

Negative results. Neither cheap speed-up of Hybrid A* (weighted heuristic, shorter Dubins range) improved time-to-solution without losing solutions, and the failing lattice layouts are blocked rather than hard (E6, E7).

Implication for integration. The filter's covariance was conservative (E1) and NDT/PF errors are small relative to lane width, but global localization is the weak link (E3); a deployed system would need place recognition or a GNSS-seeded prior for initialization and recovery.

10 Real-world applications

- **Tunnels, urban canyons, parking structures:** inertial/visual dead reckoning bridges GNSS gaps (E1); production systems add wheel odometry and map constraints.
- **Map-based localization:** point-cloud NDT scan matching against a pre-built map is used in open-source stacks such as Autoware [17]; a particle filter adds multi-hypothesis recovery at the cost of accuracy (E3).
- **Map building and updating:** LiDAR SLAM with loop closure is the core of survey-vehicle and fleet mapping pipelines (E2).
- **Tactical decision making:** BTs keep priorities explicit and let behaviours be added as subtrees; they are widely used in robotics, e.g. in the ROS 2 navigation stack [22] (E4).
- **Automated valet parking, depots, K-turns:** Hybrid A* free-space planning, as in open-space planners of Apollo [11] and Autoware [17] (E5); Frenet lattices serve on-road lane changes and obstacle nudging.

11 Limitations and threats to validity

- **Simulation only.** 2-D, one synthetic world; no 3-D LiDAR, motion distortion, weather, rolling shutter. The simulator's error models define the results; centimetre figures are not claims about real vehicles.
- **VIO is not visual.** No feature tracking or IMU–camera calibration; VO is simulated and the IMU model is planar.
- **Optimistic HD map.** Built with near-true poses from the same geometry; the only map change tested is unmapped vehicles; NDT is seeded with simulated VO.
- **Baselines.** ICP is untuned; the FSMs are my own designs and the BT–FSM comparison depends on which edges exist; E4 uses point-mass dynamics with perfect tracking.
- **Small samples, no significance tests.** E2 and E3 use 3 seeds, E5 parking 6 starts; standard deviations are indicative only.

- **Extended experiments.** E6 uses 6 starts per slot and machine-dependent timings; E7's solvability test is approximate; E8 varies only LiDAR noise and dropout; E9 has perfect perception, one layout, a kinematic plant and no VIO filter in the loop.
- **Unexplained results.** The 4 BT failures, the NDT step near the tunnel exit, the ICP heading bias and the 4 lattice layouts that fail were not root-caused.
- **Speed and integration.** Hybrid A* takes seconds in Python; the LOAM-style feature extractor is implemented but not evaluated; modules are not integrated in closed loop.

12 Conclusions

All five objectives were implemented and measured in simulation, followed by four extended experiments. **O1:** VIO keeps tunnel-exit error at 1.33 m (20 seeds), about 2× better than GPS+IMU without vision and over 25× better than the IMU alone. **O2:** NDT SLAM with loop closure reaches 0.15 m ATE over 846 m with a 0.077 m mean map error; scan-to-scan ICP does not. **O3:** NDT and PF localize to 1.4–3.4 cm (longitudinal RMS) with 15 unmapped vehicles, but global localization succeeded in only 1 of 3 seeds. **O4:** the BT completed 296/300 scenarios; a complete FSM 298/300; the BT's advantage is its compact, re-evaluated structure, not higher safety. **O5:** Hybrid A* produced feasible paths in 16 of 18 trials and the quintic lattice in 16/20 layouts, while grid-based paths were never feasible. **Extended runs:** Hybrid A* needs a parallel slot of about 6.5–7 m here (E6) and no tested search shortcut preserves its success rate; the lattice's 4 failures are blocked layouts (E7); the SLAM and HD-map conclusions hold over 20 seeds (E8); and with NDT localization the full stack completes 10/10 closed-loop runs against 0/10 for dead reckoning (E9). These conclusions hold for the simulator and parameter choices documented here; the next step is closed-loop integration and validation on real data (Section 13).

13 Future work

13.1 Planned experiments and status

Table 20: Planned experiments and status. Hypotheses are expectations, not findings; results of the experiments that were run are in the cited sections.

ID	Topic	Experiment	Hypothesis	Status
P1	LOAM features	Edge/planar-feature scan-to-map odometry (`extract_loam_features`) vs NDT	Faster per scan, worse in the tunnel	not run
P2	Root causes	Inspect the 4 BT failures; ICP rotation bias vs. turn rate; NDT step at tunnel exit	Unknown for BT; ICP bias grows with turn rate	not run
P3	Sensitivity, statistics	LiDAR noise/dropout; VO/IMU noise; map age; 20 seeds with bootstrap CIs	Graceful degradation, then a cliff	partly run (E8): E2/E3, LiDAR noise and dropout
P4	Integration	Pose → BT → lattice → tracker, closed loop	Localization error small against lane margins	partly run (E9): NDT yes, VIO filter not in loop
P5	Hybrid A*	Slot-length sweep, search speed-up, goal refinement, compiled port	Threshold near the geometric minimum slot	partly run (E6): sweep and heuristic variants; no compiled port
P6	Global localization	More particles, heading prior, place recognition; 20 seeds	Place-recognition seeding fixes most failures	not run
P7	Lattice	Solvability of the 4 failing layouts; speed dimension; moving obstacles	Failures are infeasible layouts	partly run (E7): solvability only
P8	Real data	NDT/SLAM on a public LiDAR odometry dataset (e.g. KITTI [13])	Same ranking ICP < NDT < NDT+loop	not run

13.2 Experiment log (extend this table)

Add one `\logrow` line per run (format as below), then recompile.

Table 21: Experiment log.

Date	ID	Config / seeds	Key result	Observation
Nov 2026	E1	20 seeds	VIO tunnel exit 1.33 m	metre-level; VO simulated
Nov 2026	E2	3 seeds	NDT+loop ATE 0.15 m	ICP drifts; tunnel ρ 9× lower
Nov 2026	E3	3 seeds × 3 levels	NDT 1.4/0.8 cm	global localization 1/3
Nov 2026	E4	300 scenarios	BT 296, FSM-full 298	one missing edge: 149
Nov 2026	E5	3 maps, 20 layouts	Hybrid A* 6/6, 4/6, 6/6	lattice 16/20 vs linear 0/20
Nov 2026	E6	5 slots × 6 starts; 4 variants	feasible 0/6/1/6/5/6/6/6/6/6	no speed-up without loss
Nov 2026	E7	20 layouts, 4 failing	0 of 4 solvable	blocked corridors
Nov 2026	E8	20 seeds; 3 LiDAR settings	NDT+loop 0.13 [0.10, 0.17] m	ranking holds; NDT sensitive to noise
Nov 2026	E9	3 pose sources × 10	NDT 10/10, VO 0/10	localization not limiting at 3 cm

13.3 Research ideas beyond this week

Estimation and mapping

- **I1. Real visual front end:** KLT/ORB tracking with an MSCKF or sliding-window back end and IMU preintegration [24, 12]; validate on EuRoC [3].
- **I2. Tightly coupled LiDAR-inertial odometry** with IMU de-skewing (LIO-SAM/FAST-LIO2 style [33, 38]) and 3-D LOAM features [39].
- **I3. Degeneracy-aware fusion:** feed ρ or the Hessian eigenvectors into the EKF covariance, constrain only the well-observed directions [40].
- **I4. Online calibration:** IMU–camera extrinsics and time offset; observability analysis for planar vehicle motion; wheel odometry and non-holonomic constraints.
- **I5. Robust loop closure:** Scan Context place recognition [18], switchable constraints or max-mixtures [35], injected false-loop tests.
- **I6. Consistency analysis:** NEES/NIS tests for the EKF, covariance recovery from the pose graph.
- **I7. Map maintenance:** change detection, long-term map updating, Lanelet2/OpenDRIVE export [27].
- **I8. Global localization and recovery:** multi-hypothesis NDT, GNSS-seeded priors, kidnapped-robot tests; integrity monitoring and protection levels.

Decision making

- **I9. Closed-loop BT in CARLA** [9] or ROS 2 with OpenSCENARIO scenarios; compare BT, hierarchical FSM and a learned policy; scenario coverage metrics.
- **I10. Formal verification** of BTs (model checking, runtime monitors) to turn the observed "missing edge" failure into a checkable property.
- **I11. Behaviour planning under uncertainty:** POMDP or multi-policy planning at the leaf level; learned condition nodes with calibrated confidence.

Motion planning and control

- **I12. Hybrid A* acceleration:** compiled implementation, Reeds–Shepp expansions [29], learned or lookup heuristics, anytime search.
- **I13. Trajectory refinement:** optimization-based smoothing and collision avoidance [42] on top of the Hybrid A* path; slot-length sweep for parallel parking.
- **I14. Space-time lattice:** speed and time dimension, moving obstacles, jerk limits, comparison with RRT* and optimization-based planners.
- **I15. Tracking control:** pure pursuit/Stanley/MPC on a dynamic bicycle model with tyre slip, closed-loop with the planners.

System and validation

- **I16. Closed-loop integration** of all modules with latency budget and fault injection (sensor dropout, delayed GPS, map mismatch).
- **I17. Public data:** KITTI [13], nuScenes [4], Newer College or Oxford RobotCar for SLAM/localization; ATE/RPE protocol [41].
- **I18. Statistical protocol:** more seeds, bootstrap confidence intervals, paired comparisons, pre-registered hypotheses.
- **I19. Embedded deployment:** real-time profiling on an automotive-class computer; C++/ROS 2 port of the modules that exceed their time budget.
- **I20. Safety argument:** SOTIF-style hazard analysis for the failure modes found here (global-localization failure, missed roundabout re-check, unsolved parking).

14 Reproducibility and folder structure

```
sdc_project/
|-- main.tex, main.pdf         this report (pdfLaTeX, run twice)
|-- numbers.tex, tables/       generated by make_tables.py from results/*.json
|-- make_tables.py, run_all.py, requirements.txt, README.md
|-- FUTURE_IDEAS.md           research ideas (same list as Sec. Future work)
|-- sdc/                       library
|   |-- geometry.py
|   |-- sim/             world.py, drive.py
|   |-- localization/    vio.py, ndt.py, icp.py, pose_graph.py, lidar_slam.py,
|   |                    hd_map.py, particle_filter.py
|   |-- behavior/        bt.py, scenario.py
|   `-- planning/        dubins.py, hybrid_astar.py, scenarios.py, lattice.py
|-- experiments/               common.py, exp1_vio.py ... exp9_closed_loop.py
|-- tests/                     test_core.py (6 unit tests)
`-- results/                   exp*.json (raw numbers), figures/*.pdf|png
```

```
pip install -r requirements.txt     # numpy, scipy, matplotlib, pytest
python run_all.py                   # unit tests, E1-E9, tables
python make_tables.py               # after re-running any single experiment
pdflatex main.tex && pdflatex main.tex
```

All experiments use fixed seeds; run time depends on the CPU and the number of cores (the experiment scripts use two worker processes). Figures are regenerated into `results/figures/`; the report reads numbers only through `numbers.tex` and `tables/`, so the text cannot drift from the data.

References

[1] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 14, no. 2, 1992.

[2] P. Biber and W. Strasser, "The normal distributions transform: A new approach to laser scan matching," in *Proc. IEEE/RSJ IROS*, 2003.

[3] M. Burri *et al.*, "The EuRoC micro aerial vehicle datasets," *Int. J. Robotics Research*, vol. 35, no. 10, 2016.

[4] H. Caesar *et al.*, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF CVPR*, 2020.

[5] C. Campos *et al.*, "ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM," *IEEE Trans. Robotics*, vol. 37, no. 6, 2021.

[6] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.

[7] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte Carlo localization for mobile robots," in *Proc. IEEE ICRA*, 1999.

[8] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *Int. J. Robotics Research*, vol. 29, no. 5, 2010.

[9] A. Dosovitskiy *et al.*, "CARLA: An open urban driving simulator," in *Proc. Conf. Robot Learning (CoRL)*, 2017.

[10] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," *Amer. J. Math.*, vol. 79, no. 3, 1957.

[11] H. Fan *et al.*, "Baidu Apollo EM motion planner," arXiv:1807.08048, 2018.

[12] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual–inertial odometry," *IEEE Trans. Robotics*, vol. 33, no. 1, 2017.

[13] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. IEEE CVPR*, 2012.

[14] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intell. Transp. Syst. Mag.*, vol. 2, no. 4, 2010.

[15] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Statist.*, vol. 35, no. 1, 1964.

[16] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," *Robotics and Autonomous Systems*, vol. 154, 2022.

[17] S. Kato *et al.*, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proc. ACM/IEEE ICCPS*, 2018.

[18] G. Kim and A. Kim, "Scan Context: Egocentric spatial descriptor for place recognition within 3D point cloud map," in *Proc. IEEE/RSJ IROS*, 2018.

[19] K. Koide, J. Miura, and E. Menegatti, "A portable three-dimensional LIDAR-based system for long-term and wide-area people behavior measurement," *Int. J. Advanced Robotic Systems*, vol. 16, no. 2, 2019.

[20] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: A general framework for graph optimization," in *Proc. IEEE ICRA*, 2011.

[21] S. Leutenegger *et al.*, "Keyframe-based visual–inertial odometry using nonlinear optimization," *Int. J. Robotics Research*, vol. 34, no. 3, 2015.

[22] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.

[23] M. Magnusson, "The three-dimensional normal-distributions transform," Ph.D. dissertation, Örebro University, 2009.

[24] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint Kalman filter for vision-aided inertial navigation," in *Proc. IEEE ICRA*, 2007.

[25] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intelligent Vehicles*, vol. 1, no. 1, 2016.

[26] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *J. Field Robotics*, vol. 26, no. 3, 2009.

[27] F. Poggenhans *et al.*, "Lanelet2: A high-definition map framework for the future of automated driving," in *Proc. IEEE ITSC*, 2018.

[28] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robotics*, vol. 34, no. 4, 2018.

[29] J. A. Reeds and L. A. Shepp, "Optimal paths for a car that goes both forwards and backwards," *Pacific J. Math.*, vol. 145, no. 2, 1990.

[30] S. I. Roumeliotis and J. W. Burdick, "Stochastic cloning: A generalized framework for processing relative state measurements," in *Proc. IEEE ICRA*, 2002.

[31] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Robotics: Science and Systems (RSS)*, 2009.

[32] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *Proc. IEEE/RSJ IROS*, 2018.

[33] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *Proc. IEEE/RSJ IROS*, 2020.

[34] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ IROS*, 2012.

[35] N. Sünderhauf and P. Protzel, "Switchable constraints for robust pose graph SLAM," in *Proc. IEEE/RSJ IROS*, 2012.

[36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[37] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a Frenét frame," in *Proc. IEEE ICRA*, 2010.

[38] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robotics*, vol. 38, no. 4, 2022.

[39] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," in *Robotics: Science and Systems (RSS)*, 2014.

[40] J. Zhang, M. Kaess, and S. Singh, "On degeneracy of optimization-based state estimation problems," in *Proc. IEEE ICRA*, 2016.

[41] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry," in *Proc. IEEE/RSJ IROS*, 2018.

[42] X. Zhang, A. Liniger, and F. Borrelli, "Optimization-based collision avoidance," *IEEE Trans. Control Systems Technology*, vol. 29, no. 3, 2021.

[43] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E*, vol. 62, no. 2, 2000.

[44] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, 1968.

[45] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[46] G. M. Chaikin, "An algorithm for high-speed curve generation," *Computer Graphics and Image Processing*, vol. 3, no. 4, 1974.

A Source code

Complete listings of all Python sources (library, experiments, tests, report generator).

`sdc/geometry.py`

```
"""Planar (SE(2)) geometry helpers used throughout the project.

Poses are numpy arrays ``[x, y, theta]``.  ``compose(a, b)`` is the group
product  ``a (+) b``  (pose ``b`` expressed in frame ``a`` -> world frame).
"""
from __future__ import annotations

import numpy as np


def wrap(a):
    """Wrap angle(s) to (-pi, pi]."""
    return (np.asarray(a) + np.pi) % (2.0 * np.pi) - np.pi


def rot(theta: float) -> np.ndarray:
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, -s], [s, c]])


def compose(a, b) -> np.ndarray:
    """a (+) b."""
    c, s = np.cos(a[2]), np.sin(a[2])
    return np.array([a[0] + c * b[0] - s * b[1],
                     a[1] + s * b[0] + c * b[1],
                     wrap(a[2] + b[2])])


def inverse(a) -> np.ndarray:
    c, s = np.cos(a[2]), np.sin(a[2])
    return np.array([-(c * a[0] + s * a[1]),
                     s * a[0] - c * a[1],
                     wrap(-a[2])])


def between(a, b) -> np.ndarray:
    """Pose of ``b`` expressed in the frame of ``a``  (a^-1 (+) b)."""
    return compose(inverse(a), b)


def transform_points(pose, pts: np.ndarray) -> np.ndarray:
    """Map (N,2) points from the frame of ``pose`` into the parent frame."""
    c, s = np.cos(pose[2]), np.sin(pose[2])
    out = np.empty_like(pts, dtype=float)
    out[:, 0] = c * pts[:, 0] - s * pts[:, 1] + pose[0]
    out[:, 1] = s * pts[:, 0] + c * pts[:, 1] + pose[1]
    return out


def voxel_downsample(pts: np.ndarray, res: float) -> np.ndarray:
    """Keep one (averaged) point per voxel of size ``res``."""
```

```
    if len(pts) == 0:
        return pts
    ij = np.floor(pts / res).astype(np.int64)
    key = ij[:, 0] * 1_000_003 + ij[:, 1]
    _, inv, cnt = np.unique(key, return_inverse=True, return_counts=True)
    sx = np.bincount(inv, pts[:, 0])
    sy = np.bincount(inv, pts[:, 1])
    return np.stack([sx / cnt, sy / cnt], axis=1)


def pose_errors(est: np.ndarray, gt: np.ndarray):
    """Longitudinal / lateral / heading error of est w.r.t. gt, in the gt frame."""
    d = est[:, :2] - gt[:, :2]
    c, s = np.cos(gt[:, 2]), np.sin(gt[:, 2])
    lon = c * d[:, 0] + s * d[:, 1]
    lat = -s * d[:, 0] + c * d[:, 1]
    yaw = wrap(est[:, 2] - gt[:, 2])
    return lon, lat, yaw


def rmse(x) -> float:
    x = np.asarray(x)
    return float(np.sqrt(np.mean(x ** 2)))
```

sdc/sim/world.py

```
"""Synthetic urban environment: a closed rounded-rectangle road loop with
buildings, poles, trees, a curved tunnel (GPS denied, low-texture) and an
underpass (GPS denied).  Everything is planar, matching the 2-D LiDAR model.
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np


# ---------------------------------------------------------------------------
# Route (road centre line)
# ---------------------------------------------------------------------------
def _line(p0, p1, ds):
    p0, p1 = np.asarray(p0, float), np.asarray(p1, float)
    n = max(2, int(np.hypot(*(p1 - p0)) / ds) + 1)
    return p0 + np.linspace(0, 1, n)[:, None] * (p1 - p0)


def _arc(cx, cy, r, a0, a1, ds):
    n = max(2, int(abs(a1 - a0) * r / ds) + 1)
    a = np.linspace(a0, a1, n)
    return np.stack([cx + r * np.cos(a), cy + r * np.sin(a)], axis=1)


@dataclass
class Route:
    s: np.ndarray       # arclength samples
    xy: np.ndarray      # (N,2)
    theta: np.ndarray   # unwrapped heading
    kappa: np.ndarray   # curvature
    length: float

    def at(self, s):
        """Periodic evaluation: returns xy (...,2), theta, kappa."""
        s = np.mod(s, self.length)
        x = np.interp(s, self.s, self.xy[:, 0])
        y = np.interp(s, self.s, self.xy[:, 1])
        th = np.interp(s, self.s, self.theta)
        k = np.interp(s, self.s, self.kappa)
        return np.stack([x, y], axis=-1), th, k


def build_route(W=220.0, H=140.0, r=25.0, ds=0.1) -> Route:
    """Counter-clockwise rounded rectangle starting at the middle of the bottom edge."""
    pieces = [
        _line((W / 2, 0), (W - r, 0), ds),
        _arc(W - r, r, r, -np.pi / 2, 0.0, ds),
        _line((W, r), (W, H - r), ds),
        _arc(W - r, H - r, r, 0.0, np.pi / 2, ds),
        _line((W - r, H), (r, H), ds),
        _arc(r, H - r, r, np.pi / 2, np.pi, ds),
        _line((0, H - r), (0, r), ds),
        _arc(r, r, r, np.pi, 1.5 * np.pi, ds),
        _line((r, 0), (W / 2, 0), ds),
    ]
    pts = [pieces[0]] + [p[1:] for p in pieces[1:]]
    pts = np.concatenate(pts, axis=0)
    seg = np.hypot(*np.diff(pts, axis=0).T)
    s_raw = np.concatenate([[0.0], np.cumsum(seg)])
    length = s_raw[-1]
    s = np.arange(0.0, length, ds)
    xy = np.stack([np.interp(s, s_raw, pts[:, 0]), np.interp(s, s_raw, pts[:, 1])], axis=1)
    d = np.gradient(xy, ds, axis=0)
    theta = np.unwrap(np.arctan2(d[:, 1], d[:, 0]))
```

```python
    # smooth curvature estimate
    kappa = np.gradient(theta, ds)
    k = 25
    kappa = np.convolve(np.pad(kappa, k, mode="edge"), np.ones(2 * k + 1) / (2 * k + 1), mode="valid")
    # close the loop explicitly so that interpolation is continuous across s = L
    s = np.append(s, length)
    xy = np.vstack([xy, xy[0]])
    theta = np.append(theta, theta[0] + 2.0 * np.pi)
    kappa = np.append(kappa, kappa[0])
    return Route(s, xy, theta, kappa, float(length))


# ---------------------------------------------------------------------------
# World
# ---------------------------------------------------------------------------
def _rect_segments(corners):
    c = np.asarray(corners)
    return np.concatenate([c, np.roll(c, -1, axis=0)], axis=1)  # (4,4): x1 y1 x2 y2


class World:
    """Static map + metadata (tunnel / GPS-denied intervals)."""

    W, H, R = 220.0, 140.0, 25.0

    def __init__(self, seed: int = 7):
        self.rng = np.random.default_rng(seed)
        self.route = build_route(self.W, self.H, self.R)
        L = self.route.length
        self.length = L
        self.tunnel = (200.0, 330.0)
        self.underpass = (500.0, 525.0)
        half = self.W / 2 - self.R           # length of the first/last half of the bottom edge
        arc = np.pi * self.R / 2
        vert, horiz = self.H - 2 * self.R, self.W - 2 * self.R
        # s-ranges of the straight road pieces and of the corners
        s0 = half + arc
        self.straights, self.corners = [], [(half, half + arc)]
        for length_ in (vert, horiz, vert):
            self.straights.append((s0, s0 + length_))
            self.corners.append((s0 + length_, s0 + length_ + arc))
            s0 += length_ + arc
        self.straights.append((s0, s0 + half + half))       # bottom edge, wraps through s = L

        segs, circs = [], []
        self._add_buildings(segs)
        self._add_clutter(circs)
        self._add_tunnel(segs)
        self._add_underpass(circs)
        self.segments = np.concatenate(segs, axis=0)
        self.circles = np.asarray(circs, float).reshape(-1, 3)

    # -----------------------------------------------------------------
    def _frame(self, s):
        xy, th, _ = self.route.at(s)
        t = np.array([np.cos(th), np.sin(th)])
        n = np.array([-np.sin(th), np.cos(th)])
        return xy, t, n

    def in_interval(self, s, interval, margin=0.0):
        sm = np.mod(s, self.length)
        return (sm >= interval[0] - margin) & (sm <= interval[1] + margin)

    def gps_available(self, s) -> bool:
        return not (self.in_interval(s, self.tunnel) or self.in_interval(s, self.underpass))

    def vision_factor(self, s) -> float:
        """Visual-odometry noise multiplier (low texture / dark in the tunnel)."""
        return 2.5 if self.in_interval(s, self.tunnel) else 1.0

    # -----------------------------------------------------------------
    def _add_buildings(self, segs):
        rng = self.rng
        for (a, b) in self.straights:
            for side in (+1.0, -1.0):
                s = a + 6 + rng.uniform(0, 6)
                while True:
                    w = rng.uniform(12, 28)
                    if s + w > b - 6:
                        break
                    gap = rng.uniform(4, 12)
                    if not (self.in_interval(s, self.tunnel, 12) or self.in_interval(s + w, self.tunnel, 12)):
                        off, depth = rng.uniform(9.5, 12.0), rng.uniform(8, 14)
                        c, t, n = self._frame(s + w / 2)
                        p = [c - w / 2 * t + side * off * n,
                             c + w / 2 * t + side * off * n,
                             c + w / 2 * t + side * (off + depth) * n,
                             c - w / 2 * t + side * (off + depth) * n]
                        segs.append(_rect_segments(p))
                    s += w + gap

    def _add_clutter(self, circs):
        rng = self.rng
        # poles along the road
        s = 3.0
        while s < self.length:
```

```
            if not self.in_interval(s, self.tunnel, 5):
                for side in (+1.0, -1.0):
                    if rng.random() < 0.8:
                        c, t, n = self._frame(s + rng.uniform(-2, 2))
                        p = c + side * rng.uniform(5.0, 6.5) * n
                        circs.append([p[0], p[1], 0.25])
            s += rng.uniform(14, 20)
        # trees in the corners
        for (a, b) in self.corners:
            if self.in_interval(0.5 * (a + b), self.tunnel, 25):
                continue
            s = a + 2
            while s < b:
                for side in (+1.0, -1.0):
                    c, t, n = self._frame(s)
                    p = c + side * rng.uniform(7, 10) * n
                    circs.append([p[0], p[1], rng.uniform(0.8, 1.2)])
                s += rng.uniform(6, 9)

    def _add_tunnel(self, segs, half_width=4.5, step=1.5):
        a, b = self.tunnel
        ss = np.arange(a, b + step, step)
        for side in (+1.0, -1.0):
            pts = []
            for s in ss:
                c, t, n = self._frame(s)
                pts.append(c + side * half_width * n)
            pts = np.asarray(pts)
            segs.append(np.concatenate([pts[:-1], pts[1:]], axis=1))
        # structural ribs (cross-passage frames) every 20 m, alternating sides
        k = 0
        for s in np.arange(a + 10, b - 5, 20.0):
            side = +1.0 if k % 2 == 0 else -1.0
            c, t, n = self._frame(s)
            w = c + side * half_width * n
            p = [w - 0.4 * t, w - 0.4 * t - side * 0.9 * n, w + 0.4 * t - side * 0.9 * n, w + 0.4 * t]
            segs.append(np.concatenate([np.array(p[:-1]), np.array(p[1:])], axis=1))
            k += 1

    def _add_underpass(self, circs):
        a, b = self.underpass
        for s in (a, b):
            c, t, n = self._frame(s)
            for side in (+1.0, -1.0):
                p = c + side * 7.5 * n
                circs.append([p[0], p[1], 0.7])

    # ------------------------------------------------------------------
    def dynamic_objects(self, n: int, rng: np.random.Generator) -> np.ndarray:
        """Unmapped vehicles (parked / oncoming) -> (4n,4) segment array."""
        segs = []
        for _ in range(n):
            for _try in range(20):
                s = rng.uniform(0, self.length)
                if rng.random() < 0.5:
                    lat = -rng.uniform(4.8, 5.6)      # parked on the kerb side
                    if self.in_interval(s, self.tunnel, 3):
                        continue
                else:
                    lat = rng.uniform(1.4, 2.2)       # opposite lane
                c, t, nn = self._frame(s)
                ctr = c + lat * nn
                L, Wd = 4.5, 1.9
                p = [ctr - L / 2 * t - Wd / 2 * nn, ctr + L / 2 * t - Wd / 2 * nn,
                     ctr + L / 2 * t + Wd / 2 * nn, ctr - L / 2 * t + Wd / 2 * nn]
                segs.append(_rect_segments(p))
                break
        return np.concatenate(segs, axis=0) if segs else np.zeros((0, 4))
```

sdc/sim/drive.py

```
"""Ground-truth vehicle trajectory along the route + sensor data generation."""
from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np

from ..geometry import between, wrap
from .world import World


@dataclass
class Drive:
    t: np.ndarray        # (N,) time stamps @ imu rate
    pose: np.ndarray     # (N,3) ground-truth x,y,theta
    v: np.ndarray        # speed
    omega: np.ndarray    # yaw rate
    a_long: np.ndarray   # longitudinal acceleration
    s: np.ndarray        # arclength along the route (monotone, not wrapped)
    dt: float

```

```python

def simulate_drive(world: World, laps: float = 1.25, dt: float = 0.01, seed: int = 0,
                   lane_offset: float = -1.75, wander: float = 0.5, vmax: float = 11.0,
                   a_lat: float = 2.2, a_acc: float = 1.5, a_dec: float = 2.0,
                   v0: float = 2.0) -> Drive:
    """Drive the lane (road centre + lateral offset + gentle weaving) with a speed
    profile limited by lateral acceleration, longitudinal acceleration and the
    tunnel speed limit.  The profile is computed along the *vehicle path* so that
    speed and acceleration are continuous."""
    rng = np.random.default_rng(seed)
    route, L = world.route, world.length
    ds = 0.1
    s_c = np.arange(0.0, laps * L + 60.0, ds)               # road-centre arclength
    xy_c, th_c, _ = route.at(s_c)
    phi = rng.uniform(0, 2 * np.pi)
    d_lat = lane_offset + wander * np.sin(2 * np.pi * s_c / 90.0 + phi)
    n_vec = np.stack([-np.sin(th_c), np.cos(th_c)], axis=1)
    xy_p = xy_c + d_lat[:, None] * n_vec                     # vehicle path
    kw = 40                                                  # +-4 m low-pass: curvature is continuous
    ker_p = np.ones(2 * kw + 1) / (2 * kw + 1)
    xy_p = np.stack([np.convolve(np.pad(xy_p[:, i], kw, mode="edge"), ker_p, mode="valid")
                     for i in range(2)], axis=1)
    seg = np.hypot(*np.diff(xy_p, axis=0).T)
    sp = np.concatenate([[0.0], np.cumsum(seg)])             # path arclength
    d = np.gradient(xy_p, axis=0)
    theta_p = np.unwrap(np.arctan2(d[:, 1], d[:, 0]))
    kap = np.gradient(theta_p, sp)
    k = 25
    kap = np.convolve(np.pad(kap, k, mode="edge"), np.ones(2 * k + 1) / (2 * k + 1), mode="valid")

    v_lim = np.minimum(vmax, np.sqrt(a_lat / (np.abs(kap) + 1e-6)))
    in_tun = world.in_interval(s_c, world.tunnel)
    v_lim = np.where(in_tun, np.minimum(v_lim, 9.0), v_lim)
    v2 = v_lim ** 2
    v2[0] = min(v2[0], v0 ** 2)
    for i in range(1, len(v2)):
        v2[i] = min(v2[i], v2[i - 1] + 2 * a_acc * seg[i - 1])
    for i in range(len(v2) - 2, -1, -1):
        v2[i] = min(v2[i], v2[i + 1] + 2 * a_dec * seg[i])
    v_prof = np.sqrt(v2)

    s_end = laps * L
    sp_list, cur = [0.0], 0.0
    while np.interp(cur, sp, s_c) < s_end:
        cur += np.interp(cur, sp, v_prof) * dt
        sp_list.append(cur)
    sp_t = np.asarray(sp_list)
    t = np.arange(len(sp_t)) * dt
    x = np.interp(sp_t, sp, xy_p[:, 0])
    y = np.interp(sp_t, sp, xy_p[:, 1])
    theta = np.interp(sp_t, sp, theta_p)
    speed = np.interp(sp_t, sp, v_prof)
    kk = 4                                                   # 9-sample moving average
    ker = np.ones(2 * kk + 1) / (2 * kk + 1)
    sm = lambda a: np.convolve(np.pad(a, kk, mode="edge"), ker, mode="valid")
    omega = sm(np.gradient(theta, dt))
    a_long = sm(np.gradient(speed, dt))
    s_arr = np.interp(sp_t, sp, s_c)
    pose = np.column_stack([x, y, wrap(theta)])
    return Drive(t, pose, speed, omega, a_long, s_arr, dt)


# ---------------------------------------------------------------------------
# Sensors
# ---------------------------------------------------------------------------
@dataclass
class ImuParams:
    gyro_noise: float = 0.002      # rad/s   (white, per sample)
    accel_noise: float = 0.05      # m/s^2   (white, per sample)
    gyro_bias0: float = 0.005      # rad/s   (1-sigma initial bias)
    accel_bias0: float = 0.05      # m/s^2
    gyro_bias_rw: float = 1e-4     # rad/s / sqrt(s)
    accel_bias_rw: float = 2e-3    # m/s^2 / sqrt(s)


def simulate_imu(drive: Drive, p: ImuParams, rng: np.random.Generator):
    """Planar IMU: gyro-z and body-frame (x,y) specific force.  Returns (N,3) and true biases."""
    n, dt = len(drive.t), drive.dt
    bg = np.cumsum(np.r_[rng.normal(0, p.gyro_bias0), rng.normal(0, p.gyro_bias_rw * np.sqrt(dt), n - 1)])
    bax = np.cumsum(np.r_[rng.normal(0, p.accel_bias0), rng.normal(0, p.accel_bias_rw * np.sqrt(dt), n - 1)])
    bay = np.cumsum(np.r_[rng.normal(0, p.accel_bias0), rng.normal(0, p.accel_bias_rw * np.sqrt(dt), n - 1)])
    gz = drive.omega + bg + rng.normal(0, p.gyro_noise, n)
    ax = drive.a_long + bax + rng.normal(0, p.accel_noise, n)
    ay = drive.v * drive.omega + bay + rng.normal(0, p.accel_noise, n)
    return np.column_stack([gz, ax, ay]), np.column_stack([bg, bax, bay])


@dataclass
class VoParams:
    """Stereo visual-odometry error model (relative pose between consecutive frames)."""
    sigma_t0: float = 0.010
    sigma_t1: float = 0.010        # proportional to travelled distance
    sigma_r0: float = 0.0015
    sigma_r1: float = 0.002        # proportional to |dtheta|
    scale_err_std: float = 0.004   # systematic scale error (calibration)
    rate_div: int = 10             # camera frame every rate_div IMU samples (10 Hz)
```

```python


@dataclass
class VoMeas:
    idx: np.ndarray    # IMU sample index of the *current* frame
    z: np.ndarray      # (M,3) relative pose [dx,dy,dtheta] in previous body frame
    sigma: np.ndarray  # (M,2) reported (sigma_t, sigma_r)


def simulate_vo(drive: Drive, world: World, p: VoParams, rng: np.random.Generator) -> VoMeas:
    idx = np.arange(p.rate_div, len(drive.t), p.rate_div)
    scale = 1.0 + rng.normal(0, p.scale_err_std)
    zs, sig = [], []
    for i in idx:
        j = i - p.rate_div
        d = between(drive.pose[j], drive.pose[i])
        f = world.vision_factor(drive.s[i])
        dist = np.hypot(d[0], d[1])
        st = f * (p.sigma_t0 + p.sigma_t1 * dist)
        sr = f * (p.sigma_r0 + p.sigma_r1 * abs(d[2]))
        z = np.array([d[0] * scale + rng.normal(0, st), d[1] * scale + rng.normal(0, st),
                      d[2] + rng.normal(0, sr)])
        zs.append(z)
        sig.append([st, sr])
    return VoMeas(idx, np.asarray(zs), np.asarray(sig))


@dataclass
class GpsMeas:
    idx: np.ndarray
    z: np.ndarray       # (M,2)
    sigma: float


def simulate_gps(drive: Drive, world: World, rng: np.random.Generator, sigma: float = 0.6,
                 rate_div: int = 100) -> GpsMeas:
    idx = np.arange(0, len(drive.t), rate_div)
    keep = np.array([world.gps_available(drive.s[i]) for i in idx])
    idx = idx[keep]
    z = drive.pose[idx, :2] + rng.normal(0, sigma, (len(idx), 2))
    return GpsMeas(idx, z, sigma)


# ---------------------------------------------------------------------------
# LiDAR (2-D, 360 beams) with vectorised ray casting
# ---------------------------------------------------------------------------
def raycast(origin, angles, segs, circs, max_range):
    d = np.stack([np.cos(angles), np.sin(angles)], axis=1)     # (B,2)
    best = np.full(len(angles), np.inf)
    if len(segs):
        mid = 0.5 * (segs[:, :2] + segs[:, 2:])
        rad = 0.5 * np.hypot(segs[:, 2] - segs[:, 0], segs[:, 3] - segs[:, 1])
        keep = np.hypot(mid[:, 0] - origin[0], mid[:, 1] - origin[1]) < max_range + rad
        sg = segs[keep]
        if len(sg):
            p = sg[:, :2]
            e = sg[:, 2:] - sg[:, :2]
            w = p - origin[:2]
            denom = d[:, 0:1] * e[None, :, 1] - d[:, 1:2] * e[None, :, 0]
            with np.errstate(divide="ignore", invalid="ignore"):
                t = (w[:, 0] * e[:, 1] - w[:, 1] * e[:, 0])[None, :] / denom
                u = (w[:, 0][None, :] * d[:, 1:2] - w[:, 1][None, :] * d[:, 0:1]) / denom
            ok = (np.abs(denom) > 1e-9) & (t > 0) & (u >= 0) & (u <= 1)
            t = np.where(ok, t, np.inf)
            best = np.minimum(best, t.min(axis=1))
    if len(circs):
        keep = np.hypot(circs[:, 0] - origin[0], circs[:, 1] - origin[1]) < max_range + circs[:, 2]
        cc = circs[keep]
        if len(cc):
            oc = origin[:2] - cc[:, :2]                        # (K,2)
            b = d @ oc.T                                        # (B,K)
            cterm = (oc ** 2).sum(1) - cc[:, 2] ** 2
            disc = b ** 2 - cterm[None, :]
            with np.errstate(invalid="ignore"):
                t = -b - np.sqrt(disc)
            ok = (disc >= 0) & (t > 0)
            t = np.where(ok, t, np.inf)
            best = np.minimum(best, t.min(axis=1))
    best[best > max_range] = np.inf
    return best


@dataclass
class LidarParams:
    n_beams: int = 360
    max_range: float = 40.0
    sigma0: float = 0.02
    sigma1: float = 0.001     # range proportional noise
    dropout: float = 0.02


def lidar_scan(world: World, pose, rng: np.random.Generator, p: LidarParams = LidarParams(),
               extra_segments: np.ndarray | None = None, noise: bool = True) -> np.ndarray:
    """Return the (M,2) point cloud in the sensor (= body) frame."""
    segs = world.segments if extra_segments is None or len(extra_segments) == 0 else \
        np.concatenate([world.segments, extra_segments], axis=0)
```

```
    ang_body = np.linspace(-np.pi, np.pi, p.n_beams, endpoint=False)
    r = raycast(pose, ang_body + pose[2], segs, world.circles, p.max_range)
    ok = np.isfinite(r)
    if noise:
        ok &= rng.random(len(r)) > p.dropout
        r = r + rng.normal(0, 1, len(r)) * (p.sigma0 + p.sigma1 * np.where(ok, r, 0))
    a, r = ang_body[ok], r[ok]
    return np.stack([r * np.cos(a), r * np.sin(a)], axis=1)


def simulate_lidar(drive: Drive, world: World, rng: np.random.Generator, rate_div: int = 10,
                   extra_segments=None, p: LidarParams = LidarParams()):
    """Scans at 10 Hz.  Returns (indices, list of point clouds)."""
    idx = np.arange(0, len(drive.t), rate_div)
    scans = [lidar_scan(world, drive.pose[i], rng, p, extra_segments) for i in idx]
    return idx, scans
```

sdc/localization/vio.py

```
"""Loosely-coupled visual-inertial odometry as an EKF with *stochastic cloning*.

State (8):  p=(x,y) world position, theta heading, v=(vx,vy) world velocity,
            b_g gyro bias, b_a=(bax,bay) body-frame accelerometer bias.
Clone (3):  the pose at the previous camera frame.  A visual-odometry (VO)
            measurement is the *relative* pose between the clone and the
            current pose; keeping the clone in the state keeps the filter
            consistent (the same information is never used twice).

The same class is used for the three estimators compared in the experiments:
    imu_only : propagation only
    gps_imu  : propagation + GPS position updates
    vio      : propagation + GPS + visual-odometry relative-pose updates
"""
from __future__ import annotations

import numpy as np

from ..geometry import wrap
from ..sim.drive import Drive, GpsMeas, ImuParams, VoMeas

NX = 8
IDX_P, IDX_TH, IDX_V, IDX_BG, IDX_BA = slice(0, 2), 2, slice(3, 5), 5, slice(6, 8)


def _R(th):
    c, s = np.cos(th), np.sin(th)
    return np.array([[c, -s], [s, c]])


def _dR(th):
    c, s = np.cos(th), np.sin(th)
    return np.array([[-s, -c], [c, -s]])


class VioEKF:
    def __init__(self, pose0, v0, imu: ImuParams, P0_diag=None, use_vo=True):
        self.x = np.zeros(NX)
        self.x[0:2] = pose0[:2]
        self.x[2] = pose0[2]
        self.x[3:5] = v0
        self.use_vo = use_vo
        n = NX + 3
        self.P = np.zeros((n, n))
        if P0_diag is None:
            P0_diag = [0.6 ** 2, 0.6 ** 2, 0.03 ** 2, 0.3 ** 2, 0.3 ** 2,
                       (1.5 * imu.gyro_bias0) ** 2, (1.5 * imu.accel_bias0) ** 2, (1.5 * imu.accel_bias0) ** 2]
        self.P[:NX, :NX] = np.diag(P0_diag)
        self.imu = imu
        self._clone()
        self.last_nis = np.nan

    # ------------------------------------------------------------------
    def _clone(self):
        """(Re)create the clone of the current pose; marginalises the old one."""
        A = np.zeros((NX + 3, NX))
        A[:NX, :] = np.eye(NX)
        A[NX:, :3] = np.eye(3)[:, [0, 1, 2]]
        Pxx = self.P[:NX, :NX]
        self.P = A @ Pxx @ A.T
        self.clone = self.x[:3].copy()

    # ------------------------------------------------------------------
    def propagate(self, imu_meas, dt):
        gz, ax, ay = imu_meas
        x = self.x
        th = x[IDX_TH]
        w = gz - x[IDX_BG]
        ab = np.array([ax - x[6], ay - x[7]])
        R, dR = _R(th), _dR(th)
        aw = R @ ab
        # state update
        x[0:2] += x[3:5] * dt + 0.5 * aw * dt * dt
```

```
        x[3:5] += aw * dt
        x[IDX_TH] = wrap(th + w * dt)
        # Jacobian (additive error state)
        F = np.eye(NX)
        F[0:2, 3:5] = np.eye(2) * dt
        F[0:2, 2] = 0.5 * dt * dt * (dR @ ab)
        F[0:2, 6:8] = -0.5 * dt * dt * R
        F[3:5, 2] = dt * (dR @ ab)
        F[3:5, 6:8] = -dt * R
        F[2, 5] = -dt
        q = self.imu
        Q = np.zeros((NX, NX))
        Q[2, 2] = (q.gyro_noise * dt) ** 2
        Q[3:5, 3:5] = np.eye(2) * (q.accel_noise * dt) ** 2
        Q[0:2, 0:2] = np.eye(2) * (0.5 * q.accel_noise * dt * dt) ** 2
        Q[5, 5] = q.gyro_bias_rw ** 2 * dt
        Q[6:8, 6:8] = np.eye(2) * q.accel_bias_rw ** 2 * dt
        P = self.P
        Pxx = F @ P[:NX, :NX] @ F.T + Q
        P[:NX, :NX] = 0.5 * (Pxx + Pxx.T)                       # keep P symmetric (round-off)
        P[:NX, NX:] = F @ P[:NX, NX:]
        P[NX:, :NX] = P[:NX, NX:].T

    # ------------------------------------------------------------------
    def _update(self, y, H, Rm, gate=None):
        S = H @ self.P @ H.T + Rm
        nis = float(y @ np.linalg.solve(S, y))
        self.last_nis = nis
        if gate is not None and nis > gate:
            return False
        K = self.P @ H.T @ np.linalg.inv(S)
        dx = K @ y
        self.x += dx[:NX]
        self.x[IDX_TH] = wrap(self.x[IDX_TH])
        self.clone = self.clone + dx[NX:]
        self.clone[2] = wrap(self.clone[2])
        I = np.eye(self.P.shape[0])
        P = (I - K @ H) @ self.P @ (I - K @ H).T + K @ Rm @ K.T
        self.P = 0.5 * (P + P.T)
        return True

    def update_gps(self, z, sigma):
        H = np.zeros((2, NX + 3))
        H[0, 0] = H[1, 1] = 1.0
        y = z - self.x[0:2]
        return self._update(y, H, np.eye(2) * sigma ** 2, gate=13.8)   # chi2(2) 99.9 %

    def update_vo(self, z, sigma_t, sigma_r, infl=1.3):
        """Relative-pose update between the clone and the current pose, then re-clone."""
        pc, thc = self.clone[:2], self.clone[2]
        p, th = self.x[0:2], self.x[IDX_TH]
        Rc = _R(thc)
        h = np.concatenate([Rc.T @ (p - pc), [wrap(th - thc)]])
        y = z - h
        y[2] = wrap(y[2])
        H = np.zeros((3, NX + 3))
        H[0:2, 0:2] = Rc.T
        H[2, 2] = 1.0
        H[0:2, NX:NX + 2] = -Rc.T
        H[0:2, NX + 2] = _dR(thc).T @ (p - pc)
        H[2, NX + 2] = -1.0
        Rm = np.diag([(infl * sigma_t) ** 2, (infl * sigma_t) ** 2, (infl * sigma_r) ** 2])
        ok = self._update(y, H, Rm, gate=16.3)                      # chi2(3) 99.9 %
        self._clone()
        return ok


# ---------------------------------------------------------------------------
def run_estimator(drive: Drive, imu: np.ndarray, imu_params: ImuParams, gps: GpsMeas,
                  vo: VoMeas | None, mode: str, rng: np.random.Generator):
    """Run one estimator over the whole drive.  Returns (N,3) pose estimates and sigma_xy."""
    assert mode in ("imu_only", "gps_imu", "vio", "vo_only")
    n, dt = len(drive.t), drive.dt
    # initialisation from a short GPS fix and approximate heading / speed
    pose0 = drive.pose[0].copy()
    pose0[:2] += rng.normal(0, 0.5, 2)
    pose0[2] += rng.normal(0, 0.02)
    v0 = drive.v[0] * np.array([np.cos(drive.pose[0, 2]), np.sin(drive.pose[0, 2])]) + rng.normal(0, 0.2, 2)
    est = np.zeros((n, 3))
    sig = np.zeros(n)
    if mode == "vo_only":                       # pure visual dead reckoning (interpolated between frames)
        from ..geometry import compose
        pose = pose0.copy()
        key_idx, key_pose = [0], [pose.copy()]
        for j in range(vo.idx.size):
            pose = compose(pose, vo.z[j])
            key_idx.append(int(vo.idx[j]))
            key_pose.append(pose.copy())
        key_idx, key_pose = np.array(key_idx), np.array(key_pose)
        th = np.unwrap(key_pose[:, 2])
        t_all = np.arange(n)
        est[:, 0] = np.interp(t_all, key_idx, key_pose[:, 0])
        est[:, 1] = np.interp(t_all, key_idx, key_pose[:, 1])
        est[:, 2] = (np.interp(t_all, key_idx, th) + np.pi) % (2 * np.pi) - np.pi
        return est, sig
    f = VioEKF(pose0, v0, imu_params, use_vo=(mode == "vio"))
```

```
    gps_ptr, vo_ptr = 0, 0
    gps_idx = set(gps.idx.tolist()) if mode in ("gps_imu", "vio") else set()
    gps_map = {int(i): k for k, i in enumerate(gps.idx)}
    vo_map = {int(i): k for k, i in enumerate(vo.idx)} if mode == "vio" else {}
    est[0] = f.x[[0, 1, 2]]
    for i in range(1, n):
        f.propagate(imu[i], dt)
        if i in vo_map:
            k = vo_map[i]
            f.update_vo(vo.z[k], vo.sigma[k, 0], vo.sigma[k, 1])
        if i in gps_idx and i > 0:
            f.update_gps(gps.z[gps_map[i]], gps.sigma)
        est[i] = f.x[[0, 1, 2]]
        sig[i] = np.sqrt(f.P[0, 0] + f.P[1, 1])
    return est, sig
```

sdc/localization/ndt.py

```
"""2-D Normal Distributions Transform (NDT) scan registration.

Implements the P2D-NDT of Biber & Strasser (2003) with the Gaussian-plus-
uniform likelihood approximation of Magnusson (2009):

    cost(x) = d1 * exp(-d2/2 * (x'-mu)^T Sigma^-1 (x'-mu)),   x' = T(p) x

minimised with damped Newton + backtracking line search.  Four half-cell
shifted grids are used to remove discretisation artefacts.  The final Hessian
is returned so that registration *degeneracy* (e.g. a featureless tunnel) can be
analysed from its eigenvalues.
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np

_OFF = 1 << 20
_K = 1 << 22


class NDTGrid:
    def __init__(self, points: np.ndarray, cell: float, min_pts: int = 4,
                 shifts=((0.0, 0.0), (0.5, 0.0), (0.0, 0.5), (0.5, 0.5)),
                 min_ratio: float = 1e-3, min_var: float = 4e-4):
        self.cell = float(cell)
        self.grids = [self._build(points, shift, min_pts, min_ratio, min_var) for shift in shifts]
        self.shifts = shifts
        # Magnusson (2009) parameters of the Gaussian approximation
        p_o = 0.1
        c1, c2 = 10.0 * (1 - p_o), p_o / (self.cell ** 2)
        d3 = -np.log(c2)
        self.d1 = -np.log(c1 + c2) - d3
        self.d2 = -2.0 * np.log((-np.log(c1 * np.exp(-0.5) + c2) - d3) / self.d1)

    def _build(self, pts, shift, min_pts, min_ratio, min_var):
        ij = np.floor(pts / self.cell - np.asarray(shift)).astype(np.int64)
        key = (ij[:, 0] + _OFF) * _K + (ij[:, 1] + _OFF)
        uniq, inv, cnt = np.unique(key, return_inverse=True, return_counts=True)
        x, y = pts[:, 0], pts[:, 1]
        sx, sy = np.bincount(inv, x), np.bincount(inv, y)
        sxx, sxy, syy = np.bincount(inv, x * x), np.bincount(inv, x * y), np.bincount(inv, y * y)
        mx, my = sx / cnt, sy / cnt
        cxx, cxy, cyy = sxx / cnt - mx * mx, sxy / cnt - mx * my, syy / cnt - my * my
        keep = cnt >= min_pts
        uniq, mx, my, cxx, cxy, cyy = uniq[keep], mx[keep], my[keep], cxx[keep], cxy[keep], cyy[keep]
        C = np.zeros((len(uniq), 2, 2))
        C[:, 0, 0], C[:, 0, 1], C[:, 1, 0], C[:, 1, 1] = cxx, cxy, cxy, cyy
        if len(uniq):
            lam, V = np.linalg.eigh(C)                       # ascending
            lam = np.maximum(lam, np.maximum(min_ratio * lam[:, 1:2], min_var))
            Cinv = np.einsum("nij,nj,nkj->nik", V, 1.0 / lam, V)
        else:
            Cinv = np.zeros((0, 2, 2))
        return uniq, np.stack([mx, my], axis=1), Cinv

    def _lookup(self, g, pts_w):
        uniq, mu, Cinv = self.grids[g]
        shift = self.shifts[g]
        ij = np.floor(pts_w / self.cell - np.asarray(shift)).astype(np.int64)
        key = (ij[:, 0] + _OFF) * _K + (ij[:, 1] + _OFF)
        if len(uniq) == 0:
            return np.zeros(len(pts_w), bool), None, None
        pos = np.searchsorted(uniq, key)
        pos = np.clip(pos, 0, len(uniq) - 1)
        hit = uniq[pos] == key
        return hit, mu[pos[hit]], Cinv[pos[hit]]


@dataclass
class NDTResult:
    pose: np.ndarray
    cost: float
```

```python
    n_matched: int
    n_points: int
    iters: int
    hessian: np.ndarray

    @property
    def fitness(self) -> float:
        """Mean per-point cost (more negative = better)."""
        return self.cost / max(self.n_points, 1)

    def degeneracy(self) -> float:
        """Ratio  lambda_min / lambda_max  of the translational Hessian block (0 = degenerate)."""
        w = np.linalg.eigvalsh(self.hessian[:2, :2])
        return float(max(w[0], 0.0) / max(w[1], 1e-12))


def _cost_only(src, pose, grid: NDTGrid):
    c, s = np.cos(pose[2]), np.sin(pose[2])
    pw = np.empty_like(src)
    pw[:, 0] = c * src[:, 0] - s * src[:, 1] + pose[0]
    pw[:, 1] = s * src[:, 0] + c * src[:, 1] + pose[1]
    total, nm = 0.0, 0
    for g in range(len(grid.grids)):
        hit, mu, Ci = grid._lookup(g, pw)
        if hit.any():
            e = pw[hit] - mu
            q = np.einsum("ni,nij,nj->n", e, Ci, e)
            total += float(np.sum(grid.d1 * np.exp(-0.5 * grid.d2 * q)))
            nm += int(hit.sum())
    return total, nm


def ndt_register(src: np.ndarray, grid: NDTGrid, init, max_iter: int = 30, tol: float = 1e-5) -> NDTResult:
    """Find pose ``p`` (sensor -> map) minimising the NDT cost of ``src`` (sensor frame points)."""
    pose = np.array(init, float)
    cost, nm = _cost_only(src, pose, grid)
    H = np.eye(3)
    it = 0
    for it in range(1, max_iter + 1):
        c, s = np.cos(pose[2]), np.sin(pose[2])
        pw = np.empty_like(src)
        pw[:, 0] = c * src[:, 0] - s * src[:, 1] + pose[0]
        pw[:, 1] = s * src[:, 0] + c * src[:, 1] + pose[1]
        g = np.zeros(3)
        H = np.zeros((3, 3))
        for gi in range(len(grid.grids)):
            hit, mu, Ci = grid._lookup(gi, pw)
            if not hit.any():
                continue
            e = pw[hit] - mu
            q = np.einsum("ni,nij,nj->n", e, Ci, e)
            w = -grid.d1 * grid.d2 * np.exp(-0.5 * grid.d2 * q)     # >= 0
            sx, sy = src[hit, 0], src[hit, 1]
            J = np.zeros((len(e), 2, 3))
            J[:, 0, 0] = 1.0
            J[:, 1, 1] = 1.0
            J[:, 0, 2] = -s * sx - c * sy
            J[:, 1, 2] = c * sx - s * sy
            CiJ = np.einsum("nij,njk->nik", Ci, J)
            a = np.einsum("nik,ni->nk", J, np.einsum("nij,nj->ni", Ci, e))    # J^T Ci e
            JtCiJ = np.einsum("nik,nil->nkl", J, CiJ)
            g += (w[:, None] * a).sum(0)
            H += (w[:, None, None] * (JtCiJ - grid.d2 * a[:, :, None] * a[:, None, :])).sum(0)
            # second-order term of the rotation
            d2x = np.stack([-(c * sx - s * sy), -(s * sx + c * sy)], axis=1)   # -R x
            H[2, 2] += float((w * np.einsum("ni,nij,nj->n", e, Ci, d2x)).sum())
        # Levenberg-Marquardt style damping until the system is positive definite
        lam = 1e-6 * max(np.trace(H), 1.0)
        for _ in range(30):
            try:
                L = np.linalg.cholesky(H + lam * np.eye(3))
                break
            except np.linalg.LinAlgError:
                lam *= 10.0
        else:
            break
        delta = -np.linalg.solve(H + lam * np.eye(3), g)
        step = 1.0
        improved = False
        for _ in range(8):
            cand = pose + step * delta
            cand[2] = (cand[2] + np.pi) % (2 * np.pi) - np.pi
            c_new, nm_new = _cost_only(src, cand, grid)
            if c_new < cost:
                pose, cost, nm, improved = cand, c_new, nm_new, True
                break
            step *= 0.5
        if not improved or np.linalg.norm(step * delta) < tol:
            break
    return NDTResult(pose, cost, nm, len(src), it, H)


def ndt_register_multires(src, grids, init, **kw) -> NDTResult:
    """Coarse-to-fine registration over a list of ``NDTGrid`` (largest cell first)."""
    pose = init
    res = None
```

```
    for g in grids:
        res = ndt_register(src, g, pose, **kw)
        pose = res.pose
    return res
```

sdc/localization/icp.py

```
"""Point-to-point 2-D ICP (trimmed, KD-tree) -- the scan-to-scan baseline."""
from __future__ import annotations

import numpy as np
from scipy.spatial import cKDTree

from ..geometry import compose, wrap


def _best_fit(A: np.ndarray, B: np.ndarray):
    """Rigid transform (x,y,theta) minimising |R A + t - B|^2 (closed form)."""
    ca, cb = A.mean(0), B.mean(0)
    Ac, Bc = A - ca, B - cb
    h = Ac.T @ Bc
    th = np.arctan2(h[0, 1] - h[1, 0], h[0, 0] + h[1, 1])
    c, s = np.cos(th), np.sin(th)
    t = cb - np.array([c * ca[0] - s * ca[1], s * ca[0] + c * ca[1]])
    return np.array([t[0], t[1], th])


def icp(src: np.ndarray, dst: np.ndarray, init, tree: cKDTree | None = None, max_iter: int = 40,
        max_dist: float = 2.0, trim: float = 0.85, tol: float = 1e-5):
    """Estimate the pose of ``src`` frame in the ``dst`` frame.  Returns (pose, rms, n_corr)."""
    tree = tree or cKDTree(dst)
    pose = np.array(init, float)
    rms, n = np.inf, 0
    for _ in range(max_iter):
        c, s = np.cos(pose[2]), np.sin(pose[2])
        pw = np.stack([c * src[:, 0] - s * src[:, 1] + pose[0], s * src[:, 0] + c * src[:, 1] + pose[1]], axis=1)
        d, j = tree.query(pw, distance_upper_bound=max_dist)
        ok = np.isfinite(d)
        if ok.sum() < 10:
            break
        idx = np.where(ok)[0]
        if trim < 1.0:
            keep = idx[np.argsort(d[idx])[: int(trim * len(idx))]]
        else:
            keep = idx
        inc = _best_fit(pw[keep], dst[j[keep]])
        new = compose(inc, pose)
        rms = float(np.sqrt(np.mean(d[keep] ** 2)))
        n = len(keep)
        done = np.linalg.norm(inc[:2]) < tol and abs(inc[2]) < tol
        pose = new
        if done:
            break
    pose[2] = wrap(pose[2])
    return pose, rms, n


def _normals(pts: np.ndarray, k: int = 3) -> np.ndarray:
    """Unit normals of an angularly ordered 2-D scan from neighbouring points (PCA)."""
    n = len(pts)
    nrm = np.zeros((n, 2))
    pad = np.vstack([np.repeat(pts[:1], k, 0), pts, np.repeat(pts[-1:], k, 0)])
    for i in range(n):
        nb = pad[i:i + 2 * k + 1]
        c = nb - nb.mean(0)
        w, v = np.linalg.eigh(c.T @ c)
        nrm[i] = v[:, 0]
    return nrm


def icp_point_to_line(src, dst, init, max_iter: int = 40, max_dist: float = 1.5, trim: float = 0.85,
                      tol: float = 1e-5, dst_normals=None, tree=None):
    """Point-to-line ICP (Censi 2008 style linearisation) - the stronger scan-to-scan baseline."""
    tree = tree or cKDTree(dst)
    nrm = dst_normals if dst_normals is not None else _normals(dst)
    pose = np.array(init, float)
    for _ in range(max_iter):
        c, s = np.cos(pose[2]), np.sin(pose[2])
        pw = np.stack([c * src[:, 0] - s * src[:, 1] + pose[0], s * src[:, 0] + c * src[:, 1] + pose[1]], axis=1)
        d, j = tree.query(pw, distance_upper_bound=max_dist)
        ok = np.where(np.isfinite(d))[0]
        if len(ok) < 10:
            break
        if trim < 1.0:
            ok = ok[np.argsort(d[ok])[: int(trim * len(ok))]]
        nn = nrm[j[ok]]
        r = np.einsum("ij,ij->i", nn, pw[ok] - dst[j[ok]])
        Jt = nn
        Jr = nn[:, 0] * (-pw[ok][:, 1]) + nn[:, 1] * pw[ok][:, 0]     # n . perp(p')
        J = np.column_stack([Jt, Jr])
        delta = -np.linalg.lstsq(J.T @ J + 1e-9 * np.eye(3), J.T @ r, rcond=None)[0]
        # apply the increment in the world frame (rotation about the origin)
```

```
        cd, sd = np.cos(delta[2]), np.sin(delta[2])
        pose = np.array([cd * pose[0] - sd * pose[1] + delta[0],
                         sd * pose[0] + cd * pose[1] + delta[1],
                         wrap(pose[2] + delta[2])])
        if np.linalg.norm(delta[:2]) < tol and abs(delta[2]) < tol:
            break
    return pose
```

sdc/localization/pose_graph.py

```
"""SE(2) pose-graph optimisation (vectorised Gauss-Newton on a sparse system)."""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla

from ..geometry import wrap


@dataclass
class Edge:
    i: int
    j: int
    z: np.ndarray       # measured pose of j expressed in the frame of i
    omega: np.ndarray   # 3x3 information matrix
    kind: str = "odom"


def _linearize(x, I, J, Z):
    """Residuals e (E,3) and Jacobians A=de/dxi, B=de/dxj for all edges at once."""
    xi, xj = x[I], x[J]
    ci, si = np.cos(xi[:, 2]), np.sin(xi[:, 2])
    cz, sz = np.cos(Z[:, 2]), np.sin(Z[:, 2])
    dx, dy = xj[:, 0] - xi[:, 0], xj[:, 1] - xi[:, 1]
    zt0, zt1 = ci * dx + si * dy, -si * dx + ci * dy              # Ri^T (tj - ti)
    r0, r1 = zt0 - Z[:, 0], zt1 - Z[:, 1]
    e = np.stack([cz * r0 + sz * r1, -sz * r0 + cz * r1, wrap(xj[:, 2] - xi[:, 2] - Z[:, 2])], axis=1)
    E = len(I)
    RzT = np.zeros((E, 2, 2)); RzT[:, 0, 0] = cz; RzT[:, 0, 1] = sz; RzT[:, 1, 0] = -sz; RzT[:, 1, 1] = cz
    RiT = np.zeros((E, 2, 2)); RiT[:, 0, 0] = ci; RiT[:, 0, 1] = si; RiT[:, 1, 0] = -si; RiT[:, 1, 1] = ci
    dRiT_dt = np.stack([-si * dx + ci * dy, -ci * dx - si * dy], axis=1)   # d(Ri^T)/dth_i (tj - ti)
    RzRi = np.einsum("eij,ejk->eik", RzT, RiT)
    A = np.zeros((E, 3, 3)); B = np.zeros((E, 3, 3))
    A[:, :2, :2] = -RzRi
    A[:, :2, 2] = np.einsum("eij,ej->ei", RzT, dRiT_dt)
    A[:, 2, 2] = -1.0
    B[:, :2, :2] = RzRi
    B[:, 2, 2] = 1.0
    return e, A, B


def optimize(poses: np.ndarray, edges: list, iters: int = 10, tol: float = 1e-6, robust: float | None = 3.0):
    """Optimise ``poses`` (N,3) holding node 0 fixed.  Returns (poses, chi2)."""
    x = poses.copy()
    n = len(x)
    I = np.array([e.i for e in edges]); J = np.array([e.j for e in edges])
    Z = np.array([e.z for e in edges]); Om = np.array([e.omega for e in edges])
    is_loop = np.array([e.kind == "loop" for e in edges])
    ar = np.arange(3)
    chi2 = np.inf
    for _ in range(iters):
        e, A, B = _linearize(x, I, J, Z)
        c2 = np.einsum("ei,eij,ej->e", e, Om, e)
        w = np.ones(len(edges))
        if robust is not None:
            big = is_loop & (c2 > robust ** 2)
            w[big] = robust / np.sqrt(c2[big])
        chi2 = float(np.sum(w * c2))
        Omw = Om * w[:, None, None]
        Hii = np.einsum("eki,ekl,elj->eij", A, Omw, A)
        Hij = np.einsum("eki,ekl,elj->eij", A, Omw, B)
        Hjj = np.einsum("eki,ekl,elj->eij", B, Omw, B)
        bi = np.einsum("eki,ekl,el->ei", A, Omw, e)
        bj = np.einsum("eki,ekl,el->ei", B, Omw, e)
        rows, cols, vals = [], [], []
        for P_, Q_, M in ((I, I, Hii), (I, J, Hij), (J, I, np.transpose(Hij, (0, 2, 1))), (J, J, Hjj)):
            rows.append((3 * P_[:, None, None] + ar[None, :, None] + 0 * ar[None, None, :]).ravel())
            cols.append((3 * Q_[:, None, None] + ar[None, None, :] + 0 * ar[None, :, None]).ravel())
            vals.append(M.ravel())
        H = sp.coo_matrix((np.concatenate(vals), (np.concatenate(rows), np.concatenate(cols))),
                          shape=(3 * n, 3 * n)).tocsc()
        b = np.zeros(3 * n)
        np.add.at(b, (3 * I[:, None] + ar[None, :]).ravel(), bi.ravel())
        np.add.at(b, (3 * J[:, None] + ar[None, :]).ravel(), bj.ravel())
        H = H + sp.identity(3 * n, format="csc") * 1e-9
        H = H + sp.csc_matrix((np.full(3, 1e9), (ar, ar)), shape=(3 * n, 3 * n))      # anchor node 0
        dx = spla.spsolve(H, -b)
        x += dx.reshape(n, 3)
        x[:, 2] = wrap(x[:, 2])
```

```
        if np.linalg.norm(dx) < tol:
            break
    return x, chi2
```

sdc/localization/lidar_slam.py

```
"""LiDAR SLAM in the spirit of HDL-Graph-SLAM:

  * front end : keyframe-based NDT *scan-to-submap* odometry (coarse-to-fine),
  * loop detection : distance-based candidates, multi-start NDT verification
                     with a KD-tree inlier-ratio test,
  * back end  : SE(2) pose-graph optimisation with a Huber kernel on loop edges.

A LOAM-style curvature feature extractor is provided as well (``extract_loam_features``);
it is used for diagnostics / visualisation of edge and planar points.
"""
from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np
from scipy.spatial import cKDTree

from ..geometry import between, compose, transform_points, voxel_downsample
from .icp import icp
from .ndt import NDTGrid, ndt_register_multires
from .pose_graph import Edge, optimize


# ---------------------------------------------------------------------------
@dataclass
class Keyframe:
    idx: int
    pose: np.ndarray
    pts: np.ndarray
    dist: float


@dataclass
class SlamConfig:
    cells: tuple = (3.0, 1.5)
    kf_dist: float = 1.5
    kf_rot: float = 0.15
    submap_kfs: int = 20
    voxel: float = 0.25
    loop: bool = True
    loop_min_travel: float = 80.0
    loop_radius: float = 20.0
    loop_every: int = 3
    loop_inlier_thr: float = 0.65
    odom_sigma: tuple = (0.05, 0.05, 0.01)
    loop_sigma: tuple = (0.15, 0.15, 0.02)
    optimize_every: int = 10


class LidarSlam:
    def __init__(self, init_pose, cfg: SlamConfig = SlamConfig()):
        self.cfg = cfg
        self.kfs: list[Keyframe] = []
        self.edges: list[Edge] = []
        self.cur = np.array(init_pose, float)
        self.delta = np.zeros(3)
        self._grids = None
        self.scan_log: list[dict] = []
        self.n_loops = 0
        self._since_opt = 0
        self._kf_since_loop = 0
        self.loop_pairs: list[tuple[int, int]] = []

    # ------------------------------------------------------------------
    def _world_points(self, ids):
        pts = [transform_points(self.kfs[k].pose, self.kfs[k].pts) for k in ids]
        return voxel_downsample(np.concatenate(pts, axis=0), self.cfg.voxel)

    def _rebuild_submap(self):
        lo = max(0, len(self.kfs) - self.cfg.submap_kfs)
        wp = self._world_points(range(lo, len(self.kfs)))
        self._grids = [NDTGrid(wp, c, min_pts=3) for c in self.cfg.cells]

    def _add_keyframe(self, idx, pts, dist):
        prev = self.kfs[-1] if self.kfs else None
        self.kfs.append(Keyframe(idx, self.cur.copy(), pts, dist))
        if prev is not None:
            z = between(prev.pose, self.cur)
            om = np.diag(1.0 / np.array(self.cfg.odom_sigma) ** 2)
            self.edges.append(Edge(len(self.kfs) - 2, len(self.kfs) - 1, z, om, "odom"))
        self._rebuild_submap()

    # ------------------------------------------------------------------
    def process(self, pts: np.ndarray, idx: int):
        if not self.kfs:
            self._add_keyframe(idx, pts, 0.0)
```

```python
            return self.cur
        guess = compose(self.cur, self.delta)
        res = ndt_register_multires(pts, self._grids, guess)
        self.delta = between(self.cur, res.pose)
        self.cur = res.pose
        self.scan_log.append(dict(idx=idx, pose=res.pose.copy(), deg=res.degeneracy(),
                                  fit=res.fitness, nm=res.n_matched))
        last = self.kfs[-1]
        d = between(last.pose, self.cur)
        if np.hypot(d[0], d[1]) > self.cfg.kf_dist or abs(d[2]) > self.cfg.kf_rot:
            self._add_keyframe(idx, pts, last.dist + float(np.hypot(d[0], d[1])))
            self._kf_since_loop += 1
            if self.cfg.loop and self._kf_since_loop >= self.cfg.loop_every:
                self._try_loop()
        return self.cur

    # ------------------------------------------------------------------
    def _try_loop(self):
        i = len(self.kfs) - 1
        kf = self.kfs[i]
        pos = np.array([k.pose[:2] for k in self.kfs[:i]])
        trav = kf.dist - np.array([k.dist for k in self.kfs[:i]])
        dist = np.hypot(*(pos - kf.pose[:2]).T)
        cand = np.where((trav > self.cfg.loop_min_travel) & (dist < self.cfg.loop_radius))[0]
        if len(cand) == 0:
            return
        self._kf_since_loop = 0
        j = int(cand[np.argmin(dist[cand])])
        ids = range(max(0, j - 6), min(i - 1, j + 6) + 1)
        wp = self._world_points(ids)
        tree = cKDTree(wp)
        grids = [NDTGrid(wp, c, min_pts=3) for c in self.cfg.cells]
        best, best_inl = None, 0.0
        for dyaw in (0.0, 0.08, -0.08):
            for dx, dy in ((0, 0), (2, 0), (-2, 0), (0, 2), (0, -2)):
                init = kf.pose + np.array([dx, dy, dyaw])
                r = ndt_register_multires(kf.pts, grids, init)
                pw = transform_points(r.pose, kf.pts)
                dd, _ = tree.query(pw, distance_upper_bound=0.3)
                inl = float(np.mean(np.isfinite(dd)))
                if inl > best_inl:
                    best, best_inl = r, inl
        if best is None or best_inl < self.cfg.loop_inlier_thr:
            return
        z = between(self.kfs[j].pose, best.pose)
        om = np.diag(1.0 / np.array(self.cfg.loop_sigma) ** 2)
        self.edges.append(Edge(j, i, z, om, "loop"))
        self.loop_pairs.append((j, i))
        self.n_loops += 1
        self._since_opt += 1
        if self._since_opt >= 1 and (self.n_loops == 1 or self.n_loops % self.cfg.optimize_every == 0):
            self.optimize()

    def optimize(self):
        poses = np.array([k.pose for k in self.kfs])
        new, chi2 = optimize(poses, self.edges)
        for k, p in zip(self.kfs, new):
            k.pose = p
        self.cur = self.kfs[-1].pose.copy()
        self._since_opt = 0
        self._rebuild_submap()
        return chi2

    # ------------------------------------------------------------------
    def keyframe_poses(self):
        return np.array([k.pose for k in self.kfs]), np.array([k.idx for k in self.kfs])

    def global_map(self, voxel: float = 0.15):
        return voxel_downsample(self._world_points(range(len(self.kfs))), voxel)


# ----------------------------------------------------------------------------
def icp_odometry(scans, init_pose, priors=None):
    """Scan-to-scan point-to-line ICP odometry (open loop) - the baseline.

    ``priors[k-1]`` (optional) is an initial guess for the relative pose between
    scan k-1 and k (e.g. from visual odometry); otherwise constant velocity is used.
    """
    from .icp import icp_point_to_line, _normals
    from scipy.spatial import cKDTree
    poses = [np.array(init_pose, float)]
    delta = np.zeros(3)
    for k in range(1, len(scans)):
        init = delta if priors is None else priors[k - 1]
        rel = icp_point_to_line(scans[k], scans[k - 1], init, dst_normals=_normals(scans[k - 1]),
                                tree=cKDTree(scans[k - 1]))
        delta = rel
        poses.append(compose(poses[-1], rel))
    return np.array(poses)


def extract_loam_features(pts: np.ndarray, k: int = 5, n_edge: int = 20, n_planar: int = 40,
                          edge_thr: float = 0.08, planar_thr: float = 0.01):
    """LOAM-style curvature features on one (angularly ordered) scan line.

    c_i = | sum_{j in S}(p_i - p_j) | / (|S| |p_i|),  S = k neighbours on each side.
```

```
    High curvature -> edge points; low curvature -> planar points.  Points near
    range discontinuities (occlusions) and nearly parallel to the beam are rejected.
    """
    n = len(pts)
    if n < 4 * k:
        return np.zeros(0, int), np.zeros(0, int), np.zeros(n)
    c = np.full(n, np.nan)
    r = np.hypot(pts[:, 0], pts[:, 1])
    for i in range(k, n - k):
        nb = np.r_[pts[i - k:i], pts[i + 1:i + k + 1]]
        c[i] = np.linalg.norm((pts[i] - nb).sum(0)) / (2 * k * r[i])
    bad = np.zeros(n, bool)
    jump = np.abs(np.diff(r)) > 0.5 + 0.1 * r[:-1]                  # range discontinuity
    for i in np.where(jump)[0]:
        bad[max(0, i - k):i + k + 1] = True
    bad |= np.isnan(c)
    order = np.argsort(-np.nan_to_num(c, nan=-1))
    edges, taken = [], np.zeros(n, bool)
    for i in order:
        if len(edges) >= n_edge or c[i] < edge_thr:
            break
        if bad[i] or taken[max(0, i - k):i + k + 1].any():
            continue
        edges.append(i)
        taken[max(0, i - k):i + k + 1] = True
    cc = np.where(bad, np.inf, c)
    planars = [i for i in np.argsort(cc)[:n_planar] if cc[i] < planar_thr]
    return np.array(edges, int), np.array(planars, int), c
```

sdc/localization/hd_map.py

```
"""High-definition (HD) point-cloud map with the two lookup structures used for
localisation: an NDT grid (scan-to-map alignment) and a Euclidean distance
field (particle-filter likelihood)."""
from __future__ import annotations

import numpy as np
from scipy.ndimage import distance_transform_edt

from ..geometry import transform_points, voxel_downsample
from ..sim.drive import Drive, LidarParams, lidar_scan
from ..sim.world import World
from .ndt import NDTGrid


class HDMap:
    def __init__(self, points: np.ndarray, res: float = 0.1, cells=(3.0, 1.5), max_dist: float = 3.0):
        self.points = voxel_downsample(points, 0.1)
        self.res = res
        self.max_dist = max_dist
        lo = self.points.min(0) - 5.0
        hi = self.points.max(0) + 5.0
        self.origin = lo
        shape = np.ceil((hi - lo) / res).astype(int) + 1
        occ = np.zeros(shape, bool)
        ij = np.floor((self.points - lo) / res).astype(int)
        occ[ij[:, 0], ij[:, 1]] = True
        self.edt = (distance_transform_edt(~occ) * res).astype(np.float32)
        self.shape = shape
        self.grids = [NDTGrid(self.points, c, min_pts=4) for c in cells]

    def distance(self, pts: np.ndarray) -> np.ndarray:
        """Distance to the nearest map point (clipped); works for arbitrary-shaped input (...,2)."""
        ij = np.floor((pts - self.origin) / self.res).astype(int)
        ok = (ij[..., 0] >= 0) & (ij[..., 0] < self.shape[0]) & (ij[..., 1] >= 0) & (ij[..., 1] < self.shape[1])
        ii = np.clip(ij[..., 0], 0, self.shape[0] - 1)
        jj = np.clip(ij[..., 1], 0, self.shape[1] - 1)
        d = self.edt[ii, jj]
        return np.where(ok, np.minimum(d, self.max_dist), self.max_dist)


def survey_map(world: World, drive: Drive, rng: np.random.Generator, pose_sigma=(0.02, 0.02, 0.0015),
               lidar: LidarParams = LidarParams(), rate_div: int = 10) -> HDMap:
    """Build the HD map from a survey drive with (near) ground-truth poses - the stand-in for an
    RTK-GNSS/INS-referenced mapping vehicle."""
    pts = []
    for i in range(0, len(drive.t), rate_div):
        pose = drive.pose[i] + rng.normal(0, pose_sigma)
        scan = lidar_scan(world, drive.pose[i], rng, lidar)
        pts.append(transform_points(pose, scan))
    return HDMap(np.concatenate(pts, axis=0))
```

sdc/localization/particle_filter.py

```
"""Monte-Carlo localisation (particle filter) against the HD map's distance field,
and an NDT scan-to-map localiser driven by an odometry prediction."""
from __future__ import annotations
```

```python

import numpy as np

from ..geometry import compose, wrap
from .hd_map import HDMap
from .ndt import ndt_register_multires


class ParticleFilter:
    def __init__(self, hdmap: HDMap, n: int, rng: np.random.Generator, sigma_hit: float = 0.2,
                 n_beams: int = 72, beam_power: float = 0.35, z_rand: float = 0.08,
                 motion_noise=(0.02, 0.04, 0.004, 0.01)):
        self.map, self.n, self.rng = hdmap, n, rng
        self.sigma_hit, self.n_beams, self.beam_power, self.z_rand = sigma_hit, n_beams, beam_power, z_rand
        self.motion_noise = motion_noise       # (trans const, trans prop., rot const, rot prop.)
        self.p = np.zeros((n, 3))
        self.logw = np.full(n, -np.log(n))
        self.neff = float(n)

    # ------------------------------------------------------------------
    def init_gaussian(self, mean, std):
        self.p = np.asarray(mean) + self.rng.normal(0, 1, (self.n, 3)) * np.asarray(std)
        self.logw[:] = -np.log(self.n)

    def init_uniform(self, sampler):
        """``sampler(n, rng)`` returns (n,3) poses - e.g. uniform over the drivable corridor."""
        self.p = sampler(self.n, self.rng)
        self.logw[:] = -np.log(self.n)

    # ------------------------------------------------------------------
    def predict(self, delta):
        """Apply an odometry increment (in the body frame) with noise to every particle."""
        a, b, c, d = self.motion_noise
        dist = np.hypot(delta[0], delta[1])
        n = self.n
        dx = delta[0] + self.rng.normal(0, a + b * dist, n)
        dy = delta[1] + self.rng.normal(0, a + b * dist, n)
        dth = delta[2] + self.rng.normal(0, c + d * abs(delta[2]), n)
        cs, sn = np.cos(self.p[:, 2]), np.sin(self.p[:, 2])
        self.p[:, 0] += cs * dx - sn * dy
        self.p[:, 1] += sn * dx + cs * dy
        self.p[:, 2] = wrap(self.p[:, 2] + dth)

    def update(self, scan: np.ndarray):
        if len(scan) == 0:
            return
        sel = np.linspace(0, len(scan) - 1, min(self.n_beams, len(scan))).astype(int)
        s = scan[sel]
        cs, sn = np.cos(self.p[:, 2])[:, None], np.sin(self.p[:, 2])[:, None]
        wx = self.p[:, 0:1] + cs * s[None, :, 0] - sn * s[None, :, 1]
        wy = self.p[:, 1:2] + sn * s[None, :, 0] + cs * s[None, :, 1]
        d = self.map.distance(np.stack([wx, wy], axis=-1))
        lik = (1 - self.z_rand) * np.exp(-0.5 * (d / self.sigma_hit) ** 2) + self.z_rand
        ll = self.beam_power * np.log(lik).sum(axis=1)
        self.logw += ll
        self.logw -= np.logaddexp.reduce(self.logw)
        w = np.exp(self.logw)
        self.neff = 1.0 / np.sum(w * w)
        if self.neff < 0.5 * self.n:
            self._resample(w)

    def _resample(self, w):
        n = self.n
        pos = (self.rng.random() + np.arange(n)) / n
        idx = np.minimum(np.searchsorted(np.cumsum(w), pos), n - 1)
        self.p = self.p[idx]
        self.p[:, :2] += self.rng.normal(0, 0.02, (n, 2))
        self.p[:, 2] = wrap(self.p[:, 2] + self.rng.normal(0, 0.003, n))
        self.logw[:] = -np.log(n)

    # ------------------------------------------------------------------
    def estimate(self):
        w = np.exp(self.logw)
        xy = (w[:, None] * self.p[:, :2]).sum(0)
        th = np.arctan2((w * np.sin(self.p[:, 2])).sum(), (w * np.cos(self.p[:, 2])).sum())
        spread = np.sqrt((w * ((self.p[:, :2] - xy) ** 2).sum(1)).sum())
        return np.array([xy[0], xy[1], th]), spread


class NdtLocalizer:
    """Scan-to-map NDT alignment seeded by an odometry prediction."""

    def __init__(self, hdmap: HDMap, init_pose):
        self.map = hdmap
        self.pose = np.array(init_pose, float)
        self.last_degeneracy = np.nan

    def step(self, scan: np.ndarray, odom_delta):
        pred = compose(self.pose, odom_delta)
        res = ndt_register_multires(scan, self.map.grids, pred)
        self.pose = res.pose
        self.last_degeneracy = res.degeneracy()
        return self.pose, res
```

sdc/behavior/bt.py

```python
"""A small, dependency-free behaviour-tree library (tick-based, reactive, with a blackboard).

Node semantics follow the usual BT conventions (Colledanchise & Ogren, 2018):
  * Sequence  ("->")  : ticks children left to right, returns the first non-SUCCESS status.
  * Fallback  ("?")   : ticks children left to right, returns the first non-FAILURE status.
  * With ``memory=True`` a Sequence resumes at the child that returned RUNNING instead of
    re-evaluating earlier children (used for committed multi-step manoeuvres).
  * When a higher-priority child takes over, lower-priority children that were RUNNING are
    *halted* (their state is reset) -- this is what gives BTs their reactivity / pre-emption.
"""
from __future__ import annotations

from enum import Enum
from typing import Callable

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse, FancyBboxPatch


class Status(Enum):
    IDLE = 0
    SUCCESS = 1
    FAILURE = 2
    RUNNING = 3


class Node:
    kind = "node"

    def __init__(self, name: str, children=None):
        self.name = name
        self.children = list(children or [])
        self.status = Status.IDLE
        self.ticks = 0

    def tick(self, bb: dict) -> Status:
        self.ticks += 1
        self.status = self._tick(bb)
        return self.status

    def _tick(self, bb) -> Status:
        raise NotImplementedError

    def halt(self, bb: dict):
        for c in self.children:
            if c.status == Status.RUNNING:
                c.halt(bb)
        self.on_halt(bb)
        self.status = Status.IDLE

    def on_halt(self, bb):
        pass

    def walk(self, depth=0):
        yield self, depth
        for c in self.children:
            yield from c.walk(depth + 1)


class Sequence(Node):
    kind = "sequence"

    def __init__(self, name, children, memory=False):
        super().__init__(name, children)
        self.memory = memory
        self._idx = 0

    def on_halt(self, bb):
        self._idx = 0

    def _tick(self, bb):
        start = self._idx if self.memory else 0
        for i in range(start, len(self.children)):
            s = self.children[i].tick(bb)
            if s == Status.RUNNING:
                self._idx = i
                self._halt_after(i, bb)
                return s
            if s == Status.FAILURE:
                self._idx = 0
                self._halt_after(i, bb)
                return s
        self._idx = 0
        return Status.SUCCESS

    def _halt_after(self, i, bb):
        for c in self.children[i + 1:]:
            if c.status == Status.RUNNING:
                c.halt(bb)


```

```python
class Fallback(Node):
    kind = "fallback"

    def _tick(self, bb):
        for i, c in enumerate(self.children):
            s = c.tick(bb)
            if s != Status.FAILURE:
                for later in self.children[i + 1:]:
                    if later.status == Status.RUNNING:
                        later.halt(bb)          # pre-emption of lower-priority branches
                return s
        return Status.FAILURE


class Condition(Node):
    kind = "condition"

    def __init__(self, name, fn: Callable[[dict], bool]):
        super().__init__(name)
        self.fn = fn

    def _tick(self, bb):
        return Status.SUCCESS if self.fn(bb) else Status.FAILURE


class Action(Node):
    kind = "action"

    def __init__(self, name, fn: Callable[[dict], Status], on_halt: Callable[[dict], None] | None = None):
        super().__init__(name)
        self.fn = fn
        self._on_halt = on_halt

    def _tick(self, bb):
        s = self.fn(bb)
        if s in (Status.RUNNING, Status.SUCCESS):
            bb["active_leaf"] = self.name
        return s

    def on_halt(self, bb):
        if self._on_halt:
            self._on_halt(bb)


class Inverter(Node):
    kind = "decorator"

    def _tick(self, bb):
        s = self.children[0].tick(bb)
        return {Status.SUCCESS: Status.FAILURE, Status.FAILURE: Status.SUCCESS}.get(s, s)


# ---------------------------------------------------------------------------
# Visualisation
# ---------------------------------------------------------------------------
def draw_tree(root: Node, path: str, title: str | None = None, active: set | None = None):
    """Render the tree left-to-right with matplotlib (rounded box = fallback/sequence/action,
    ellipse = condition). Root on the left, one row per leaf, so labels stay readable at page width.
    Data units equal inches."""
    import textwrap
    XSTEP, W, H, YSTEP = 1.42, 1.16, 0.36, 0.44
    pos, nxt = {}, [0]

    def layout(n, depth):
        if not n.children:
            pos[id(n)] = (depth * XSTEP, -nxt[0] * YSTEP)
            nxt[0] += 1
        else:
            for c in n.children:
                layout(c, depth + 1)
            ys = [pos[id(c)][1] for c in n.children]
            pos[id(n)] = (depth * XSTEP, sum(ys) / len(ys))

    layout(root, 0)
    nodes = list(root.walk())
    maxd = max(d for _, d in nodes)
    fig, ax = plt.subplots(figsize=(maxd * XSTEP + W + 0.3, YSTEP * nxt[0] + 0.3))
    colors = {"fallback": "#d9e8fb", "sequence": "#fde7c8", "condition": "#e3f4e1", "action": "#f7d9d9",
              "decorator": "#ece0f5"}
    gap = (XSTEP - W) / 2
    for n, d in nodes:
        for c in n.children:
            (x0, y0), (x1, y1) = pos[id(n)], pos[id(c)]
            xm = x0 + W / 2 + gap
            ax.plot([x0 + W / 2, xm, xm, x1 - W / 2], [y0, y0, y1, y1], color="#666", lw=0.8, zorder=1)
    for n, d in nodes:
        x, y = pos[id(n)]
        col = colors[n.kind]
        label = {"fallback": "?", "sequence": "->"}.get(n.kind, "")
        if n.kind == "condition":
            ax.add_patch(Ellipse((x, y), W, H, fc=col, ec="#2f7d32", lw=1.0, zorder=2))
        else:
            ec = "#b03a3a" if n.kind == "action" else "#555"
            ax.add_patch(FancyBboxPatch((x - W / 2, y - H / 2), W, H, boxstyle="round,pad=0.01",
                                        fc=col, ec=ec, lw=1.0, zorder=2))
        txt = ("\u2192 " if n.kind == "sequence" else "? " if n.kind == "fallback" else "") + n.name.replace("_", " ")
```

```
        ax.text(x, y, textwrap.fill(txt, 17), ha="center", va="center", fontsize=6.8, linespacing=0.95, zorder=3)
    ax.set_xlim(-W / 2 - 0.05, maxd * XSTEP + W / 2 + 0.05)
    ax.set_ylim(-(nxt[0] - 1) * YSTEP - H, H)
    ax.set_aspect("equal")
    ax.axis("off")
    if title:
        ax.set_title(title, fontsize=10)
    fig.tight_layout(pad=0.1)
    fig.savefig(path, dpi=200)
    plt.close(fig)
    return path
```

sdc/behavior/scenario.py

```
"""Frenet-frame traffic scenario used to compare behaviour trees with finite-state machines.

Road (s = longitudinal, d = lateral, d=0 own lane, d=+3.5 opposite lane):

    s=0 ..... slow lead vehicle ........ [no-overtake zone] roundabout ..... crosswalk ..... s=520

* A slow lead vehicle (turns off at a random junction) invites an overtake.
* Oncoming vehicles are only perceived within ``SENSE_RANGE`` -> an unseen vehicle can
  appear *during* the manoeuvre and force an abort.
* A roundabout with a random stream of circulating vehicles (ego must yield).
* A pedestrian who starts crossing when the ego gets close (ego must yield).
"""
from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np

from .bt import Action, Condition, Fallback, Inverter, Node, Sequence, Status

DT = 0.1
ROUTE_END = 520.0
CAR_L, CAR_W = 4.5, 1.9
LANE_W = 3.5
SENSE_RANGE = 90.0
RB_LINE, RB_CONF = 255.0, 272.0        # yield line, conflict point (s)
RB_ZONE = (RB_CONF - 4.5, RB_CONF + 4.5)
CW_S = 430.0                           # crosswalk
CW_LINE, CW_ZONE = CW_S - 7.0, (CW_S - 2.5 - 2.25, CW_S + 2.5 + 2.25)
NO_OVERTAKE = (RB_LINE - 70.0, RB_CONF + 20.0)
V_CRUISE, V_OVERTAKE, V_RB = 12.0, 15.0, 6.0
A_COMF, B_COMF, B_MAX = 1.8, 2.5, 8.0


# ---------------------------------------------------------------------------
@dataclass
class Scenario:
    v_lead: float
    s_lead0: float
    s_turn: float
    oncoming: list            # [(s0, v, t_spawn)]  vehicles driving in -s direction in the opposite lane
    rb_times: list            # absolute times at which a circulating vehicle is at the conflict point
    ped_trigger: float        # ego distance to the crosswalk at which the pedestrian starts to cross
    ped_dur: float
    v0: float = 8.0
    surprise: bool = False


def sample_scenario(rng: np.random.Generator, surprise_prob: float = 0.5) -> Scenario:
    oncoming = []
    t = rng.uniform(0, 8)
    for _ in range(4):                                  # stream of oncoming vehicles
        oncoming.append((520.0 + rng.uniform(0, 30), rng.uniform(9, 13), t))
        t += rng.uniform(5, 22)
    surprise = rng.random() < surprise_prob
    if surprise:                                        # an unseen vehicle that arrives mid-overtake
        oncoming.append((rng.uniform(205, 235) + 0.0, rng.uniform(10, 14), 0.0))
    rb, t = [], rng.uniform(15, 25)
    for _ in range(14):
        rb.append(t)
        t += rng.exponential(5.0) + 2.2
    return Scenario(v_lead=rng.uniform(3.0, 6.0), s_lead0=rng.uniform(45, 70), s_turn=rng.uniform(170, 330),
                    oncoming=oncoming, rb_times=rb, ped_trigger=rng.uniform(28, 55), ped_dur=rng.uniform(5, 9),
                    surprise=surprise)


# ---------------------------------------------------------------------------
class Sim:
    """Ground truth + perception."""

    def __init__(self, sc: Scenario):
        self.sc = sc
        self.t = 0.0
        self.s, self.v, self.a = 0.0, sc.v0, 0.0
        self.d = 0.0
        self.lc = None                                  # (t0, d0, d1, dur)
        self.s_lead = sc.s_lead0
        self.lead_alive = True
```

```
        self.onc = [dict(s=s0, v=v, t_spawn=ts) for (s0, v, ts) in sc.oncoming]
        self.ped_t0 = None
        self.collision = None
        self.log = []

    # ---- perception -----------------------------------------------------
    def perceive(self) -> dict:
        sc = self.sc
        lead = None
        if self.lead_alive and self.s_lead > self.s - CAR_L and self.s_lead - self.s < SENSE_RANGE:
            lead = (self.s_lead, sc.v_lead)
        onc = [(o["s"], o["v"]) for o in self.onc if self.t >= o["t_spawn"] and 0 < o["s"] - self.s < SENSE_RANGE]
        rb = [t_ for t_ in sc.rb_times if 0.0 <= t_ - self.t < 9.0 or (self.t - t_ < 1.2 and self.t >= t_)]
        ped = "none"
        if self.ped_t0 is not None:
            ped = "waiting" if self.t < self.ped_t0 else ("crossing" if self.t < self.ped_t0 + sc.ped_dur else "none")
        return dict(t=self.t, s=self.s, v=self.v, d=self.d, lead=lead, oncoming=onc, rb=rb, ped=ped,
                    ped_clear_t=(None if self.ped_t0 is None else self.ped_t0 + sc.ped_dur))

    # ---- dynamics -------------------------------------------------------
    def step(self, cmd: dict):
        sc = self.sc
        # lateral
        tgt = cmd.get("d_target", 0.0)
        if self.lc is None or abs(self.lc[2] - tgt) > 1e-6:
            self.lc = (self.t, self.d, tgt, 3.0)
        t0, d0, d1, dur = self.lc
        u = min(max((self.t - t0) / dur, 0.0), 1.0)
        self.d = d0 + (d1 - d0) * (u * u * u * (u * (6 * u - 15) + 10))
        # longitudinal: IDM towards v_des with an optional virtual obstacle (stop line / lead)
        v_des = cmd.get("v_des", V_CRUISE)
        a = A_COMF * (1.0 - (self.v / max(v_des, 0.1)) ** 4)
        for gap, vr in cmd.get("obstacles", []):          # (distance to obstacle front, obstacle speed)
            if gap is None:
                continue
            gap = max(gap, 0.05)
            s_star = 2.0 + max(0.0, self.v * 1.2 + self.v * (self.v - vr) / (2 * np.sqrt(A_COMF * B_COMF)))
            a = min(a, A_COMF * (1.0 - (self.v / max(v_des, 0.1)) ** 4 - (s_star / gap) ** 2))
        if cmd.get("emergency"):
            a = -B_MAX
        a = float(np.clip(a, -B_MAX, 2.5))
        a_prev = self.a
        self.a = a
        self.v = max(0.0, self.v + a * DT)
        self.s += self.v * DT
        # world
        self.t += DT
        if self.lead_alive:
            self.s_lead += sc.v_lead * DT
            if self.s_lead > sc.s_turn:
                self.lead_alive = False
        for o in self.onc:
            if self.t >= o["t_spawn"]:
                o["s"] -= o["v"] * DT
        if self.ped_t0 is None and CW_S - self.s < sc.ped_trigger:
            self.ped_t0 = self.t + 1.5                    # waits 1.5 s at the kerb, then crosses
        self._check_collisions()
        self.log.append((self.t, self.s, self.d, self.v, a, a_prev, cmd.get("tag", "")))

    def _check_collisions(self):
        if self.collision:
            return
        sc = self.sc
        if self.lead_alive and abs(self.s - self.s_lead) < CAR_L and abs(self.d) < CAR_W:
            self.collision = "lead"
        for o in self.onc:
            if self.t >= o["t_spawn"] and abs(self.s - o["s"]) < CAR_L and abs(self.d - LANE_W) < CAR_W:
                self.collision = "oncoming"
        for tr in sc.rb_times:
            if abs(self.t - tr) < 1.1 and RB_ZONE[0] <= self.s <= RB_ZONE[1]:
                self.collision = "roundabout"
        if self.ped_t0 is not None and self.ped_t0 <= self.t < self.ped_t0 + sc.ped_dur \
                and CW_ZONE[0] <= self.s <= CW_ZONE[1] and self.d < 2.5:
            self.collision = "pedestrian"

    @property
    def done(self):
        return self.s >= ROUTE_END or self.collision is not None or self.t > 140.0


# ---------------------------------------------------------------------------
# Shared decision primitives (used by both the BT and the FSM so the comparison is about structure)
# ---------------------------------------------------------------------------
def brake_dist(v, b=B_COMF):
    return v * v / (2 * b)


def ttc_ahead(p: dict) -> float:
    """Smallest time-to-collision with a perceived object in the lane currently occupied."""
    best = np.inf
    if p["lead"] is not None and abs(p["d"]) < 1.9:
        gap = p["lead"][0] - p["s"] - CAR_L
        cl = p["v"] - p["lead"][1]
        if cl > 0.1:
            best = min(best, max(gap, 0.0) / cl)
    if abs(p["d"] - LANE_W) < 1.9:
```

```
        for so, vo in p["oncoming"]:
            gap = so - p["s"] - CAR_L
            best = min(best, max(gap, 0.0) / (p["v"] + vo))
    return best


def overtake_time_needed(p: dict) -> float:
    """Time until the ego is back in its lane, ahead of the lead, if it starts/continues now."""
    sl, vl = p["lead"]
    rel = (sl - p["s"]) + CAR_L + 10.0
    t_pass = max(rel, 0.0) / max(V_OVERTAKE - vl, 1.0)
    t_out = 0.0 if p["d"] > 3.0 else 3.0 * (1 - p["d"] / LANE_W)
    return t_out + t_pass + 3.0


def oncoming_clear(p: dict, margin=14.0) -> bool:
    """No perceived oncoming vehicle can reach the overtaking corridor before we are back in lane."""
    if p["lead"] is None:
        return True
    tn = overtake_time_needed(p)
    for so, vo in p["oncoming"]:
        if so - p["s"] - (V_OVERTAKE + vo) * tn < margin:
            return False
    return True


def in_no_overtake_zone(p: dict, lookahead_t: float = 0.0) -> bool:
    end = p["s"] + overtake_time_needed(p) * V_OVERTAKE if p["lead"] else p["s"]
    return (end > NO_OVERTAKE[0] and p["s"] < NO_OVERTAKE[1]) or (p["s"] > CW_S - 90 and p["s"] < CW_S + 10)


def rb_gap_ok(p: dict) -> bool:
    """Can the ego traverse the conflict zone without meeting a circulating vehicle?"""
    dist = RB_ZONE[0] - 2.25 - p["s"]
    if p["s"] > RB_ZONE[1] + 2.25:
        return True
    v0 = max(p["v"], 0.3)
    # accelerate at 1.5 m/s^2 up to V_RB
    def t_for(x):
        if x <= 0:
            return 0.0
        x_acc = (V_RB ** 2 - v0 ** 2) / (2 * 1.5) if V_RB > v0 else 0.0
        if x <= x_acc:
            return (-v0 + np.sqrt(v0 ** 2 + 2 * 1.5 * x)) / 1.5
        return max((V_RB - v0) / 1.5, 0.0) + (x - x_acc) / V_RB
    t_in = t_for(dist)
    t_out = t_for(dist + (RB_ZONE[1] + 2.25) - (RB_ZONE[0] - 2.25))
    lo, hi = p["t"] + t_in - 1.2, p["t"] + t_out + 1.2
    return not any(lo - 1.1 < tr < hi + 1.1 for tr in p["rb"])


def stop_obstacle(line_s: float, p: dict):
    return (line_s - p["s"], 0.0)


def lead_obstacle(p: dict):
    if p["lead"] is None or abs(p["d"]) > 1.9:
        return (None, 0.0)
    return (p["lead"][0] - p["s"] - CAR_L, p["lead"][1])


def speed_limit(p: dict) -> float:
    v = V_CRUISE
    if RB_LINE - 40 < p["s"] < RB_ZONE[1] + 5:
        v = min(v, V_RB + 2.0)
    return v


# ---------------------------------------------------------------------------
# Behaviour tree
# ---------------------------------------------------------------------------
def build_bt() -> Node:
    def cmd(bb, **kw):
        c = bb["cmd"]
        c.update(kw)

    # ---- conditions ---------------------------------------------------
    c_imminent = Condition("imminent_collision", lambda bb: ttc_ahead(bb["p"]) < 1.6)
    c_ped = Condition("pedestrian_at_crosswalk",
                      lambda bb: bb["p"]["ped"] != "none" and bb["p"]["s"] < CW_ZONE[1] and CW_S - bb["p"]["s"] < 70)
    c_rb = Condition("approaching_roundabout", lambda bb: RB_LINE - 60 < bb["p"]["s"] < RB_ZONE[1] + 2.25)
    c_gap = Condition("roundabout_gap_ok",
                      lambda bb: rb_gap_ok(bb["p"]) or (RB_LINE - bb["p"]["s"] < brake_dist(bb["p"]["v"], 4.0) and bb["p"]["s"] >
      RB_LINE - 30))
    c_unsafe = Condition("overtake_unsafe", lambda bb: bb["p"]["d"] > 0.4 and bb["ot_phase"] is not None
                         and not (bb["p"]["lead"] is None or oncoming_clear(bb["p"], 6.0)))
    c_need = Condition("slow_lead_ahead", lambda bb: bb["ot_phase"] is not None or (
        bb["p"]["lead"] is not None and bb["p"]["lead"][1] < V_CRUISE - 3.0
        and 0 < bb["p"]["lead"][0] - bb["p"]["s"] < 40.0))
    c_legal = Condition("overtake_legal", lambda bb: not in_no_overtake_zone(bb["p"]) or bb["ot_phase"] is not None)
    c_feasible = Condition("oncoming_clear", lambda bb: bb["ot_phase"] is not None or oncoming_clear(bb["p"]))
    c_follow = Condition("lead_close", lambda bb: lead_obstacle(bb["p"])[0] is not None and lead_obstacle(bb["p"])[0] < 60)

    # ---- actions ------------------------------------------------------
    def a_emergency(bb):
        cmd(bb, emergency=True, tag="EMERGENCY")
```

```
        return Status.RUNNING

    def a_stop_ped(bb):
        p = bb["p"]
        cmd(bb, obstacles=[stop_obstacle(CW_LINE, p), lead_obstacle(p)], v_des=speed_limit(p), d_target=0.0, tag="YIELD_PED")
        return Status.RUNNING

    def a_roundabout_hold(bb):
        p = bb["p"]
        cmd(bb, obstacles=[stop_obstacle(RB_LINE, p), lead_obstacle(p)], v_des=V_RB + 2, d_target=0.0, tag="YIELD_RB")
        return Status.RUNNING

    def a_roundabout_go(bb):
        p = bb["p"]
        cmd(bb, v_des=V_RB if p["s"] > RB_LINE - 15 else V_RB + 2, d_target=0.0,
            obstacles=[lead_obstacle(p)], tag="RB_TRAVERSE")
        return Status.RUNNING

    def a_abort(bb):
        p = bb["p"]
        if p["lead"] is None or p["s"] - p["lead"][0] > 6.0:          # already ahead: merge back in front
            cmd(bb, d_target=0.0, v_des=V_OVERTAKE, tag="ABORT_MERGE_AHEAD")
        else:                                                          # fall back behind the lead
            cmd(bb, v_des=max(p["lead"][1] - 2.5, 0.5), d_target=0.0 if p["lead"][0] - p["s"] > CAR_L + 6 else p["d"],
                obstacles=[], tag="ABORT_FALL_BACK")
        if p["d"] < 0.3:
            bb["ot_phase"] = None
        else:
            bb["ot_phase"] = "abort"
        return Status.RUNNING

    def a_overtake(bb):
        p = bb["p"]
        sl, vl = p["lead"] if p["lead"] else (p["s"] - 100, 0.0)
        if bb["ot_phase"] in (None, "abort"):
            bb["ot_phase"] = "out"
        ph = bb["ot_phase"]
        ahead = p["s"] - sl
        if ph == "out" and p["d"] > 3.0:
            bb["ot_phase"] = ph = "pass"
        if ph in ("out", "pass") and ahead > 10.0 + 0:
            bb["ot_phase"] = ph = "in"
        if ph == "in" and p["d"] < 0.25:
            bb["ot_phase"] = None
            return Status.SUCCESS
        if ph == "out":
            cmd(bb, d_target=LANE_W, v_des=V_OVERTAKE, obstacles=[(lead_obstacle(p)[0], lead_obstacle(p)[1])] if p["d"] < 1.2 else [],
      tag="OT_OUT")
        elif ph == "pass":
            cmd(bb, d_target=LANE_W, v_des=V_OVERTAKE, obstacles=[], tag="OT_PASS")
        else:
            cmd(bb, d_target=0.0, v_des=V_OVERTAKE, obstacles=[], tag="OT_IN")
        return Status.RUNNING

    def a_follow(bb):
        p = bb["p"]
        cmd(bb, obstacles=[lead_obstacle(p)], v_des=speed_limit(p), d_target=0.0, tag="FOLLOW")
        return Status.RUNNING

    def a_cruise(bb):
        p = bb["p"]
        cmd(bb, v_des=speed_limit(p), d_target=0.0, obstacles=[], tag="CRUISE")
        return Status.RUNNING

    root = Fallback("drive", [
        Sequence("emergency", [c_imminent, Action("emergency_brake", a_emergency)]),
        Sequence("yield_pedestrian", [c_ped, Action("stop_at_crosswalk", a_stop_ped)]),
        Sequence("roundabout", [c_rb, Fallback("enter_or_yield", [
            Sequence("enter", [c_gap, Action("traverse_roundabout", a_roundabout_go)]),
            Action("hold_at_yield_line", a_roundabout_hold)])]),
        Sequence("abort_overtake", [c_unsafe, Action("abort_and_return", a_abort)]),
        Sequence("overtake", [c_need, c_legal, c_feasible, Action("overtake_manoeuvre", a_overtake)]),
        Sequence("follow", [c_follow, Action("follow_lead", a_follow)]),
        Action("cruise", a_cruise),
    ])
    return root


class BTController:
    name = "BT"

    def __init__(self):
        self.root = build_bt()
        self.bb = {"ot_phase": None, "cmd": {}}
        self.switches = 0
        self._last = None

    def decide(self, p: dict) -> dict:
        self.bb["p"] = p
        self.bb["cmd"] = {}
        self.root.tick(self.bb)
        c = self.bb["cmd"]
        if c.get("tag") != self._last:
            self.switches += 1
            self._last = c.get("tag")
        return c
```

```python


# ---------------------------------------------------------------------------
# Finite-state machine baselines
# ---------------------------------------------------------------------------
class FSMController:
    """Hand-written FSM.

    variant "basic": the transitions a designer writes for the nominal scenarios
                     (cruise <-> follow -> overtake out/pass/in, yield states from cruise/follow only)
                     plus a global emergency brake.
    variant "full" : adds the pre-emption transitions (abort overtake, yield from every state ...)
                     -- what is needed to match the reactivity of the BT.
    """

    def __init__(self, variant="basic", rb_recheck=True):
        self.variant, self.rb_recheck = variant, rb_recheck
        self.name = f"FSM-{variant}" + ("" if (variant == "basic" or rb_recheck) else "-norecheck")
        self.state = "CRUISE"
        self.switches = 0
        T = {
            "CRUISE": ["FOLLOW", "OT_OUT", "YIELD_RB", "YIELD_PED"],
            "FOLLOW": ["CRUISE", "OT_OUT", "YIELD_RB", "YIELD_PED"],
            "OT_OUT": ["OT_PASS", "OT_IN"], "OT_PASS": ["OT_IN"], "OT_IN": ["CRUISE"],
            "YIELD_RB": ["RB_TRAVERSE"], "RB_TRAVERSE": ["CRUISE"], "YIELD_PED": ["CRUISE"],
            "EMERGENCY": ["CRUISE"],
        }
        if variant == "full":
            T["OT_OUT"] += ["ABORT", "YIELD_RB", "YIELD_PED"]
            T["OT_PASS"] += ["ABORT", "YIELD_RB", "YIELD_PED"]
            T["OT_IN"] += ["YIELD_RB", "YIELD_PED"]
            T["ABORT"] = ["CRUISE", "YIELD_RB", "YIELD_PED"]
            T["RB_TRAVERSE"] += ["YIELD_PED"] + (["YIELD_RB"] if rb_recheck else [])
        for s in list(T):
            if s != "EMERGENCY":
                T[s] = T[s] + ["EMERGENCY"]
        self.T = T

    @property
    def n_transitions(self):
        return sum(len(v) for v in self.T.values())

    def _go(self, new):
        if new != self.state:
            assert new in self.T[self.state], f"illegal transition {self.state}->{new}"
            self.state = new

    def decide(self, p: dict) -> dict:
        st, full = self.state, self.variant == "full"
        before = st
        ph_ahead = None if p["lead"] is None else p["s"] - p["lead"][0]
        # global emergency
        if ttc_ahead(p) < 1.6 and st != "EMERGENCY":
            self._go("EMERGENCY")
        st = self.state
        ped = p["ped"] != "none" and p["s"] < CW_ZONE[1] and CW_S - p["s"] < 70
        rb = RB_LINE - 60 < p["s"] < RB_ZONE[1] + 2.25
        cmd = {}
        if st == "EMERGENCY":
            if ttc_ahead(p) > 2.5:
                self._go("CRUISE")
            cmd = dict(emergency=True)
        elif st in ("CRUISE", "FOLLOW"):
            if ped:
                self._go("YIELD_PED")
            elif rb:
                self._go("YIELD_RB")
            elif p["lead"] is not None and p["lead"][1] < V_CRUISE - 3 and 0 < p["lead"][0] - p["s"] < 40 \
                    and not in_no_overtake_zone(p) and oncoming_clear(p):
                self._go("OT_OUT")
            elif lead_obstacle(p)[0] is not None and lead_obstacle(p)[0] < 60:
                self._go("FOLLOW")
            else:
                self._go("CRUISE")
        elif st in ("OT_OUT", "OT_PASS", "OT_IN"):
            if full and ped:
                self._go("YIELD_PED")
            elif full and rb and st == "OT_IN":
                self._go("YIELD_RB")
            elif full and st in ("OT_OUT", "OT_PASS") and rb and p["d"] > 0.4:
                self._go("ABORT")
            elif full and st in ("OT_OUT", "OT_PASS") and p["lead"] is not None and not oncoming_clear(p, 6.0):
                self._go("ABORT")
            elif st == "OT_OUT" and p["d"] > 3.0:
                self._go("OT_PASS")
            elif st in ("OT_OUT", "OT_PASS") and (p["lead"] is None or ph_ahead > 10.0):
                self._go("OT_IN" if p["lead"] is not None else "OT_IN")
            elif st == "OT_IN" and p["d"] < 0.25:
                self._go("CRUISE")
        elif st == "ABORT":
            if p["d"] < 0.25:
                self._go("CRUISE")
        elif st == "YIELD_PED":
            if not ped:
                self._go("CRUISE")
        elif st == "YIELD_RB":
```

```
            if rb_gap_ok(p) or (RB_LINE - p["s"] < brake_dist(p["v"], 4.0) and p["s"] > RB_LINE - 30):
                self._go("RB_TRAVERSE")
            elif full and ped:
                self._go("YIELD_PED")
        elif st == "RB_TRAVERSE":
            committed = RB_LINE - p["s"] < brake_dist(p["v"], 4.0) and p["s"] > RB_LINE - 30
            if p["s"] > RB_ZONE[1] + 2.25:
                self._go("CRUISE")
            elif full and ped and p["s"] > RB_ZONE[1]:
                self._go("YIELD_PED")
            elif full and self.rb_recheck and not (rb_gap_ok(p) or committed):
                self._go("YIELD_RB")
        st = self.state
        if st != "EMERGENCY":
            cmd = self._command(st, p)
        cmd["tag"] = st
        if st != before:
            self.switches += 1
        return cmd

    def _command(self, st, p):
        if st == "CRUISE":
            return dict(v_des=speed_limit(p), d_target=0.0, obstacles=[])
        if st == "FOLLOW":
            return dict(v_des=speed_limit(p), d_target=0.0, obstacles=[lead_obstacle(p)])
        if st == "YIELD_PED":
            return dict(v_des=speed_limit(p), d_target=0.0, obstacles=[stop_obstacle(CW_LINE, p), lead_obstacle(p)])
        if st == "YIELD_RB":
            return dict(v_des=V_RB + 2, d_target=0.0, obstacles=[stop_obstacle(RB_LINE, p), lead_obstacle(p)])
        if st == "RB_TRAVERSE":
            return dict(v_des=V_RB, d_target=0.0, obstacles=[lead_obstacle(p)])
        if st == "OT_OUT":
            return dict(v_des=V_OVERTAKE, d_target=LANE_W, obstacles=[lead_obstacle(p)] if p["d"] < 1.2 else [])
        if st == "OT_PASS":
            return dict(v_des=V_OVERTAKE, d_target=LANE_W, obstacles=[])
        if st == "OT_IN":
            return dict(v_des=V_OVERTAKE, d_target=0.0, obstacles=[])
        if st == "ABORT":
            if p["lead"] is None or p["s"] - p["lead"][0] > 6.0:
                return dict(v_des=V_OVERTAKE, d_target=0.0, obstacles=[])
            return dict(v_des=max(p["lead"][1] - 2.5, 0.5), d_target=0.0 if p["lead"][0] - p["s"] > CAR_L + 6 else p["d"], obstacles
      =[])
        raise KeyError(st)


# ---------------------------------------------------------------------------
def run_scenario(sc: Scenario, ctrl):
    sim = Sim(sc)
    while not sim.done:
        sim.step(ctrl.decide(sim.perceive()))
    log = np.array([r[:6] for r in sim.log])
    tags = [r[6] for r in sim.log]
    reached = sim.s >= ROUTE_END
    return dict(sim=sim, log=log, tags=tags, collision=sim.collision, reached=reached, time=sim.t,
                switches=ctrl.switches,
                hard_brakes=int(np.sum((log[1:, 4] < -4.0) & (log[:-1, 4] >= -4.0))),
                max_jerk=float(np.max(np.abs(np.diff(log[:, 4])) / DT)) if len(log) > 2 else 0.0,
                overtook=any(t.startswith("OT_PASS") for t in tags))
```

sdc/planning/dubins.py

```
"""Dubins shortest paths (forward-only, bounded curvature) -- the analytic "shot" used by Hybrid A*.

Reverse-driving connections are obtained from the same routine by computing the forward path
in the opposite direction (q1 -> q0) and traversing it backwards (time reversal).
"""
from __future__ import annotations

import numpy as np

TWO_PI = 2.0 * np.pi


def _m(a):
    return a % TWO_PI


def _words(alpha, beta, d):
    """Candidate (t, p, q) for all six words in the normalised problem; None if infeasible."""
    sa, sb, ca, cb = np.sin(alpha), np.sin(beta), np.cos(alpha), np.cos(beta)
    c_ab = np.cos(alpha - beta)
    out = {}
    # LSL
    p2 = 2 + d * d - 2 * c_ab + 2 * d * (sa - sb)
    if p2 >= 0:
        t1 = np.arctan2(cb - ca, d + sa - sb)
        out["LSL"] = (_m(-alpha + t1), np.sqrt(p2), _m(beta - t1))
    # RSR
    p2 = 2 + d * d - 2 * c_ab + 2 * d * (sb - sa)
    if p2 >= 0:
        t1 = np.arctan2(ca - cb, d - sa + sb)
        out["RSR"] = (_m(alpha - t1), np.sqrt(p2), _m(-beta + t1))
```

```
    # LSR
    p2 = -2 + d * d + 2 * c_ab + 2 * d * (sa + sb)
    if p2 >= 0:
        p = np.sqrt(p2)
        t1 = np.arctan2(-ca - cb, d + sa + sb) - np.arctan2(-2.0, p)
        out["LSR"] = (_m(-alpha + t1), p, _m(-_m(beta) + t1))
    # RSL
    p2 = d * d - 2 + 2 * c_ab - 2 * d * (sa + sb)
    if p2 >= 0:
        p = np.sqrt(p2)
        t1 = np.arctan2(ca + cb, d - sa - sb) - np.arctan2(2.0, p)
        out["RSL"] = (_m(alpha - t1), p, _m(beta - t1))
    # RLR
    t0 = (6 - d * d + 2 * c_ab + 2 * d * (sa - sb)) / 8.0
    if abs(t0) <= 1:
        p = _m(TWO_PI - np.arccos(t0))
        t = _m(alpha - np.arctan2(ca - cb, d - sa + sb) + _m(p / 2))
        out["RLR"] = (t, p, _m(alpha - beta - t + _m(p)))
    # LRL
    t0 = (6 - d * d + 2 * c_ab + 2 * d * (sb - sa)) / 8.0
    if abs(t0) <= 1:
        p = _m(TWO_PI - np.arccos(t0))
        t = _m(-alpha - np.arctan2(ca - cb, d + sa - sb) + p / 2)
        out["LRL"] = (t, p, _m(_m(beta) - alpha - t + _m(p)))
    return out


def dubins_shortest(q0, q1, rho):
    """Return (length, word, (t,p,q)) of the shortest Dubins path, or None."""
    dx, dy = q1[0] - q0[0], q1[1] - q0[1]
    D = np.hypot(dx, dy)
    d = D / rho
    theta = _m(np.arctan2(dy, dx)) if D > 1e-12 else 0.0
    alpha, beta = _m(q0[2] - theta), _m(q1[2] - theta)
    best = None
    for w, (t, p, q) in _words(alpha, beta, d).items():
        L = (t + p + q) * rho
        if best is None or L < best[0]:
            best = (L, w, (t, p, q))
    return best


def dubins_sample(q0, rho, word, tpq, step):
    """Sample poses (x,y,theta) every <= ``step`` metres along the path (inclusive of both ends)."""
    x, y, th = float(q0[0]), float(q0[1]), float(q0[2])
    out = [np.array([[x, y, th]])]
    for typ, norm_len in zip(word, tpq):
        length = norm_len * rho                          # t, p, q are normalised by rho
        if length < 1e-9:
            continue
        n = max(int(np.ceil(length / step)), 1)
        sd = np.linspace(0.0, length, n + 1)[1:]
        if typ == "S":
            seg = np.stack([x + sd * np.cos(th), y + sd * np.sin(th), np.full(n, th)], axis=1)
        else:
            sgn = 1.0 if typ == "L" else -1.0
            dth = sgn * sd / rho
            seg = np.stack([x + sgn * rho * (np.sin(th + dth) - np.sin(th)),
                            y - sgn * rho * (np.cos(th + dth) - np.cos(th)), th + dth], axis=1)
        x, y, th = seg[-1]
        out.append(seg)
    pts = np.vstack(out)
    pts[:, 2] = (pts[:, 2] + np.pi) % TWO_PI - np.pi
    return pts


def dubins_path(q0, q1, rho, step=0.3):
    """Forward-only Dubins path poses and its length (None if no path)."""
    r = dubins_shortest(q0, q1, rho)
    if r is None:
        return None
    L, w, tpq = r
    return dubins_sample(q0, rho, w, tpq, step), L


def reverse_dubins_path(q0, q1, rho, step=0.3):
    """Path from q0 to q1 driven in *reverse* (headings are those of the car body).

    Driving backwards from q0 to q1 is the time-reversal of driving *forwards* from q1 to q0,
    so we compute the forward Dubins path q1 -> q0 and traverse it in the opposite order.
    """
    r = dubins_path(q1, q0, rho, step)
    if r is None:
        return None
    pts, L = r
    return pts[::-1].copy(), L
```

sdc/planning/hybrid_astar.py

```
"""Hybrid A* (Dolgov et al., 2010) for car-like robots, plus a grid-A* baseline.

* Search state  : continuous (x, y, theta) stored in a discretised (0.5 m, 5 deg) grid -> every
```

```
                expanded node keeps its *exact* continuous pose, so paths are kinematically feasible.
* Motion model  : kinematic bicycle, 5 steering angles x {forward, reverse}, 1.5 m arcs.
* Cost          : arc length, reverse and direction-switch penalties, steering and steering-change penalties.
* Heuristic     : holonomic-with-obstacles distance (Dijkstra from the goal over the clearance-inflated grid).
* Analytic shot : Dubins connections (forward and reverse) to the goal, collision-checked.
* Collision     : 3-disc footprint on a Euclidean distance transform of the occupancy grid.
"""
from __future__ import annotations

import heapq
import time
from dataclasses import dataclass, field

import numpy as np
from matplotlib.path import Path as MplPath
from scipy.ndimage import distance_transform_edt

from .dubins import dubins_path, dubins_shortest, reverse_dubins_path


# ---------------------------------------------------------------------------
@dataclass
class Vehicle:
    wheelbase: float = 2.7
    length: float = 4.5
    width: float = 1.9
    rear_overhang: float = 0.9
    delta_max: float = 0.52          # rad

    @property
    def rho(self) -> float:
        return self.wheelbase / np.tan(self.delta_max)

    @property
    def kappa_max(self) -> float:
        return 1.0 / self.rho

    def disc_offsets_radius(self, n=3):
        """Centres (along the heading, from the rear axle) and common radius of n covering discs."""
        seg = self.length / n
        offs = -self.rear_overhang + seg * (np.arange(n) + 0.5)
        r = np.hypot(seg / 2, self.width / 2)
        return offs, r

    def corners(self, pose):
        x, y, th = pose
        c, s = np.cos(th), np.sin(th)
        xs = np.array([-self.rear_overhang, self.length - self.rear_overhang] * 2)
        ys = np.array([-self.width / 2, -self.width / 2, self.width / 2, self.width / 2])
        order = [0, 1, 3, 2]
        return np.stack([x + c * xs[order] - s * ys[order], y + s * xs[order] + c * ys[order]], axis=1)


class OccGrid:
    def __init__(self, x0, y0, x1, y1, res=0.1):
        self.res, self.x0, self.y0 = res, x0, y0
        self.nx, self.ny = int(np.ceil((x1 - x0) / res)), int(np.ceil((y1 - y0) / res))
        self.occ = np.zeros((self.nx, self.ny), bool)
        self.occ[0, :] = self.occ[-1, :] = self.occ[:, 0] = self.occ[:, -1] = True   # outer wall
        self._edt = None

    def add_polygon(self, poly):
        poly = np.asarray(poly, float)
        lo = np.floor((poly.min(0) - [self.x0, self.y0]) / self.res).astype(int) - 1
        hi = np.ceil((poly.max(0) - [self.x0, self.y0]) / self.res).astype(int) + 1
        lo, hi = np.maximum(lo, 0), np.minimum(hi, [self.nx - 1, self.ny - 1])
        ii, jj = np.meshgrid(np.arange(lo[0], hi[0] + 1), np.arange(lo[1], hi[1] + 1), indexing="ij")
        cen = np.stack([self.x0 + (ii + 0.5) * self.res, self.y0 + (jj + 0.5) * self.res], axis=-1).reshape(-1, 2)
        inside = MplPath(poly).contains_points(cen).reshape(ii.shape)
        self.occ[ii[inside], jj[inside]] = True
        self._edt = None

    def add_rect(self, x0, y0, x1, y1):
        self.add_polygon([[x0, y0], [x1, y0], [x1, y1], [x0, y1]])

    @property
    def edt(self):
        if self._edt is None:
            self._edt = (distance_transform_edt(~self.occ) * self.res).astype(np.float32)
        return self._edt

    def occupied(self, x, y):
        i = np.floor((np.asarray(x) - self.x0) / self.res).astype(int)
        j = np.floor((np.asarray(y) - self.y0) / self.res).astype(int)
        ok = (i >= 0) & (i < self.nx) & (j >= 0) & (j < self.ny)
        return np.where(ok, self.occ[np.clip(i, 0, self.nx - 1), np.clip(j, 0, self.ny - 1)], True)

    def dist(self, x, y):
        i = np.floor((np.asarray(x) - self.x0) / self.res).astype(int)
        j = np.floor((np.asarray(y) - self.y0) / self.res).astype(int)
        ok = (i >= 0) & (i < self.nx) & (j >= 0) & (j < self.ny)
        d = self.edt[np.clip(i, 0, self.nx - 1), np.clip(j, 0, self.ny - 1)]
        return np.where(ok, d, 0.0)


class FootprintChecker:
```

```
    """Collision check of the oriented car rectangle.

    The rectangle is sampled on a regular lattice (``spacing`` m, border included); a pose is free
    when every sample keeps at least ``margin`` clearance on the Euclidean distance field.  This is much
    tighter than a 3-disc cover, which over-extends ~0.5 m beyond the bumpers and makes tight slots
    infeasible.  ``exact_free`` re-checks a path against raw occupancy with a very dense raster.
    """

    def __init__(self, grid: OccGrid, veh: Vehicle, margin=0.08, spacing=0.3):
        self.g, self.veh, self.margin = grid, veh, margin
        nl = int(np.ceil(veh.length / spacing)) + 1
        nw = int(np.ceil(veh.width / spacing)) + 1
        lx = np.linspace(-veh.rear_overhang, veh.length - veh.rear_overhang, nl)
        ly = np.linspace(-veh.width / 2, veh.width / 2, nw)
        X, Y = np.meshgrid(lx, ly, indexing="ij")
        self.px, self.py = X.ravel(), Y.ravel()
        lx2 = np.arange(-veh.rear_overhang, veh.length - veh.rear_overhang + 1e-9, 0.04)
        ly2 = np.arange(-veh.width / 2, veh.width / 2 + 1e-9, 0.04)
        X2, Y2 = np.meshgrid(lx2, ly2, indexing="ij")
        self.dx, self.dy = X2.ravel(), Y2.ravel()

    def _world(self, x, y, th, px, py):
        c, s = np.cos(th)[..., None], np.sin(th)[..., None]
        return x[..., None] + c * px - s * py, y[..., None] + s * px + c * py

    def free(self, x, y, th):
        """Vectorised over pose arrays -> bool array (True = collision free)."""
        x, y, th = np.asarray(x, float), np.asarray(y, float), np.asarray(th, float)
        wx, wy = self._world(x, y, th, self.px, self.py)
        return np.all(self.g.dist(wx, wy) > self.margin, axis=-1)

    def exact_free(self, x, y, th):
        """Dense raster against the raw occupancy grid (no clearance)."""
        x, y, th = np.atleast_1d(x), np.atleast_1d(y), np.atleast_1d(th)
        out = np.ones(len(x), bool)
        for k in range(len(x)):
            wx, wy = self._world(x[k:k + 1], y[k:k + 1], th[k:k + 1], self.dx, self.dy)
            out[k] = not self.g.occupied(wx, wy).any()
        return out


# ---------------------------------------------------------------------------
@dataclass
class PlanResult:
    success: bool
    path: np.ndarray | None = None       # (N,4): x, y, theta, direction (+1 fwd / -1 rev)
    time_s: float = 0.0
    expansions: int = 0
    cost: float = np.inf
    used_shot: bool = False


class HybridAStar:
    def __init__(self, grid: OccGrid, veh: Vehicle = Vehicle(), xy_res=0.5, n_theta=72, steps=(0.6, 1.5), sub=0.25,
                 n_steer=5, reverse_pen=2.0, switch_pen=6.0, steer_pen=0.4, steer_chg_pen=1.5,
                 shot_every=6, max_exp=40000, margin=0.08, nh_heuristic=True, h_weight=1.0, nh_radius=1e9):
        self.g, self.veh = grid, veh
        self.chk = FootprintChecker(grid, veh, margin=margin)
        self.xy_res, self.n_theta, self.steps, self.sub = xy_res, n_theta, tuple(steps), sub
        self.reverse_pen, self.switch_pen, self.steer_pen, self.steer_chg_pen = reverse_pen, switch_pen, steer_pen, steer_chg_pen
        self.shot_every, self.max_exp = shot_every, max_exp
        self.nh_heuristic, self.h_weight, self.nh_radius = nh_heuristic, h_weight, nh_radius
        self.steers = np.linspace(-veh.delta_max, veh.delta_max, n_steer)
        self._prims()

    # primitives in the local frame, one group per arc length: (A, K, 3)
    def _prims(self):
        self.groups = []
        for step in self.steps:
            ks = max(int(round(step / self.sub)), 1)
            d = np.arange(1, ks + 1) * (step / ks)
            A, act = [], []
            for dirn in (+1, -1):
                for si, dl in enumerate(self.steers):
                    kap = np.tan(dl) / self.veh.wheelbase
                    sd = dirn * d
                    if abs(kap) < 1e-9:
                        lx, ly, lt = sd, np.zeros_like(sd), np.zeros_like(sd)
                    else:
                        lx, ly, lt = np.sin(kap * sd) / kap, (1 - np.cos(kap * sd)) / kap, kap * sd
                    A.append(np.stack([lx, ly, lt], axis=-1))
                    act.append((dirn, si))
            self.groups.append((np.array(A), act, step))

    # ------------------------------------------------------------------
    def _key(self, x, y, th):
        return (int(np.floor(x / self.xy_res)), int(np.floor(y / self.xy_res)),
                int(np.floor(((th % (2 * np.pi)) / (2 * np.pi)) * self.n_theta)) % self.n_theta)

    def _heuristic_field(self, goal):
        """Dijkstra over the grid (8-connected) from the goal, cells blocked if closer than the disc radius."""
        res = 0.25
        nx, ny = int(self.g.nx * self.g.res / res), int(self.g.ny * self.g.res / res)
        xs = self.g.x0 + (np.arange(nx) + 0.5) * res
        ys = self.g.y0 + (np.arange(ny) + 0.5) * res
        X, Y = np.meshgrid(xs, ys, indexing="ij")
```

```
        free = self.g.dist(X, Y) > self.veh.width / 2 + 0.05
        import scipy.sparse as sp
        from scipy.sparse.csgraph import dijkstra
        idx = -np.ones((nx, ny), int)
        idx[free] = np.arange(free.sum())
        rows, cols, w = [], [], []
        for dx, dy in ((1, 0), (0, 1), (1, 1), (1, -1)):
            a = idx[0:nx - dx, max(0, -dy):ny - max(0, dy)]
            b = idx[dx:nx, max(0, dy):ny - max(0, -dy)]
            m = (a >= 0) & (b >= 0)
            rows.append(a[m]); cols.append(b[m]); w.append(np.full(m.sum(), res * np.hypot(dx, dy)))
        r, c, ww = np.concatenate(rows), np.concatenate(cols), np.concatenate(w)
        n = int(free.sum())
        G = sp.coo_matrix((ww, (r, c)), shape=(n, n)).tocsr()
        G = G + G.T
        gi = int(np.clip((goal[0] - self.g.x0) / res, 0, nx - 1)), int(np.clip((goal[1] - self.g.y0) / res, 0, ny - 1))
        if idx[gi] < 0:                                  # snap to nearest free cell
            fx, fy = np.where(free)
            k = np.argmin((fx - gi[0]) ** 2 + (fy - gi[1]) ** 2)
            gi = (fx[k], fy[k])
        dist = dijkstra(G, directed=False, indices=int(idx[gi]))
        field = np.full((nx, ny), np.inf)
        field[free] = dist
        return field, res

    def _h(self, field, x, y):
        f, res = field
        i = int(np.clip((x - self.g.x0) / res, 0, f.shape[0] - 1))
        j = int(np.clip((y - self.g.y0) / res, 0, f.shape[1] - 1))
        v = f[i, j]
        if not np.isfinite(v):                           # in a blocked cell: use the nearest finite neighbour
            sl = f[max(0, i - 2):i + 3, max(0, j - 2):j + 3]
            v = sl[np.isfinite(sl)].min() + 0.5 if np.isfinite(sl).any() else 1e4
        return v

    def _h_full(self, hf, x, y, th, goal):
        """max( holonomic-with-obstacles , non-holonomic-without-obstacles ) -- Dolgov's pair of heuristics.

        The non-holonomic term is the shorter of the forward Dubins length and the reverse Dubins length
        (weighted by the reverse penalty), so it also guides reversing manoeuvres.
        """
        h = self._h(hf, x, y)
        if self.nh_heuristic and np.hypot(goal[0] - x, goal[1] - y) <= self.nh_radius:   # Dubins term only near the goal
            f = dubins_shortest((x, y, th), goal, self.veh.rho)
            r = dubins_shortest(goal, (x, y, th), self.veh.rho)          # reverse drive = time-reversed forward drive
            c = min(f[0] if f else np.inf, (r[0] * (1.0 + self.reverse_pen)) if r else np.inf)
            if np.isfinite(c):
                h = max(h, c)
        return self.h_weight * h

    # ------------------------------------------------------------------
    def _try_shot(self, node, goal):
        q0 = (node[0], node[1], node[2])
        best = None
        for fn, dirn in ((dubins_path, +1), (reverse_dubins_path, -1)):
            r = fn(q0, goal, self.veh.rho, 0.3)
            if r is None:
                continue
            pts, L = r
            if L > 60:
                continue
            if self.chk.free(pts[:, 0], pts[:, 1], pts[:, 2]).all():
                cost = L * (1.0 if dirn > 0 else 1.0 + self.reverse_pen)
                if best is None or cost < best[0]:
                    best = (cost, pts, dirn)
        return best

    def plan(self, start, goal, time_limit=20.0) -> PlanResult:
        t0 = time.time()
        start, goal = np.array(start, float), np.array(goal, float)
        if not self.chk.free(*start)[()] or not self.chk.free(*goal)[()]:
            return PlanResult(False, time_s=time.time() - t0)
        hf = self._heuristic_field(goal)
        # node: (x,y,th,g,dir,steer_idx,parent,primitive_pts)
        nodes = [(start[0], start[1], start[2], 0.0, 0, 2, -1, None)]
        openq = [(self._h_full(hf, start[0], start[1], start[2], goal), 0)]
        best_g = {self._key(*start): 0.0}
        closed = set()
        n_exp, used_shot = 0, False
        final = None
        while openq and n_exp < self.max_exp and time.time() - t0 < time_limit:
            f, ni = heapq.heappop(openq)
            x, y, th, g, dirn, sidx, par, _ = nodes[ni]
            k = self._key(x, y, th)
            if k in closed:
                continue
            closed.add(k)
            n_exp += 1
            dgoal = np.hypot(goal[0] - x, goal[1] - y)
            dth = abs((th - goal[2] + np.pi) % (2 * np.pi) - np.pi)
            if dgoal < 0.30 and dth < 0.09:           # ~5 deg; the exact Dubins shot is preferred when it exists
                final = (ni, None)
                break
            if n_exp % self.shot_every == 0 or (dgoal < 15.0 and n_exp % 2 == 0):
                shot = self._try_shot((x, y, th), goal)
                if shot is not None:
```

```
                    final = (ni, shot)
                    used_shot = True
                    break
            # vectorised expansion, one batch per primitive length
            c, s_ = np.cos(th), np.sin(th)
            for P, act, step in self.groups:
                X = x + c * P[..., 0] - s_ * P[..., 1]
                Y = y + s_ * P[..., 0] + c * P[..., 1]
                T = th + P[..., 2]
                ok = self.chk.free(X, Y, T).all(axis=1)
                for a in np.where(ok)[0]:
                    d2, si2 = act[a]
                    nx_, ny_, nt_ = X[a, -1], Y[a, -1], T[a, -1]
                    nk = self._key(nx_, ny_, nt_)
                    if nk in closed:
                        continue
                    cost = step * (1.0 if d2 > 0 else 1.0 + self.reverse_pen)
                    cost += self.steer_pen * abs(self.steers[si2]) + self.steer_chg_pen * abs(self.steers[si2] - self.steers[sidx])
                    if dirn != 0 and d2 != dirn:
                        cost += self.switch_pen
                    g2 = g + cost
                    if g2 < best_g.get(nk, np.inf):
                        best_g[nk] = g2
                        nodes.append((nx_, ny_, nt_, g2, d2, si2, ni, np.stack([X[a], Y[a], T[a]], axis=1)))
                        heapq.heappush(openq, (g2 + self._h_full(hf, nx_, ny_, nt_, goal), len(nodes) - 1))
        dt = time.time() - t0
        if final is None:
            return PlanResult(False, time_s=dt, expansions=n_exp)
        # backtrack
        ni, shot = final
        segs, cur = [], ni
        while cur != -1:
            n = nodes[cur]
            if n[7] is not None:
                segs.append(np.column_stack([n[7], np.full(len(n[7]), n[4])]))
            cur = n[6]
        segs = segs[::-1]
        path = np.vstack([[start[0], start[1], start[2], 0.0]] + segs) if segs else np.array([[*start, 0.0]])
        if shot is not None:
            cost, pts, dirn = shot
            path = np.vstack([path, np.column_stack([pts[1:], np.full(len(pts) - 1, dirn)])])
        path[0, 3] = path[1, 3] if len(path) > 1 else 1.0
        path[:, 2] = (path[:, 2] + np.pi) % (2 * np.pi) - np.pi
        return PlanResult(True, path, dt, n_exp, nodes[ni][3], used_shot)


# ---------------------------------------------------------------------------
def grid_astar(grid: OccGrid, start_xy, goal_xy, clearance: float, res: float = 0.25):
    """Classic 8-connected grid A* for a *point* robot on a clearance-inflated grid."""
    nx, ny = int(grid.nx * grid.res / res), int(grid.ny * grid.res / res)
    xs = grid.x0 + (np.arange(nx) + 0.5) * res
    ys = grid.y0 + (np.arange(ny) + 0.5) * res
    X, Y = np.meshgrid(xs, ys, indexing="ij")
    free = grid.dist(X, Y) > clearance
    si = (int((start_xy[0] - grid.x0) / res), int((start_xy[1] - grid.y0) / res))
    gi = (int((goal_xy[0] - grid.x0) / res), int((goal_xy[1] - grid.y0) / res))
    if not (free[si] and free[gi]):
        return None, 0
    openq = [(0.0, si)]
    g = {si: 0.0}
    par = {}
    closed = set()
    moves = [(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (1, -1), (-1, 1), (-1, -1)]
    n = 0
    while openq:
        _, cur = heapq.heappop(openq)
        if cur in closed:
            continue
        closed.add(cur)
        n += 1
        if cur == gi:
            break
        for dx, dy in moves:
            nb = (cur[0] + dx, cur[1] + dy)
            if not (0 <= nb[0] < nx and 0 <= nb[1] < ny) or not free[nb] or nb in closed:
                continue
            g2 = g[cur] + res * np.hypot(dx, dy)
            if g2 < g.get(nb, np.inf):
                g[nb] = g2
                par[nb] = cur
                heapq.heappush(openq, (g2 + res * np.hypot(gi[0] - nb[0], gi[1] - nb[1]), nb))
    if gi not in par and gi != si:
        return None, n
    path, cur = [gi], gi
    while cur != si:
        cur = par[cur]
        path.append(cur)
    path = np.array(path[::-1], float)
    return np.stack([grid.x0 + (path[:, 0] + 0.5) * res, grid.y0 + (path[:, 1] + 0.5) * res], axis=1), n


def chaikin(path: np.ndarray, iters: int = 3) -> np.ndarray:
    for _ in range(iters):
        q = 0.75 * path[:-1] + 0.25 * path[1:]
        r = 0.25 * path[:-1] + 0.75 * path[1:]
        path = np.vstack([path[:1], np.stack([q, r], axis=1).reshape(-1, 2), path[-1:]])
```

```
    return path


def path_metrics(xy: np.ndarray, theta: np.ndarray | None = None, kappa_max: float | None = None):
    """Length, max |curvature|, number of heading jumps > 20 deg between consecutive samples (corners),
    and fraction of the path exceeding the curvature limit.  ``theta`` only used for cusp handling."""
    d = np.diff(xy, axis=0)
    seg = np.hypot(d[:, 0], d[:, 1])
    keep = seg > 1e-6
    d, seg = d[keep], seg[keep]
    hd = np.arctan2(d[:, 1], d[:, 0])
    dh = np.abs((np.diff(hd) + np.pi) % (2 * np.pi) - np.pi)
    ds = 0.5 * (seg[:-1] + seg[1:])
    kap = dh / np.maximum(ds, 1e-6)
    return dict(length=float(seg.sum()), max_kappa=float(kap.max()) if len(kap) else 0.0,
                corners=int(np.sum(dh > np.radians(20))),
                frac_over=float(np.mean(kap > kappa_max * 1.05)) if kappa_max else np.nan)
```

sdc/planning/scenarios.py

```
"""Tight-space planning scenarios (metres).  Poses are the *rear axle* centre (x, y, theta)."""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from .hybrid_astar import OccGrid, Vehicle


@dataclass
class PlanScenario:
    name: str
    grid: OccGrid
    start: np.ndarray
    goal: np.ndarray
    cars: list            # obstacle polygons (for drawing)
    start_jitter: tuple = (1.0, 0.4, 0.15)


def _car_poly(cx, cy, th, L=4.5, W=1.9):
    c, s = np.cos(th), np.sin(th)
    p = np.array([[-L / 2, -W / 2], [L / 2, -W / 2], [L / 2, W / 2], [-L / 2, W / 2]])
    return np.stack([cx + c * p[:, 0] - s * p[:, 1], cy + s * p[:, 0] + c * p[:, 1]], axis=1)


def perpendicular_parking(veh: Vehicle = Vehicle()) -> PlanScenario:
    g = OccGrid(0, 0, 40, 22)
    cars = []
    slot_w, y_lo, y_hi = 2.7, 12.5, 17.5
    target = 6
    for k in range(10):
        xc = 4.0 + slot_w * (k + 0.5)
        if k == target:
            continue
        poly = _car_poly(xc, 15.0, np.pi / 2)
        g.add_polygon(poly); cars.append(poly)
    g.add_rect(0, 17.6, 40, 22)                 # building behind the slots
    g.add_rect(0, 0, 40, 5.6)                   # opposite kerb / wall: aisle y in [5.6, 12.5]
    xc = 4.0 + slot_w * (target + 0.5)
    goal = np.array([xc, 15.0 - (veh.length / 2 - veh.rear_overhang), np.pi / 2])
    return PlanScenario("Perpendicular parking", g, np.array([3.0, 8.4, 0.0]), goal, cars)


def parallel_parking(veh: Vehicle = Vehicle(), slot: float = 6.2) -> PlanScenario:
    """Parallel slot of ``slot`` = 6.2 m by default (car 4.5 m): parked cars and the goal sit 0.2 m off the kerb."""
    g = OccGrid(0, 0, 40, 12)
    cars = []
    yc = 0.3 + 0.2 + 0.95                       # kerb top + 0.2 m + half width
    for xc in (11.0, 11.0 + 4.5 + slot):
        poly = _car_poly(xc, yc, 0.0)
        g.add_polygon(poly); cars.append(poly)
    g.add_rect(0, 0, 40, 0.3)                   # kerb
    g.add_rect(0, 8.0, 40, 12)                  # opposite side of the street (lane y in [0.3, 8.0])
    xc = 11.0 + 2.25 + slot / 2                     # slot centre
    goal = np.array([xc - (veh.length / 2 - veh.rear_overhang), yc, 0.0])
    return PlanScenario("Parallel parking", g, np.array([20.0, 5.0, 0.0]), goal, cars, (1.5, 0.3, 0.1))


def k_turn(veh: Vehicle = Vehicle()) -> PlanScenario:
    g = OccGrid(0, 0, 36, 12)
    g.add_rect(0, 0, 36, 2.0)
    g.add_rect(0, 8.6, 36, 12)                  # street width 6.6 m < 2 * rho + width: needs a 3-point turn
    g.add_rect(33.0, 0, 36, 12)                 # dead end
    return PlanScenario("K-turn in a narrow street", g, np.array([8.0, 4.0, 0.0]), np.array([8.0, 6.6, np.pi]), [],
                        (0.8, 0.2, 0.1))
```

sdc/planning/lattice.py

```python
"""Conformal state lattice in the Frenet frame of the road centre line.

Candidates are quintic lateral polynomials d(s) with a *spatial* parametrisation,

    d(s0)=d0, d'(s0)=d0', d''(s0)=d0''   ->   d(s0+S_f)=d_f, d'=0, d''=0,

for a lattice of terminal offsets d_f and horizons S_f.  They are C^2 by construction, so the
stitched receding-horizon path has continuous curvature.  Each candidate is scored by

    J = w_j * int (d''')^2 ds  +  w_o * ((d_f - d_ref)^2 + mean (d - d_ref)^2)  +  w_k * max(|kappa|-0.6 kmax,0)^2

and rejected if it leaves the road, exceeds the curvature limit or comes closer to an obstacle than the margin.
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np


@dataclass
class LatticeConfig:
    d_offsets: tuple = tuple(np.linspace(-3.0, 3.0, 13))
    horizons: tuple = (18.0, 26.0, 36.0)
    d_ref: float = -1.75                 # preferred lane centre
    d_min: float = -3.1
    d_max: float = 3.1                   # drivable corridor (no oncoming traffic in this test)
    ds: float = 0.5
    kappa_max: float = 0.2
    w_jerk: float = 40.0
    w_off: float = 1.0
    w_kappa: float = 200.0
    ego_discs: tuple = (-0.2, 1.5, 3.2)  # along the heading from the rear axle
    ego_radius: float = 1.15
    margin: float = 0.35


def quintic(d0, dp0, ddp0, df, T):
    A = np.array([[T ** 3, T ** 4, T ** 5], [3 * T ** 2, 4 * T ** 3, 5 * T ** 4], [6 * T, 12 * T ** 2, 20 * T ** 3]])
    b = np.array([df - (d0 + dp0 * T + 0.5 * ddp0 * T ** 2), -(dp0 + ddp0 * T), -ddp0])
    a3, a4, a5 = np.linalg.solve(A, b)
    return np.array([d0, dp0, 0.5 * ddp0, a3, a4, a5])


def eval_poly(c, x):
    d = c[0] + c[1] * x + c[2] * x ** 2 + c[3] * x ** 3 + c[4] * x ** 4 + c[5] * x ** 5
    dp = c[1] + 2 * c[2] * x + 3 * c[3] * x ** 2 + 4 * c[4] * x ** 3 + 5 * c[5] * x ** 4
    ddp = 2 * c[2] + 6 * c[3] * x + 12 * c[4] * x ** 2 + 20 * c[5] * x ** 3
    dddp = 6 * c[3] + 24 * c[4] * x + 60 * c[5] * x ** 2
    return d, dp, ddp, dddp


class Frenet:
    """Frenet <-> world conversions for a periodic ``Route``."""

    def __init__(self, route):
        self.route = route

    def to_world(self, s, d, dp=None):
        xy, th, kr = self.route.at(s)
        n = np.stack([-np.sin(th), np.cos(th)], axis=-1)
        pos = xy + d[..., None] * n
        if dp is None:
            return pos, th, kr
        dth = np.arctan2(dp, 1.0 - kr * d)
        return pos, th + dth, kr

    @staticmethod
    def curvature(kr, d, dp, ddp):
        """Curvature of the path d(s) on a road of curvature kr (kr' neglected)."""
        a = 1.0 - kr * d
        dth_ds = (ddp * a + dp * (kr * dp)) / (a * a + dp * dp)
        return (kr + dth_ds) / np.sqrt(a * a + dp * dp)


class StateLattice:
    def __init__(self, route, obstacles: np.ndarray, cfg: LatticeConfig = LatticeConfig(), profile: str = "quintic",
                 ramp: float = 12.0):
        """``obstacles``: (N,3) discs [x, y, r] in the world frame.

        ``profile='quintic'``: C^2 quintic lateral profiles (the state lattice).
        ``profile='linear'`` : piecewise-linear lateral ramps over ``ramp`` m -- the "connect-the-dots" baseline.
        Both use the same terminal-offset set, collision test and road limits, so only smoothness differs.
        """
        self.route, self.cfg, self.fr = route, cfg, Frenet(route)
        self.profile, self.ramp = profile, ramp
        self.obs = np.asarray(obstacles, float).reshape(-1, 3)

    def clearance(self, pos, th):
        """Minimum (disc-to-disc) clearance along a sampled path: pos (M,2), heading th (M,)."""
        if len(self.obs) == 0:
            return np.inf
        offs = np.array(self.cfg.ego_discs)
        cx = pos[:, 0:1] + np.cos(th)[:, None] * offs[None, :]
```

```
        cy = pos[:, 1:2] + np.sin(th)[:, None] * offs[None, :]
        dx = cx[:, :, None] - self.obs[None, None, :, 0]
        dy = cy[:, :, None] - self.obs[None, None, :, 1]
        gap = np.hypot(dx, dy) - self.cfg.ego_radius - self.obs[None, None, :, 2]
        return float(gap.min())

    def _profile(self, d0, dp0, ddp0, df, Sf):
        if self.profile == "quintic":
            c = quintic(d0, dp0, ddp0, df, Sf)
            return lambda x: eval_poly(c, x)
        L, k = self.ramp, (df - d0)
        def fn(x):
            u = np.clip(x / L, 0.0, 1.0)
            d = d0 + k * u
            dp = np.where(x < L, k / L, 0.0)
            z = np.zeros_like(x)
            return d, dp, z, z
        return fn

    def plan(self, s0, d0, dp0, ddp0):
        cfg = self.cfg
        best, cands = None, []
        horizons = cfg.horizons if self.profile == "quintic" else (max(cfg.horizons[0], self.ramp + 6.0),)
        for Sf in horizons:
            x = np.arange(0.0, Sf + 1e-9, cfg.ds)
            s = s0 + x
            xy_c, th_c, kr = self.route.at(s)
            for df in cfg.d_offsets:
                fn = self._profile(d0, dp0, ddp0, df, Sf)
                d, dp, ddp, dddp = fn(x)
                ok = (d.min() >= cfg.d_min) and (d.max() <= cfg.d_max)
                kap = Frenet.curvature(kr, d, dp, ddp)
                kmax = np.abs(kap).max()
                pos, th, _ = self.fr.to_world(s, d, dp)
                clr = self.clearance(pos, th) if ok else -np.inf
                feasible = ok and clr >= cfg.margin and (self.profile != "quintic" or kmax <= cfg.kappa_max)
                jerk = np.trapezoid(dddp ** 2, x)
                off = (df - cfg.d_ref) ** 2 + np.mean((d - cfg.d_ref) ** 2)
                cost = cfg.w_jerk * jerk + cfg.w_off * off
                if self.profile == "quintic":
                    cost += cfg.w_kappa * max(kmax - 0.6 * cfg.kappa_max, 0.0) ** 2
                cost += 4.0 * max(1.2 - clr, 0.0) if feasible else 0.0        # prefer wider clearance
                cands.append(dict(Sf=Sf, df=df, cost=cost, feasible=feasible, fn=fn, kmax=kmax, clr=clr))
                if feasible and (best is None or cost < best["cost"]):
                    best = cands[-1]
        return best, cands


def run_receding_horizon(route, planner, s_start, s_end, v=8.0, replan_every=4.0, d_init=-1.75):
    """Receding-horizon driving with perfect tracking; returns the executed path and its statistics.

    Every ``replan_every`` metres the planner is re-run from the current Frenet state (d, d', d'').  If no
    candidate is feasible the previous plan is kept (and counted as a failure).
    """
    fr = planner.fr
    s, d, dp, ddp = s_start, d_init, 0.0, 0.0
    S, D, DP, DDP, plan_fail, n_plans = [], [], [], [], 0, 0
    cur_fn, cur_s0, cur_Sf = None, None, None
    while s < s_end - 1e-9:
        best, _ = planner.plan(s, d, dp, ddp)
        n_plans += 1
        if best is None:
            plan_fail += 1
            if cur_fn is None:
                break
            fn, s_ref, Sf = cur_fn, cur_s0, cur_Sf
        else:
            fn, s_ref, Sf = best["fn"], s, best["Sf"]
        x_end = min(replan_every, s_end - s)
        xs = np.arange(0.0, x_end + 1e-9, planner.cfg.ds / 2)
        xx = np.clip(xs + (s - s_ref), 0.0, Sf)
        dd, ddp_, ddpp, _ = fn(xx)
        S.append(s + xs); D.append(dd); DP.append(ddp_); DDP.append(ddpp)
        cur_fn, cur_s0, cur_Sf = fn, s_ref, Sf
        s, d, dp, ddp = s + xs[-1], dd[-1], ddp_[-1], ddpp[-1]
    S, D, DP, DDP = map(np.concatenate, (S, D, DP, DDP))
    pos, th, kr = fr.to_world(S, D, DP)
    seg = np.hypot(*np.diff(pos, axis=0).T)
    keep = np.r_[True, seg > 1e-6]
    pos_k, th_k = pos[keep], np.unwrap(th[keep])
    sl = np.r_[0.0, np.cumsum(np.hypot(*np.diff(pos_k, axis=0).T))]
    kappa = np.gradient(th_k, sl)                      # same estimator for every planner
    return dict(s=S[keep], d=D[keep], pos=pos_k, theta=th_k, kappa=kappa, arclen=sl, fails=plan_fail, plans=n_plans)
```

experiments/common.py

```
"""Shared helpers for the experiment scripts (paths, plotting style, result I/O)."""
from __future__ import annotations

import json
import os
import sys
```

```
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
RES = ROOT / "results"
FIG = RES / "figures"
RES.mkdir(exist_ok=True)
FIG.mkdir(exist_ok=True)

# Validated categorical slots (light surface); text never wears a series colour.
BLUE, ORANGE, AQUA, YELLOW, MAGENTA = "#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"
INK, INK2, MUTED, GRID, SURF = "#0b0b0b", "#52514e", "#8a8984", "#e4e3df", "#fcfcfb"
SHADE = "#e9e7e1"          # tunnel / region shading
GT = INK


def style():
    plt.rcParams.update({
        "figure.facecolor": SURF, "axes.facecolor": SURF, "savefig.facecolor": SURF,
        "axes.edgecolor": MUTED, "axes.labelcolor": INK2, "xtick.color": INK2, "ytick.color": INK2,
        "text.color": INK, "axes.spines.top": False, "axes.spines.right": False,
        "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.6, "axes.axisbelow": True,
        "font.size": 8.5, "axes.titlesize": 9.5, "axes.titleweight": "bold", "axes.titlelocation": "left",
        "legend.frameon": False, "legend.fontsize": 8, "lines.linewidth": 1.6, "figure.dpi": 130,
        "savefig.dpi": 220, "savefig.bbox": "tight", "axes.prop_cycle": plt.cycler(color=[BLUE, ORANGE, AQUA, YELLOW]),
    })


style()


def savefig(fig, name):
    fig.savefig(FIG / f"{name}.pdf")
    fig.savefig(FIG / f"{name}.png")
    plt.close(fig)
    return name


def save_json(name, obj):
    def conv(o):
        if isinstance(o, (np.floating,)):
            return float(o)
        if isinstance(o, (np.integer,)):
            return int(o)
        if isinstance(o, np.ndarray):
            return o.tolist()
        raise TypeError(type(o))
    with open(RES / f"{name}.json", "w") as f:
        json.dump(obj, f, indent=2, default=conv)


def load_json(name):
    with open(RES / f"{name}.json") as f:
        return json.load(f)


def shade(ax, intervals, y_text=None, labels=None, color=SHADE):
    for k, (a, b) in enumerate(intervals):
        ax.axvspan(a, b, color=color, lw=0, zorder=0)
        if labels:
            ax.text((a + b) / 2, y_text if y_text is not None else ax.get_ylim()[1], labels[k], ha="center",
                    va="top", fontsize=7.5, color=INK2)


def ms(x):
    x = np.asarray(x, float)
    return float(np.mean(x)), float(np.std(x, ddof=1)) if len(x) > 1 else 0.0


def pool_map(fn, args, procs=2):
    import multiprocessing as mp
    with mp.get_context("fork").Pool(procs) as p:
        return p.map(fn, args, chunksize=1)
```

experiments/exp1_vio.py

```
"""Experiment 1 -- Visual-inertial odometry through GPS-denied sections (tunnel, underpass)."""
from common import *  # noqa
from sdc.geometry import pose_errors
from sdc.localization.vio import run_estimator
from sdc.sim.drive import GpsMeas, ImuParams, VoParams, simulate_drive, simulate_gps, simulate_imu, simulate_vo
from sdc.sim.world import World

MODES = ["imu_only", "vo_only", "gps_imu", "vio"]
LABEL = {"imu_only": "IMU only", "vo_only": "Visual odometry only", "gps_imu": "GPS + IMU", "vio": "GPS + IMU + VO (VIO)",
         "vio_nogps": "VIO, GPS only for first 10 s"}
```

```

def setup(seed, vo_scale=1.0):
    w = World(seed=7)
    d = simulate_drive(w, laps=1.25, seed=0)
    rng = np.random.default_rng(1000 + seed)
    ip = ImuParams()
    imu, bias = simulate_imu(d, ip, rng)
    vp = VoParams(sigma_t0=0.010 * vo_scale, sigma_t1=0.010 * vo_scale, sigma_r0=0.0015 * vo_scale,
                  sigma_r1=0.002 * vo_scale)
    vo = simulate_vo(d, w, vp, rng)
    gps = simulate_gps(d, w, rng)
    return w, d, ip, imu, vo, gps


def metrics(w, d, est, sig=None):
    err = np.hypot(*(est[:, :2] - d.pose[:, :2]).T)
    lon, lat, yaw = pose_errors(est, d.pose)
    tun, und = w.in_interval(d.s, w.tunnel), w.in_interval(d.s, w.underpass)
    it = np.where(tun)[0]
    out = dict(rmse=float(np.sqrt(np.mean(err ** 2))), rmse_tunnel=float(np.sqrt(np.mean(err[tun] ** 2))),
               max_tunnel=float(err[tun].max()), exit_tunnel=float(err[it[-1]]),
               max_underpass=float(err[und].max()), final=float(err[-1]),
               yaw_rmse_deg=float(np.degrees(np.sqrt(np.mean(yaw ** 2)))),
               lon_tunnel=float(np.sqrt(np.mean(lon[tun] ** 2))), lat_tunnel=float(np.sqrt(np.mean(lat[tun] ** 2))))
    if sig is not None and sig.any():
        out["tunnel_sigma_mean"] = float(sig[tun].mean())
        out["within_3sigma_tunnel"] = float(np.mean(err[tun] < 3 * np.maximum(sig[tun], 1e-6)))
    return out, err


def one_run(seed):
    w, d, ip, imu, vo, gps = setup(seed)
    res = {}
    for m in MODES:
        est, sig = run_estimator(d, imu, ip, gps, vo, m, np.random.default_rng(2000 + seed))
        res[m] = metrics(w, d, est, sig)[0]
    g0 = GpsMeas(gps.idx[gps.idx < 1000], gps.z[gps.idx < 1000], gps.sigma)
    est, sig = run_estimator(d, imu, ip, g0, vo, "vio", np.random.default_rng(2000 + seed))
    r, err = metrics(w, d, est, sig)
    r["drift_pct"] = float(100 * err[-1] / d.s[-1])        # final error / distance driven
    res["vio_nogps"] = r
    return res


def sweep_run(args):
    seed, k = args
    w, d, ip, imu, vo, gps = setup(seed, k)
    est, sig = run_estimator(d, imu, ip, gps, vo, "vio", np.random.default_rng(2000 + seed))
    r = metrics(w, d, est, sig)[0]
    return k, r["exit_tunnel"], r["max_tunnel"]


def main():
    N = 20
    runs = pool_map(one_run, list(range(N)))
    keys = list(runs[0]["vio"].keys())
    table = {}
    for m in MODES + ["vio_nogps"]:
        ks = [k for k in runs[0][m].keys()]
        table[m] = {k: dict(zip(("mean", "std"), ms([r[m][k] for r in runs]))) for k in ks}
    # VO noise sweep
    scales = [0.5, 1.0, 2.0, 4.0]
    sw = pool_map(sweep_run, [(s, k) for k in scales for s in range(6)])
    sweep = {str(k): dict(exit=ms([x[1] for x in sw if x[0] == k]), maxt=ms([x[2] for x in sw if x[0] == k])) for k in scales}
    save_json("exp1_vio", dict(n_runs=N, table=table, vo_noise_sweep=sweep,
                               tunnel_len=float(w_len()), raw_exit={m: [r[m]["exit_tunnel"] for r in runs] for m in MODES}))

    # ------------------ figures (representative run: seed 0) ------------------
    w, d, ip, imu, vo, gps = setup(0)
    ests, errs = {}, {}
    for m in MODES:
        ests[m], sg = run_estimator(d, imu, ip, gps, vo, m, np.random.default_rng(2000))
        errs[m] = np.hypot(*(ests[m][:, :2] - d.pose[:, :2]).T)
        if m == "vio":
            sig_vio = sg
    col = {"imu_only": MUTED, "vo_only": YELLOW, "gps_imu": ORANGE, "vio": BLUE}
    ls = {"imu_only": ":", "vo_only": "-.", "gps_imu": "--", "vio": "-"}
    fig, ax = plt.subplots(1, 2, figsize=(10.2, 3.5), gridspec_kw=dict(width_ratios=[1.0, 1.45]))
    a = ax[0]
    a.plot(*w.route.xy.T, color=GRID, lw=7, solid_capstyle="round", zorder=0)
    tmask = w.in_interval(d.s, w.tunnel)
    a.plot(d.pose[:, 0], d.pose[:, 1], color=GT, lw=1.0, label="ground truth")
    a.plot(d.pose[tmask, 0], d.pose[tmask, 1], color=INK, lw=4.0, alpha=0.18, label="tunnel")
    for m in ("gps_imu", "vio"):
        a.plot(ests[m][:, 0], ests[m][:, 1], color=col[m], ls=ls[m], lw=1.4, label=LABEL[m])
    a.set_aspect("equal"); a.set_xlabel("x [m]"); a.set_ylabel("y [m]"); a.set_title("Route and estimates")
    a.legend(loc="upper center", ncol=2, bbox_to_anchor=(0.5, -0.20), fontsize=7.5)
    b = ax[1]
    t = d.t
    for m in MODES:
        b.plot(t, np.maximum(errs[m], 1e-2), color=col[m], ls=ls[m], label=LABEL[m])
    b.set_yscale("log"); b.set_ylim(1e-2, 2e2)
    tun_t = t[tmask][[0, -1]]; und_t = t[w.in_interval(d.s, w.underpass)][[0, -1]]
    shade(b, [tun_t, und_t])
    b.text(tun_t.mean(), 1.4e2, "tunnel", ha="center", fontsize=7.5, color=INK2)
```

```
    b.text(und_t.mean(), 1.4e2, "underpass", ha="center", fontsize=7.5, color=INK2)
    b.set_xlabel("time [s]"); b.set_ylabel("position error [m]"); b.set_title("Position error (log scale)")
    b.legend(loc="upper center", ncol=2, bbox_to_anchor=(0.5, -0.20), fontsize=7.5)
    savefig(fig, "fig_vio_overview")

    # exit-error distribution across the Monte-Carlo runs
    fig, a = plt.subplots(figsize=(5.2, 3.0))
    data = [np.array([r[m]["exit_tunnel"] for r in runs]) for m in ("vo_only", "gps_imu", "vio")]
    bp = a.boxplot(data, orientation="horizontal", widths=0.5, patch_artist=True, medianprops=dict(color=INK, lw=1.4),
                   boxprops=dict(lw=0.8, edgecolor=INK2), whiskerprops=dict(color=INK2, lw=0.8), capprops=dict(color=INK2),
                   flierprops=dict(marker="o", ms=3, mfc="none", mec=INK2))
    for p, c in zip(bp["boxes"], (YELLOW, ORANGE, BLUE)):
        p.set_facecolor(c); p.set_alpha(0.55)
    a.set_yticklabels(["VO only", "GPS + IMU", "VIO"]); a.set_xlabel("position error at tunnel exit [m]")
    a.set_title(f"Tunnel-exit error, {N} Monte-Carlo runs")
    savefig(fig, "fig_vio_exit_box")
    print("exp1 done")


def w_len():
    return 130.0


if __name__ == "__main__":
    main()
```

experiments/exp2_slam.py

```
"""Experiment 2 -- LiDAR SLAM: ICP odometry vs NDT scan-to-submap vs NDT + loop closure (pose graph)."""
from common import *  # noqa
from scipy.spatial import cKDTree
from sdc.geometry import compose
from sdc.localization.lidar_slam import LidarSlam, SlamConfig, icp_odometry
from sdc.sim.drive import lidar_scan, simulate_drive, simulate_lidar
from sdc.sim.world import World

SEEDS = [0, 1, 2]


def ate(est, gt):
    e = np.hypot(*(est[:, :2] - gt[:, :2]).T)
    return dict(rmse=float(np.sqrt(np.mean(e ** 2))), max=float(e.max()), final=float(e[-1])), e


def map_error(points, ref_tree):
    d, _ = ref_tree.query(points)
    return dict(mean=float(d.mean()), p95=float(np.percentile(d, 95)))


def one(seed):
    w = World(seed=7)
    d = simulate_drive(w, laps=1.25, seed=seed)
    rng = np.random.default_rng(seed)
    idx, scans = simulate_lidar(d, w, rng)
    gt = d.pose[idx]
    ref = np.concatenate([lidar_scan(w, d.pose[i], rng, noise=False) @ np.array([[np.cos(d.pose[i, 2]), np.sin(d.pose[i, 2])],
                          [-np.sin(d.pose[i, 2]), np.cos(d.pose[i, 2])]]) + d.pose[i, :2] for i in idx[::2]])
    tree = cKDTree(ref)
    out, keep = {}, {}
    ic = icp_odometry(scans, gt[0])
    out["icp"], e = ate(ic, gt)
    out["icp"]["yaw_final_deg"] = float(np.degrees(abs((ic[-1, 2] - gt[-1, 2] + np.pi) % (2 * np.pi) - np.pi)))
    keep["icp"] = ic
    for name, loop in (("ndt_odom", False), ("ndt_loop", True)):
        slam = LidarSlam(gt[0], SlamConfig(loop=loop))
        t0 = time.time()
        for k, s in enumerate(scans):
            slam.process(s, k)
        if loop:
            slam.optimize()
        dt = (time.time() - t0) / len(scans) * 1e3
        kp, ki = slam.keyframe_poses()
        out[name], e = ate(kp, gt[ki])
        out[name]["ms_per_scan"] = float(dt)
        out[name]["loops"] = int(slam.n_loops)
        out[name]["keyframes"] = int(len(kp))
        out[name]["map"] = map_error(slam.global_map(0.2), tree)
        keep[name] = (kp, ki)
        if name == "ndt_odom":
            deg = np.array([r["deg"] for r in slam.scan_log])
            sidx = np.array([r["idx"] for r in slam.scan_log])
            keep["deg"] = (np.mod(d.s[idx][sidx], w.length), deg)
        else:
            keep["map"] = slam.global_map(0.25)
            keep["loops"] = [(slam.kfs[a].pose[:2].copy(), slam.kfs[b].pose[:2].copy()) for a, b in slam.loop_pairs]
    keep["gt"], keep["idx"], keep["s"] = gt, idx, d.s[idx]
    return out, (keep if seed == 0 else None)


def main():
    res = pool_map(one, SEEDS)
    table = {}
```

```
    for m in ("icp", "ndt_odom", "ndt_loop"):
        table[m] = {k: (ms([r[0][m][k] for r in res]) if k != "map" else
                        {kk: ms([r[0][m]["map"][kk] for r in res]) for kk in ("mean", "p95")})
                    for k in res[0][0][m]}
    k0 = res[0][1]
    L = World(seed=7).length
    sdeg, deg = k0["deg"]
    tun = (sdeg >= 200) & (sdeg <= 330)
    table["degeneracy"] = dict(tunnel_median=float(np.median(deg[tun])), outside_median=float(np.median(deg[~tun])))
    save_json("exp2_slam", dict(seeds=SEEDS, table=table))

    gt = k0["gt"]
    kp, ki = k0["ndt_loop"]
    ko, kio = k0["ndt_odom"]
    fig, ax = plt.subplots(1, 3, figsize=(11, 3.4), gridspec_kw=dict(width_ratios=[1.1, 1.1, 1.2]))
    a = ax[0]
    a.plot(gt[:, 0], gt[:, 1], color=GT, lw=1.0, label="ground truth")
    a.plot(k0["icp"][:, 0], k0["icp"][:, 1], color=YELLOW, ls="-.", label="ICP odometry")
    a.plot(ko[:, 0], ko[:, 1], color=ORANGE, ls="--", label="NDT odometry")
    a.plot(kp[:, 0], kp[:, 1], color=BLUE, label="NDT + loop closure")
    a.set_aspect("equal"); a.set_title("Estimated trajectories"); a.set_xlabel("x [m]"); a.set_ylabel("y [m]")
    a.legend(loc="upper center", bbox_to_anchor=(0.5, -0.2), ncol=2, fontsize=7)
    b = ax[1]
    mp = k0["map"]
    b.scatter(mp[:, 0], mp[:, 1], s=0.15, color=INK2, alpha=0.6, rasterized=True)
    for p, q in k0["loops"][:60]:
        b.plot([p[0], q[0]], [p[1], q[1]], color=BLUE, lw=0.4, alpha=0.7)
    b.set_aspect("equal"); b.set_title("SLAM map and loop-closure edges"); b.set_xlabel("x [m]")
    c = ax[2]
    c.plot(k0["s"], np.hypot(*(k0["icp"][:, :2] - gt[:, :2]).T), color=YELLOW, ls="-.", label="ICP odometry")
    c.plot(k0["s"][kio], np.hypot(*(ko[:, :2] - gt[kio, :2]).T), color=ORANGE, ls="--", label="NDT odometry")
    c.plot(k0["s"][ki], np.hypot(*(kp[:, :2] - gt[ki, :2]).T), color=BLUE, label="NDT + loop closure")
    c.set_yscale("log"); c.set_xlabel("distance driven [m]"); c.set_ylabel("position error [m]")
    c.set_title("Absolute trajectory error"); c.legend(loc="upper center", bbox_to_anchor=(0.5, -0.2), ncol=2, fontsize=7)
    savefig(fig, "fig_slam_overview")

    fig, a = plt.subplots(figsize=(5.4, 2.6))
    a.plot(sdeg, deg, color=BLUE, lw=0.8)
    shade(a, [(200, 330)]); a.set_ylim(0, max(0.7, deg.max()))
    a.text(265, a.get_ylim()[1] * 0.93, "tunnel", ha="center", fontsize=7.5, color=INK2)
    a.set_xlabel("position along route [m]"); a.set_ylabel(r"$\lambda_{min}/\lambda_{max}$")
    a.set_title("NDT Hessian conditioning (low = weakly constrained)")
    savefig(fig, "fig_slam_degeneracy")
    print("exp2 done")


if __name__ == "__main__":
    import time
    main()
```

experiments/exp3_hdmap.py

```
"""Experiment 3 -- Localisation against a pre-built HD map: NDT alignment vs particle filter."""
from common import *  # noqa
import time
from sdc.geometry import compose, pose_errors
from sdc.localization.hd_map import survey_map
from sdc.localization.particle_filter import NdtLocalizer, ParticleFilter
from sdc.sim.drive import VoParams, simulate_drive, simulate_lidar, simulate_vo
from sdc.sim.world import World

_W = World(seed=7)
_MAP = None


def hdmap():
    global _MAP
    if _MAP is None:
        sd = simulate_drive(_W, laps=1.0, seed=100, lane_offset=-1.5, wander=1.0)
        _MAP = survey_map(_W, sd, np.random.default_rng(100))
    return _MAP


def data(seed, n_dyn, laps=1.25):
    d = simulate_drive(_W, laps=laps, seed=1 + seed)
    rng = np.random.default_rng(10 + seed)
    dyn = _W.dynamic_objects(n_dyn, np.random.default_rng(20 + seed)) if n_dyn else None
    idx, scans = simulate_lidar(d, _W, rng, extra_segments=dyn)
    vo = simulate_vo(d, _W, VoParams(), rng)
    return d, idx, scans, d.pose[idx], vo


def stats(est, gt):
    lon, lat, yaw = pose_errors(est, gt)
    e = np.hypot(lon, lat)
    return dict(lon_rms=float(np.sqrt(np.mean(lon ** 2))), lat_rms=float(np.sqrt(np.mean(lat ** 2))),
                yaw_rms_deg=float(np.degrees(np.sqrt(np.mean(yaw ** 2)))), p95=float(np.percentile(e, 95)),
                max=float(e.max())), e


def run_ndt(scans, vo, gt):
    nl, est, t0 = NdtLocalizer(hdmap(), gt[0]), [gt[0].copy()], time.time()
```

```
    for k in range(1, len(scans)):
        est.append(nl.step(scans[k], vo.z[k - 1])[0].copy())
    return np.array(est), (time.time() - t0) / len(scans) * 1e3


def run_pf(scans, vo, gt, n=500, seed=4, init=None):
    pf = ParticleFilter(hdmap(), n, np.random.default_rng(seed))
    if init is None:
        pf.init_gaussian(gt[0], [1.0, 1.0, 0.05])
    else:
        pf.init_uniform(init)
    est, spread, t0 = [], [], time.time()
    for k in range(len(scans)):
        if k:
            pf.predict(vo.z[k - 1])
        pf.update(scans[k])
        p, s = pf.estimate()
        est.append(p); spread.append(s)
    return np.array(est), np.array(spread), (time.time() - t0) / len(scans) * 1e3


def dead_reckon(vo, gt):
    p, out = gt[0].copy(), [gt[0].copy()]
    for k in range(len(gt) - 1):
        p = compose(p, vo.z[k]); out.append(p.copy())
    return np.array(out)


def job(args):
    seed, n_dyn = args
    d, idx, scans, gt, vo = data(seed, n_dyn)
    r = {}
    est, ms_ndt = run_ndt(scans, vo, gt); r["ndt"], e_ndt = stats(est, gt); r["ndt"]["ms"] = ms_ndt
    est2, _, ms_pf = run_pf(scans, vo, gt); r["pf"], e_pf = stats(est2, gt); r["pf"]["ms"] = ms_pf
    r["vo"], _ = stats(dead_reckon(vo, gt), gt)
    return (seed, n_dyn, r, (d.t[idx], e_ndt, e_pf) if (seed == 0 and n_dyn == 15) else None)


def global_job(seed):
    d, idx, scans, gt, vo = data(seed, 0, laps=0.6)
    sc = scans[:450]; gt = gt[:450]
    w = _W

    def sampler(n, rng):
        s = rng.uniform(0, w.length, n)
        xy, th, _ = w.route.at(s)
        nrm = np.stack([-np.sin(th), np.cos(th)], 1)
        xy = xy + rng.uniform(-3.5, 3.5, n)[:, None] * nrm
        return np.column_stack([xy, rng.uniform(-np.pi, np.pi, n)])
    est, spread, _ = run_pf(sc, vo, gt, n=20000, seed=seed, init=sampler)
    e = np.hypot(*(est[:, :2] - gt[:, :2]).T)
    ok = np.where((e < 0.5) & (spread < 1.0))[0]
    conv = None
    for k in ok:                          # first time it converges and stays converged
        if np.all(e[k:] < 1.0):
            conv = float(k * 0.1); break
    return dict(seed=seed, converge_s=conv, final=float(e[-1])), (d.t[idx][:450], e)


def main():
    hdmap()
    jobs = [(s, n) for s in range(3) for n in (0, 15, 40)]
    out = pool_map(job, jobs)
    table = {}
    for n in (0, 15, 40):
        sub = [o[2] for o in out if o[1] == n]
        table[str(n)] = {m: {k: ms([x[m][k] for x in sub]) for k in sub[0][m]} for m in ("ndt", "pf", "vo")}
    # particle-count sweep (seed 0, 15 unmapped vehicles)
    d, idx, scans, gt, vo = data(0, 15)
    sweep = {}
    for n in (50, 200, 1000, 3000):
        est, _, t = run_pf(scans, vo, gt, n=n); s, _ = stats(est, gt); s["ms"] = t; sweep[str(n)] = s
    gl = pool_map(global_job, [0, 1, 2])
    save_json("exp3_hdmap", dict(table=table, pf_sweep=sweep, global_loc=[g[0] for g in gl],
                                 map_points=int(len(hdmap().points))))
    # figures
    t, e_ndt, e_pf = [o[3] for o in out if o[3] is not None][0]
    fig, ax = plt.subplots(1, 3, figsize=(11, 3.0))
    a = ax[0]
    a.plot(t, e_ndt * 100, color=BLUE, lw=1.0, label="NDT"); a.plot(t, e_pf * 100, color=ORANGE, lw=1.0, ls="--", label="particle
      filter")
    a.set_xlabel("time [s]"); a.set_ylabel("position error [cm]"); a.set_title("Error along the drive"); a.legend(fontsize=7.5)
    b = ax[1]
    lv = [0, 15, 40]
    for j, (m, c, lab) in enumerate((("ndt", BLUE, "NDT"), ("pf", ORANGE, "PF (500)"))):
        b.bar(np.arange(3) + (j - 0.5) * 0.36, [100 * np.hypot(table[str(n)][m]["lon_rms"][0], table[str(n)][m]["lat_rms"][0]) for n in
      lv],
               0.34, color=c, label=lab)
    b.set_xticks(range(3)); b.set_xticklabels([str(n) for n in lv]); b.set_xlabel("unmapped vehicles in scene")
    b.set_ylabel("position RMS [cm]"); b.set_title("Robustness to map changes"); b.legend(fontsize=7.5)
    c = ax[2]
    for g, col in zip(gl, (BLUE, ORANGE, AQUA)):
        tt, ee = g[1]; c.plot(tt, np.maximum(ee, 1e-2), color=col, lw=1.0, label=f"seed {g[0]['seed']}")
    c.set_yscale("log"); c.set_xlabel("time [s]"); c.set_ylabel("position error [m]")
    c.set_title("Global localisation (20k particles)"); c.legend(fontsize=7.5)
    savefig(fig, "fig_hdmap_overview")
```

```
    print("exp3 done")


if __name__ == "__main__":
    main()
```

experiments/exp4_behavior.py

```
"""Experiment 4 -- Behaviour tree vs finite-state machines on randomised traffic scenarios."""
from common import *  # noqa
import collections
from sdc.behavior.bt import draw_tree
from sdc.behavior.scenario import *  # noqa

N = 300
CTRL = [("BT", BTController), ("FSM-basic", lambda: FSMController("basic")), ("FSM-full", lambda: FSMController("full")),
        ("FSM-full, no RB re-check", lambda: FSMController("full", rb_recheck=False))]


def summarise(res):
    col = collections.Counter(r["collision"] for r in res if r["collision"])
    ok = [r for r in res if r["reached"] and not r["collision"]]
    emg = sum(sum(1 for k in range(1, len(r["tags"])) if r["tags"][k] == "EMERGENCY" and r["tags"][k - 1] != "EMERGENCY") for r in res)
    return dict(success=len(ok), n=len(res), collisions=dict(col), timeouts=sum(1 for r in res if not r["reached"] and not r["collision
      "]),
                time=float(np.mean([r["time"] for r in ok])) if ok else None, overtakes=sum(r["overtook"] for r in res),
                aborts=sum(any(t.startswith("ABORT") for t in r["tags"]) for r in res),
                hard_brakes=float(np.mean([r["hard_brakes"] for r in res])), emergency=emg,
                switches=float(np.mean([r["switches"] for r in res])))


def main():
    rng = np.random.default_rng(123)
    scs = [sample_scenario(rng) for _ in range(N)]
    out, runs = {}, {}
    for name, mk in CTRL:
        runs[name] = [run_scenario(sc, mk()) for sc in scs]
        out[name] = summarise(runs[name])
    surprise = [sc.surprise for sc in scs]
    for name in runs:
        out[name]["success_surprise"] = sum(1 for r, s in zip(runs[name], surprise) if s and r["reached"] and not r["collision"])
        out[name]["n_surprise"] = sum(surprise)
    # structure
    root = build_bt()
    nodes = list(root.walk())
    kinds = collections.Counter(n.kind for n, _ in nodes)
    full, basic = FSMController("full"), FSMController("basic")
    n_states = len(full.T)
    out["structure"] = dict(bt_nodes=len(nodes), bt_conditions=kinds["condition"], bt_actions=kinds["action"],
                            bt_composites=kinds["sequence"] + kinds["fallback"], fsm_states=n_states,
                            fsm_transitions_basic=basic.n_transitions, fsm_transitions_full=full.n_transitions,
                            bt_add_behaviour=3, fsm_add_behaviour=n_states + 1)
    save_json("exp4_behavior", out)
    draw_tree(root, str(FIG / "fig_bt_tree.png"), "Behaviour tree (priority order: top to bottom)")
    draw_tree(root, str(FIG / "fig_bt_tree.pdf"))

    # timeline of a scenario where the BT aborts an overtake
    pick = next((i for i, r in enumerate(runs["BT"]) if any(t.startswith("ABORT") for t in r["tags"]) and r["overtook"] is False
                 and not r["collision"]), None)
    if pick is None:
        pick = next(i for i, r in enumerate(runs["BT"]) if any(t.startswith("ABORT") for t in r["tags"]))
    sc, r = scs[pick], runs["BT"][pick]
    L = r["log"]; t = L[:, 0]
    fig, ax = plt.subplots(3, 1, figsize=(7.2, 5.4), sharex=True, gridspec_kw=dict(height_ratios=[2, 1, 1]))
    ax[0].plot(t, L[:, 1], color=BLUE, label="ego")
    ax[0].plot(t, np.minimum(sc.s_lead0 + sc.v_lead * t, sc.s_turn + 5), color=ORANGE, ls="--", label="lead vehicle")
    for o in sc.oncoming:
        s0, v, ts = o
        m = (t >= ts) & (s0 - v * (t - ts) > 0)
        if m.any() and abs(s0 - v * (t[m][0] - ts) - L[m, 1][0]) < 500:
            ax[0].plot(t[m], s0 - v * (t[m] - ts), color=MUTED, lw=0.8, ls=":")
    ax[0].plot([], [], color=MUTED, ls=":", label="oncoming vehicles")
    ax[0].set_ylabel("position s [m]"); ax[0].set_ylim(0, 540); ax[0].legend(ncol=3, fontsize=7.5, loc="upper left", frameon=True,
      facecolor=SURF, edgecolor="none", framealpha=0.95)
    ax[0].axhspan(RB_ZONE[0], RB_ZONE[1], color=SHADE, lw=0); ax[0].text(t[0] + 1, RB_CONF + 8, "roundabout", ha="left", fontsize=7,
      color=INK2)
    ax[0].axhspan(CW_ZONE[0], CW_ZONE[1], color=SHADE, lw=0); ax[0].text(t[0] + 1, CW_S + 8, "crosswalk", ha="left", fontsize=7, color=
      INK2)
    ax[1].plot(t, L[:, 2], color=BLUE); ax[1].set_ylabel("lateral d [m]"); ax[1].axhline(LANE_W, color=GRID, lw=0.8)
    ax[2].plot(t, L[:, 3], color=BLUE); ax[2].set_ylabel("speed [m/s]"); ax[2].set_xlabel("time [s]")
    tags = r["tags"]
    start, row = 0, 0
    for k in range(1, len(tags) + 1):
        if k == len(tags) or tags[k] != tags[start]:
            tg, t1 = tags[start], t[min(k, len(t) - 1)]
            if tg not in ("CRUISE", "FOLLOW") and (t1 - t[start]) > 0.8:
                ax[2].annotate(tg.replace("_", " "), xy=((t[start] + t1) / 2, L[start, 3]), xytext=((t[start] + t1) / 2, 8.6 + 1.9 * (
      row % 3)),
                               ha="center", fontsize=6.2, color=INK2, arrowprops=dict(arrowstyle="-", color=MUTED, lw=0.5))
                row += 1
            start = k
    ax[2].set_ylim(0, 15.5)
```

```
    ax[0].set_title("Behaviour-tree run with an aborted overtake (labels = active leaf)")
    savefig(fig, "fig_bt_timeline")

    fig, a = plt.subplots(figsize=(5.4, 2.8))
    names = [n for n, _ in CTRL]
    succ = [100 * out[n]["success"] / N for n in names]
    a.barh(range(4), succ, color=[BLUE, YELLOW, ORANGE, MUTED], height=0.55)
    for i, v in enumerate(succ):
        a.text(v + 1, i, f"{v:.0f}%", va="center", fontsize=8)
    a.set_yticks(range(4)); a.set_yticklabels(names, fontsize=8); a.invert_yaxis(); a.set_xlim(0, 112)
    a.set_xlabel(f"scenarios completed without collision [%] (N={N})"); a.set_title("Controller outcomes")
    savefig(fig, "fig_bt_outcomes")
    print("exp4 done")


if __name__ == "__main__":
    main()
```

experiments/exp5_planning.py

```
"""Experiment 5 -- Motion planning: Hybrid A* vs grid A* (parking, K-turn); Frenet state lattice vs linear ramps."""
from common import *  # noqa
import time
from matplotlib.patches import Polygon
from sdc.planning.hybrid_astar import *  # noqa
from sdc.planning.lattice import Frenet, StateLattice, run_receding_horizon
from sdc.planning.scenarios import k_turn, parallel_parking, perpendicular_parking
from sdc.sim.world import World

VEH = Vehicle()
MAKERS = [perpendicular_parking, parallel_parking, k_turn]
NTRIALS = 6


def wrap(a):
    return (a + np.pi) % (2 * np.pi) - np.pi


def kappa_pose_path(p):
    """|curvature| along a pose path, ignoring cusps (direction changes)."""
    ds = np.hypot(*np.diff(p[:, :2], axis=0).T)
    dth = np.abs(wrap(np.diff(p[:, 2])))
    ok = (ds > 1e-3) & (p[1:, 3] == p[:-1, 3] if p.shape[1] > 3 else True)
    return dth[ok] / ds[ok]


def jitter(sc, rng, chk):
    for _ in range(50):
        s = sc.start + rng.uniform(-1, 1, 3) * np.array(sc.start_jitter)
        if chk.free(*s) and chk.exact_free(*[np.array([v]) for v in s])[0]:
            return s
    return sc.start


def evaluate_grid(sc, start, chk, smooth):
    xy, nexp = grid_astar(sc.grid, start[:2], sc.goal[:2], clearance=VEH.width / 2 + 0.05)
    if xy is None:
        return dict(found=False, feasible=False)
    if smooth:
        xy = chaikin(xy, 3)
    d = np.diff(xy, axis=0)
    th = np.arctan2(d[:, 1], d[:, 0]); th = np.r_[th, th[-1]]
    m = path_metrics(xy, kappa_max=VEH.kappa_max)
    coll = float(np.mean(~chk.exact_free(xy[:, 0], xy[:, 1], th)))
    gerr = float(np.degrees(abs(wrap(th[-1] - sc.goal[2])))); serr = float(np.degrees(abs(wrap(th[0] - start[2]))))
    feas = coll == 0 and m["max_kappa"] <= 1.05 * VEH.kappa_max and gerr < 10 and serr < 10
    return dict(found=True, feasible=bool(feas), length=m["length"], max_kappa=m["max_kappa"], corners=m["corners"],
                collision_frac=coll, goal_heading_err_deg=gerr)


def trial(args):
    mi, ti = args
    sc = MAKERS[mi](VEH)
    chk = FootprintChecker(sc.grid, VEH)
    rng = np.random.default_rng(100 * mi + ti)
    start = sc.start if ti == 0 else jitter(sc, rng, chk)
    hp = HybridAStar(sc.grid, VEH, max_exp=60000)
    r = hp.plan(start, sc.goal, time_limit=40)
    out = dict(mi=mi, ti=ti, grid=evaluate_grid(sc, start, chk, False), grid_smooth=evaluate_grid(sc, start, chk, True))
    if r.success:
        p = r.path
        k = kappa_pose_path(p)
        dirs = p[1:, 3]
        coll = bool((~chk.exact_free(p[:, 0], p[:, 1], p[:, 2])).any())
        end = float(np.hypot(*(p[-1, :2] - sc.goal[:2]))); eth = float(np.degrees(abs(wrap(p[-1, 2] - sc.goal[2]))))
        out["hybrid"] = dict(found=True, feasible=bool((not coll) and k.max() <= 1.05 * VEH.kappa_max and end < 0.35 and eth < 6),
                             time=r.time_s, expansions=r.expansions, length=float(np.sum(np.hypot(*np.diff(p[:, :2], axis=0).T))),
                             cusps=int(np.sum(dirs[1:] * dirs[:-1] < 0)), max_kappa=float(k.max()), end_err=end, end_heading_err_deg=
      eth,
                             collision=coll, shot=bool(r.used_shot))
        if ti == 0:
            out["path"] = p
```

```
    else:
        out["hybrid"] = dict(found=False, feasible=False, time=r.time_s, expansions=r.expansions)
    return out


def lattice_mc(n=20):
    w = World(seed=7); fr = Frenet(w.route)
    res = {"quintic": [], "linear": []}
    for seed in range(n):
        rng = np.random.default_rng(seed)
        obs = []
        for s in np.sort(rng.uniform(60, 330, rng.integers(5, 9))):
            d = rng.choice([-1.75, -1.75, -1.0, -2.6, 1.75])
            if rng.random() < 0.6:
                for o in (-1.5, 0.0, 1.5):
                    obs.append([*fr.to_world(np.array([s + o]), np.array([d]))[0][0], 1.0])
            else:
                obs.append([*fr.to_world(np.array([s]), np.array([d]))[0][0], rng.uniform(0.3, 0.5)])
        obs = np.array(obs)
        for prof in res:
            pl = StateLattice(w.route, obs, profile=prof)
            r = run_receding_horizon(w.route, pl, 30.0, 360.0)
            cl = pl.clearance(r["pos"], r["theta"])
            res[prof].append(dict(clearance=float(cl), collision_free=bool(cl > 0), fails=int(r["fails"]),
                                  max_kappa=float(np.abs(r["kappa"]).max()),
                                  max_dkappa=float(np.max(np.abs(np.diff(r["kappa"]) / np.maximum(np.diff(r["arclen"]), 1e-3)))),
                                  ok=bool(cl > 0 and np.abs(r["kappa"]).max() <= 1.05 * VEH.kappa_max)))
    return res


def main():
    out = pool_map(trial, [(m, t) for m in range(3) for t in range(NTRIALS)])
    table = {}
    for m, mk in enumerate(MAKERS):
        sub = [o for o in out if o["mi"] == m]
        row = {}
        for meth in ("hybrid", "grid", "grid_smooth"):
            rs = [o[meth] for o in sub]
            row[meth] = dict(found=sum(r["found"] for r in rs), feasible=sum(r["feasible"] for r in rs), n=len(rs))
            for k in ("time", "expansions", "length", "cusps", "max_kappa", "corners", "collision_frac", "goal_heading_err_deg", "
      end_err"):
                v = [r[k] for r in rs if r.get("found") and k in r]
                if v:
                    row[meth][k] = float(np.mean(v))
        table[mk(VEH).name] = row
    mc = lattice_mc()
    lat = {p: dict(ok=sum(r["ok"] for r in rs), n=len(rs), collision_free=sum(r["collision_free"] for r in rs),
                   max_kappa=ms([r["max_kappa"] for r in rs]), max_dkappa=ms([r["max_dkappa"] for r in rs]),
                   plan_fails=sum(r["fails"] for r in rs)) for p, rs in mc.items()}
    save_json("exp5_planning", dict(parking=table, lattice=lat, kappa_limit=VEH.kappa_max, rho=VEH.rho))

    # figure: Hybrid A* solutions (nominal start) with grid A* overlay
    fig, ax = plt.subplots(1, 3, figsize=(11, 3.3))
    for m, mk in enumerate(MAKERS):
        sc = mk(VEH); a = ax[m]
        a.imshow(sc.grid.occ.T, origin="lower", extent=[sc.grid.x0, sc.grid.x0 + sc.grid.nx * sc.grid.res, sc.grid.y0, sc.grid.y0 + sc.
      grid.ny * sc.grid.res],
                 cmap="Greys", alpha=0.35, interpolation="nearest")
        xy, _ = grid_astar(sc.grid, sc.start[:2], sc.goal[:2], VEH.width / 2 + 0.05)
        if xy is not None:
            a.plot(xy[:, 0], xy[:, 1], color=ORANGE, ls="--", lw=1.0, label="grid A*")
        o = [x for x in out if x["mi"] == m and x["ti"] == 0][0]
        if "path" in o:
            p = o["path"]; a.plot(p[:, 0], p[:, 1], color=BLUE, lw=1.4, label="Hybrid A*")
            for k in np.linspace(0, len(p) - 1, 9).astype(int):
                a.add_patch(Polygon(VEH.corners(p[k, :3]), fill=False, ec=BLUE, lw=0.5, alpha=0.7))
            a.add_patch(Polygon(VEH.corners(sc.goal), fill=False, ec=INK, lw=1.0, ls="--"))
        a.set_aspect("equal"); a.set_title(sc.name); a.grid(False)
        xs0, ys0 = sc.start[0], sc.start[1]
        a.set_xlim(max(0, (sc.goal[0] + xs0) / 2 - 17), (sc.goal[0] + xs0) / 2 + 17); a.set_ylim(sc.grid.y0, sc.grid.y0 + sc.grid.ny *
      sc.grid.res)
        if m == 0:
            a.legend(loc="lower left", fontsize=7)
    savefig(fig, "fig_parking")

    # figure: lattice vs linear
    w = World(seed=7); fr = Frenet(w.route)
    def cd(s, d): return [[*fr.to_world(np.array([s + o]), np.array([d]))[0][0], 1.0] for o in (-1.5, 0.0, 1.5)]
    def dc(s, d, r): return [[*fr.to_world(np.array([s]), np.array([d]))[0][0], r]]
    obs = np.array(cd(80, -1.75) + dc(120, -1.0, 0.3) + cd(150, -1.75) + dc(190, -2.6, 0.4) + cd(235, -1.75) + cd(300, -1.75) + cd(318,
       1.75))
    fig, ax = plt.subplots(1, 2, figsize=(10.5, 3.0), gridspec_kw=dict(width_ratios=[1.3, 1]))
    for prof, col, ls, lab in (("quintic", BLUE, "-", "quintic lattice"), ("linear", ORANGE, "--", "linear ramps")):
        pl = StateLattice(w.route, obs, profile=prof); r = run_receding_horizon(w.route, pl, 30.0, 360.0)
        ax[0].plot(r["s"], r["d"], color=col, ls=ls, label=lab)
        ax[1].plot(r["arclen"], r["kappa"], color=col, ls=ls, lw=1.0, label=lab)
    sob = []
    for o in obs:
        i = np.argmin(np.hypot(*(w.route.xy - o[:2]).T)); sob.append((w.route.s[i], o[2]))
    ax[0].set_xlabel("route position s [m]"); ax[0].set_ylabel("lateral offset d [m]"); ax[0].set_title("Executed lateral profile")
    ax[0].axhline(-1.75, color=GRID, lw=0.8); ax[0].legend(fontsize=7.5)
    ax[1].axhline(VEH.kappa_max, color=INK2, lw=0.8, ls=":"); ax[1].axhline(-VEH.kappa_max, color=INK2, lw=0.8, ls=":")
    ax[1].text(5, VEH.kappa_max + 0.02, r"$\kappa_{max}$", fontsize=7.5, color=INK2)
    ax[1].set_xlabel("path length [m]"); ax[1].set_ylabel(r"curvature [m$^{-1}$]"); ax[1].set_title("Path curvature"); ax[1].legend(
      fontsize=7.5)
    savefig(fig, "fig_lattice")
```

```
    print("exp5 done")


if __name__ == "__main__":
    main()
```

experiments/exp6_hybrid_sweep.py

```
"""Experiment 6 -- Hybrid A* parallel parking: slot-length sweep and search-speed study (heuristic weight)."""
from common import *  # noqa
import sys
from exp5_planning import VEH, jitter, kappa_pose_path
from sdc.geometry import wrap
from sdc.planning.hybrid_astar import FootprintChecker, HybridAStar
from sdc.planning.scenarios import parallel_parking

SLOTS = [5.5, 6.0, 6.5, 7.0, 7.5]
CONFIGS = {"baseline": (1.0, 1e9), "w = 1.5": (1.5, 1e9), "w = 2.0": (2.0, 1e9), "Dubins h <= 10 m": (1.0, 10.0)}
STARTS = range(6)                       # same 6 start poses as E5 (index 0 = nominal)


def run(args):
    slot, ti, cfg = args
    hw, rad = CONFIGS[cfg]
    sc = parallel_parking(VEH, slot=slot)
    chk = FootprintChecker(sc.grid, VEH)
    start = sc.start if ti == 0 else jitter(sc, np.random.default_rng(100 + ti), chk)
    r = HybridAStar(sc.grid, VEH, max_exp=60000, h_weight=hw, nh_radius=rad).plan(start, sc.goal, time_limit=40)
    out = dict(slot=slot, ti=ti, cfg=cfg, found=bool(r.success), feasible=False, time=float(r.time_s), exp=int(r.expansions))
    if r.success:
        p = r.path
        k = kappa_pose_path(p)
        coll = bool((~chk.exact_free(p[:, 0], p[:, 1], p[:, 2])).any())
        end = float(np.hypot(*(p[-1, :2] - sc.goal[:2])))
        eth = float(np.degrees(abs(wrap(p[-1, 2] - sc.goal[2]))))
        out.update(feasible=bool((not coll) and k.max() <= 1.05 * VEH.kappa_max and end < 0.35 and eth < 6),
                   length=float(np.sum(np.hypot(*np.diff(p[:, :2], axis=0).T))),
                   cusps=int(np.sum(p[1:, 3] * p[:-1, 3] < 0)), end_err=end)
    return out


def agg(rows):
    ok = [r for r in rows if r["feasible"]]
    return dict(n=len(rows), feasible=len(ok),
                time_med=float(np.median([r["time"] for r in ok])) if ok else None,
                time_mean=float(np.mean([r["time"] for r in ok])) if ok else None,
                exp_mean=float(np.mean([r["exp"] for r in ok])) if ok else None,
                length_mean=float(np.mean([r["length"] for r in ok])) if ok else None,
                cusps_mean=float(np.mean([r["cusps"] for r in ok])) if ok else None)


def main():
    speed_jobs = [(6.2, ti, c) for c in CONFIGS for ti in STARTS]
    if "--replot" in sys.argv:                          # redraw the figure from results/exp6_hybrid.json
        rows = load_json("exp6_hybrid")["rows"]
    elif "--speed-only" in sys.argv:                      # keep the sweep rows from the previous run
        old = [dict(r, cfg="baseline") for r in load_json("exp6_hybrid")["rows"] if r.get("hw", 1.0) == 1.0 and r.get("cfg", "baseline"
    ) == "baseline" and r["slot"] in SLOTS]
        rows = old + pool_map(run, speed_jobs)
    else:
        rows = pool_map(run, [(sl, ti, "baseline") for sl in SLOTS for ti in STARTS] + speed_jobs)
    sweep = {str(sl): agg([r for r in rows if r["slot"] == sl and r["cfg"] == "baseline"]) for sl in SLOTS}
    speed = {c: agg([r for r in rows if r["slot"] == 6.2 and r["cfg"] == c and r in rows[-len(speed_jobs):]]) for c in CONFIGS}
    save_json("exp6_hybrid", dict(slots=SLOTS, configs=list(CONFIGS), sweep=sweep, speed=speed, rows=rows))
    fig, ax = plt.subplots(1, 3, figsize=(11, 2.9))
    a = ax[0]
    a.plot(SLOTS, [sweep[str(s)]["feasible"] for s in SLOTS], "o-", color=BLUE)
    a.set_ylim(-0.3, 6.3); a.set_xlabel("slot length [m] (car 4.5 m)"); a.set_ylabel("feasible starts of 6")
    a.set_title("Success vs slot length")
    b = ax[1]
    for sl in SLOTS:
        t = [r["time"] for r in rows if r["slot"] == sl and r["cfg"] == "baseline" and r["feasible"]]
        b.plot([sl] * len(t), t, "o", color=BLUE, alpha=0.6, ms=4)
    b.set_xticks(SLOTS); b.set_xlabel("slot length [m]"); b.set_ylabel("solve time [s] (feasible runs)"); b.set_title("Search time")
    c = ax[2]
    last = rows[-len(speed_jobs):]
    for j, cf in enumerate(CONFIGS):
        sub = [r for r in last if r["cfg"] == cf]
        c.scatter([j] * len(sub), [r["time"] for r in sub], c=[BLUE if r["feasible"] else ORANGE for r in sub], s=18, zorder=3)
        c.text(j, 44, f"{sum(r['feasible'] for r in sub)}/6", ha="center", fontsize=8, color=INK2)
    c.set_xticks(range(len(CONFIGS))); c.set_xticklabels(list(CONFIGS), fontsize=7, rotation=12); c.set_ylim(0, 48)
    c.set_ylabel("time [s]"); c.set_title("Search speed-up (slot 6.2 m)")
    c.text(0.02, 0.04, "blue = feasible, orange = not", transform=c.transAxes, fontsize=7, color=INK2)
    savefig(fig, "fig_hybrid_sweep")
    print("exp6 done")


if __name__ == "__main__":
    main()
```

experiments/exp7_lattice_failures.py

```python
"""Experiment 7 -- why does the quintic lattice fail in some random layouts?  Solvability (Frenet DP) and variants."""
from common import *  # noqa
from exp5_planning import VEH
from sdc.planning.lattice import Frenet, LatticeConfig, StateLattice, run_receding_horizon
from sdc.sim.world import World

N, S0, S1 = 20, 30.0, 360.0


def make_layout(seed, fr):                      # identical to exp5_planning.lattice_mc
    rng = np.random.default_rng(seed)
    obs = []
    for s in np.sort(rng.uniform(60, 330, rng.integers(5, 9))):
        d = rng.choice([-1.75, -1.75, -1.0, -2.6, 1.75])
        if rng.random() < 0.6:
            for o in (-1.5, 0.0, 1.5):
                obs.append([*fr.to_world(np.array([s + o]), np.array([d]))[0][0], 1.0])
        else:
            obs.append([*fr.to_world(np.array([s]), np.array([d]))[0][0], rng.uniform(0.3, 0.5)])
    return np.array(obs)


def gap_map(pl, fr, ss, dd):
    """Clearance of the 3-disc ego body at road heading for every (s,d) cell -> (len(ss), len(dd))."""
    S, D = np.meshgrid(ss, dd, indexing="ij")
    pos, th, _ = fr.to_world(S.ravel(), D.ravel())
    offs = np.array(pl.cfg.ego_discs)
    cx = pos[:, 0:1] + np.cos(th)[:, None] * offs
    cy = pos[:, 1:2] + np.sin(th)[:, None] * offs
    g = np.hypot(cx[:, :, None] - pl.obs[None, None, :, 0], cy[:, :, None] - pl.obs[None, None, :, 1]) \
        - pl.cfg.ego_radius - pl.obs[None, None, :, 2]
    return g.min(axis=(1, 2)).reshape(S.shape)


def point_gap(pl, pos, th):
    offs = np.array(pl.cfg.ego_discs)
    cx, cy = pos[:, 0:1] + np.cos(th)[:, None] * offs, pos[:, 1:2] + np.sin(th)[:, None] * offs
    g = np.hypot(cx[:, :, None] - pl.obs[None, None, :, 0], cy[:, :, None] - pl.obs[None, None, :, 1]) \
        - pl.cfg.ego_radius - pl.obs[None, None, :, 2]
    return g.min(axis=(1, 2))


def reachable(valid, d0_idx, max_step=2):
    """Forward reachability over s with |d-index change| <= max_step per column (lateral slope <= 0.4)."""
    r = np.zeros(valid.shape[1], bool); r[d0_idx] = valid[0, d0_idx]
    for i in range(1, valid.shape[0]):
        nr = np.zeros_like(r)
        for k in range(-max_step, max_step + 1):
            nr |= np.roll(r, k)
        r = nr & valid[i]
        if not r.any():
            return False, i
    return True, valid.shape[0]


def evaluate(w, obs, cfg=None, replan=4.0):
    pl = StateLattice(w.route, obs, cfg or LatticeConfig())
    r = run_receding_horizon(w.route, pl, S0, S1, replan_every=replan)
    cl = pl.clearance(r["pos"], r["theta"])
    return dict(ok=bool(cl > 0 and np.abs(r["kappa"]).max() <= 1.05 * VEH.kappa_max), clearance=float(cl),
                fails=int(r["fails"]), max_kappa=float(np.abs(r["kappa"]).max()), r, pl


def one(seed):
    w = World(seed=7); fr = Frenet(w.route)
    obs = make_layout(seed, fr)
    nom, r, pl = evaluate(w, obs)
    out = dict(seed=seed, n_obs=int(len(obs)), nominal=nom)
    ss, dd = np.arange(S0, S1 + 0.5, 0.5), np.arange(-3.1, 3.1001, 0.1)
    gm = gap_map(pl, fr, ss, dd)
    d0 = int(np.argmin(abs(dd + 1.75)))
    out["solvable_collision_free"] = reachable(gm > 0.0, d0)[0]
    out["solvable_margin"] = reachable(gm >= pl.cfg.margin, d0)[0]
    if not nom["ok"]:
        dense = LatticeConfig(d_offsets=tuple(np.linspace(-3.0, 3.0, 25)), horizons=(12.0, 18.0, 26.0, 36.0, 50.0))
        out["dense"] = evaluate(w, obs, dense)[0]
        out["frequent"] = evaluate(w, obs, None, replan=2.0)[0]
        out["dense_frequent"] = evaluate(w, obs, dense, replan=2.0)[0]
        out["keep"] = dict(ss=ss, dd=dd, gm=gm, path_s=r["s"], path_d=r["d"], obs=obs, fail_s=float(r["s"][int(np.argmax(point_gap(pl,
          r["pos"], r["theta"]) <= 0))]))
    return out


def main():
    res = pool_map(one, list(range(N)))
    bad = [r for r in res if not r["nominal"]["ok"]]
    summary = dict(n=N, nominal_ok=N - len(bad), failing=len(bad),
                   failing_solvable_collision_free=sum(r["solvable_collision_free"] for r in bad),
                   failing_solvable_margin=sum(r["solvable_margin"] for r in bad),
                   dense_ok=sum(r["dense"]["ok"] for r in bad), frequent_ok=sum(r["frequent"]["ok"] for r in bad),
                   dense_frequent_ok=sum(r["dense_frequent"]["ok"] for r in bad),
                   ok_layouts_unsolvable=sum(1 for r in res if r["nominal"]["ok"] and not r["solvable_margin"]))
    rows = [{k: v for k, v in r.items() if k != "keep"} for r in res]
    save_json("exp7_lattice", dict(summary=summary, rows=rows))
```

```
    fig, ax = plt.subplots(1, max(len(bad), 1), figsize=(2.9 * max(len(bad), 1), 2.6), squeeze=False)
    fr = Frenet(World(seed=7).route)
    for a, r in zip(ax[0], bad):
        k = r["keep"]
        a.imshow((k["gm"] > 0.0).T, origin="lower", aspect="auto", cmap="Greys_r", vmin=-0.4, vmax=1.4,
                 extent=[k["ss"][0], k["ss"][-1], k["dd"][0] - 0.05, k["dd"][-1] + 0.05])
        a.plot(k["path_s"], k["path_d"], color=BLUE, lw=1.2)
        a.axvline(k["fail_s"], color=ORANGE, lw=0.8, ls="--")
        a.set_title(f"layout {r['seed']}", fontsize=8.5)
        a.set_xlabel("s [m]"); a.grid(False)
        tag = "solvable" if r["solvable_margin"] else ("solvable only w/o margin" if r["solvable_collision_free"] else "no path")
        a.text(0.02, 0.04, tag, transform=a.transAxes, fontsize=7, color=ORANGE if not r["solvable_margin"] else AQUA,
               bbox=dict(fc=SURF, ec="none", pad=1))
    ax[0][0].set_ylabel("lateral offset d [m]")
    savefig(fig, "fig_lattice_failures")
    print("exp7 done", summary)


if __name__ == "__main__":
    main()
```

experiments/exp8_sensitivity.py

```
"""Experiment 8 -- 20-seed statistics with bootstrap CIs: E2 (SLAM) and E3 (HD-map localisation) under LiDAR degradation."""
from common import *  # noqa
import sys
import exp2_slam as e2
import exp3_hdmap as e3
from sdc.sim.drive import LidarParams, VoParams, simulate_drive, simulate_lidar, simulate_vo

N = 20
LP = {"nominal": LidarParams(), "noise x4": LidarParams(sigma0=0.08, sigma1=0.004),
      "dropout 40%": LidarParams(dropout=0.40)}


def boot(x, n=5000, seed=0):
    """Mean and 95% percentile-bootstrap CI of the mean."""
    x = np.asarray(x, float)
    m = np.random.default_rng(seed).choice(x, (n, len(x))).mean(1)
    return dict(mean=float(x.mean()), lo=float(np.percentile(m, 2.5)), hi=float(np.percentile(m, 97.5)),
                median=float(np.median(x)), std=float(x.std(ddof=1)), n=len(x))


def slam_job(seed):
    out, _ = e2.one(seed)
    return {m: out[m]["rmse"] for m in ("icp", "ndt_odom", "ndt_loop")}


def loc_job(args):
    cfg, seed = args
    w = e3._W
    d = simulate_drive(w, laps=1.25, seed=1 + seed)
    rng = np.random.default_rng(10 + seed)
    dyn = w.dynamic_objects(15, np.random.default_rng(20 + seed))
    idx, scans = simulate_lidar(d, w, rng, extra_segments=dyn, p=LP[cfg])
    vo = simulate_vo(d, w, VoParams(), rng)
    gt = d.pose[idx]
    est, _ = e3.run_ndt(scans, vo, gt)
    sn, _ = e3.stats(est, gt)
    est2, _, _ = e3.run_pf(scans, vo, gt)
    sp, _ = e3.stats(est2, gt)
    r = lambda s: float(np.hypot(s["lon_rms"], s["lat_rms"]))
    return dict(cfg=cfg, seed=seed, ndt=r(sn), pf=r(sp), ndt_max=sn["max"], pf_max=sp["max"])


def main():
    if "--replot" in sys.argv:                          # redraw the figure from results/exp8_sensitivity.json
        e8 = load_json("exp8_sensitivity")
        slam, s_stats, l_stats = e8["slam"]["raw"], e8["slam"], e8["loc"]
    else:
        e3.hdmap()
        slam = pool_map(slam_job, list(range(N)))
        s_stats = {m: boot([r[m] for r in slam]) for m in ("icp", "ndt_odom", "ndt_loop")}
        s_stats["raw"] = slam
        loc = pool_map(loc_job, [(c, s) for c in LP for s in range(N)])
        l_stats = {}
        for c in LP:
            sub = [r for r in loc if r["cfg"] == c]
            l_stats[c] = {m: boot([r[m] for r in sub]) for m in ("ndt", "pf")}
            l_stats[c]["ndt_fail"] = int(sum(r["ndt_max"] > 1.0 for r in sub))
            l_stats[c]["pf_fail"] = int(sum(r["pf_max"] > 1.0 for r in sub))
    if "--replot" not in sys.argv:
        save_json("exp8_sensitivity", dict(n=N, slam=s_stats, loc=l_stats, loc_raw=loc))
    fig, ax = plt.subplots(1, 2, figsize=(9.5, 3.0))
    a = ax[0]
    for j, (m, lab, c) in enumerate((("icp", "ICP odometry", YELLOW), ("ndt_odom", "NDT odometry", ORANGE), ("ndt_loop", "NDT + loop",
      BLUE))):
        v = [r[m] for r in slam]
        a.scatter(np.full(len(v), j) + np.random.default_rng(j).uniform(-0.12, 0.12, len(v)), v, s=10, color=c, alpha=0.7, zorder=3)
        a.errorbar(j, s_stats[m]["mean"], yerr=[[s_stats[m]["mean"] - s_stats[m]["lo"]], [s_stats[m]["hi"] - s_stats[m]["mean"]]],
                   color=INK, capsize=4, marker="_", ms=14, zorder=4)
    a.set_yscale("log"); a.set_xticks(range(3)); a.set_xticklabels(["ICP", "NDT odom.", "NDT + loop"])
```

```
    a.set_ylabel("ATE RMSE [m]"); a.set_title(f"E2, {N} seeds (mean, 95% CI)")
    b = ax[1]
    for j, (m, lab, c) in enumerate((("ndt", "NDT", BLUE), ("pf", "PF (500)", ORANGE))):
        for i, cfg in enumerate(LP):
            s = l_stats[cfg][m]
            b.errorbar(i + (j - 0.5) * 0.18, 100 * s["mean"], yerr=[[100 * (s["mean"] - s["lo"])], [100 * (s["hi"] - s["mean"])]],
                       color=c, marker="o", ms=4, capsize=3, label=lab if i == 0 else None)
    b.set_ylim(0, 5.5); b.set_xticks(range(len(LP))); b.set_xticklabels(list(LP)); b.set_ylabel("position RMS [cm]")
    b.set_title(f"E3 under LiDAR degradation, {N} seeds"); b.legend()
    savefig(fig, "fig_sensitivity")
    print("exp8 done")


if __name__ == "__main__":
    import time  # noqa  (exp2_slam.one uses `time` from the script namespace)
    e2.time = time
    main()
```

experiments/exp9_closed_loop.py

```
"""Experiment 9 -- closed loop: localisation -> behaviour tree -> state lattice -> pure pursuit -> kinematic bicycle.

Three pose sources feed the same stack: ground truth, NDT against the HD map (with simulated VO prior) and VO dead reckoning.
Perception of the 7 obstacles is perfect (known world positions) but the obstacles are *absent from the HD map*, so
the NDT localizer sees them as unmapped objects.
"""
from common import *  # noqa
from exp3_hdmap import hdmap, _W
from exp5_planning import VEH
from sdc.behavior.bt import Action, Condition, Fallback, Sequence, Status
from sdc.geometry import between, compose, wrap
from sdc.localization.particle_filter import NdtLocalizer
from sdc.planning.lattice import Frenet, StateLattice
from sdc.sim.drive import lidar_scan

S_START, S_END, DT, L = 30.0, 360.0, 0.05, VEH.wheelbase
MODES = ["gt", "ndt", "dr"]
LABEL = {"gt": "ground truth", "ndt": "NDT on HD map", "dr": "VO dead reckoning"}


def layout(fr):
    def cd(s, d): return [[*fr.to_world(np.array([s + o]), np.array([d]))[0][0], 1.0] for o in (-1.5, 0.0, 1.5)]
    def dc(s, d, r): return [[*fr.to_world(np.array([s]), np.array([d]))[0][0], r]]
    obs = np.array(cd(80, -1.75) + dc(120, -1.0, 0.3) + cd(150, -1.75) + dc(190, -2.6, 0.4) + cd(235, -1.75)
                   + cd(300, -1.75) + cd(318, 1.75))
    s_obs = np.array([80, 120, 150, 190, 235, 300, 318], float)
    return obs, s_obs


def octagons(obs):
    a = np.linspace(0, 2 * np.pi, 9)
    segs = []
    for x, y, r in obs:
        p = np.stack([x + r * np.cos(a), y + r * np.sin(a)], 1)
        segs.append(np.hstack([p[:-1], p[1:]]))
    return np.concatenate(segs)


def to_frenet(route, xy, th, s_hint):
    idx = np.where(np.abs(route.s - s_hint) < 25)[0]
    k = idx[np.argmin(np.hypot(*(route.xy[idx] - xy).T))]
    t = route.theta[k]
    dxy = xy - route.xy[k]
    d = -np.sin(t) * dxy[0] + np.cos(t) * dxy[1]
    s = route.s[k] + np.cos(t) * dxy[0] + np.sin(t) * dxy[1]
    return float(s), float(d), float((1 - route.kappa[k] * d) * np.tan(wrap(th - t)))


def build_bt():
    def setv(v):
        def f(bb):
            bb["v_ref"] = v
            return Status.RUNNING
        return f
    return Fallback("drive", [
        Sequence("blocked", [Condition("no_feasible_plan", lambda bb: bb["no_plan"]), Action("stop", setv(0.0))]),
        Sequence("avoid", [Condition("obstacle_ahead", lambda bb: bb["obs_ahead"] < 45.0), Action("slow_nudge", setv(5.0))]),
        Action("cruise", setv(8.0))])


def vo_measure(prev, cur, rng, s, scale):
    rel = between(prev, cur)
    k = _W.vision_factor(s)
    d = np.hypot(rel[0], rel[1])
    st, sr = (0.01 + 0.01 * d) * k, (0.0015 + 0.002 * abs(rel[2])) * k
    return np.array([rel[0] * scale, rel[1] * scale, rel[2]]) + rng.normal(0, 1, 3) * np.array([st, st, sr])


def run(args):
    mode, seed = args
    rng = np.random.default_rng(500 + seed)
    fr = Frenet(_W.route)
```

```
    obs, s_obs = layout(fr)
    segs = octagons(obs)
    pl = StateLattice(_W.route, obs)
    bt, bb = build_bt(), {"no_plan": False, "obs_ahead": 99.0, "v_ref": 8.0}
    p0, th0, _ = fr.to_world(np.array([S_START]), np.array([-1.75]), np.array([0.0]))
    true = np.array([p0[0, 0], p0[0, 1], th0[0]])
    est, prev, v, delta, scale = true.copy(), true.copy(), 0.0, 0.0, 1.0 + rng.normal(0, 0.004)
    loc = NdtLocalizer(hdmap(), true) if mode == "ndt" else None
    plan_xy, s_est, n_ctl = None, S_START, 0
    tr, e_loc, cte, tags, collided, min_clr, no_plan_ct = [], [], [], [], False, np.inf, 0
    t = 0.0
    while t < 120.0:
        if n_ctl % 2 == 0:                                      # 10 Hz pose sample
            if mode == "gt":
                est = true.copy()
            else:
                z = vo_measure(prev, true, rng, s_est, scale)
                if mode == "ndt":
                    est = loc.step(lidar_scan(_W, true, rng, extra_segments=segs), z)[0].copy()
                else:
                    est = compose(est, z)
            prev = true.copy()
            e_loc.append(float(np.hypot(*(est[:2] - true[:2]))))
        if n_ctl % 10 == 0:                                     # 2 Hz decision + planning
            s_est, d_est, dp_est = to_frenet(_W.route, est[:2], est[2], s_est)
            ahead = s_obs[s_obs > s_est - 3.0] - s_est
            bb["obs_ahead"] = float(ahead.min()) if len(ahead) else 99.0
            bt.tick(bb)
            best, _ = pl.plan(s_est, d_est, dp_est, 0.0)
            bb["no_plan"] = best is None
            no_plan_ct += best is None
            if best is not None:
                x = np.arange(0.0, best["Sf"] + 1e-9, 0.5)
                dd, dpp, _, _ = best["fn"](x)
                plan_xy = fr.to_world(s_est + x, dd, dpp)[0]
            tags.append(bb.get("active_leaf", "?"))
        # pure pursuit on the estimated pose
        if plan_xy is not None:
            k0 = int(np.argmin(np.hypot(*(plan_xy - est[:2]).T)))
            Ld = float(np.clip(1.0 * v + 3.0, 4.0, 10.0))
            cum = np.r_[0, np.cumsum(np.hypot(*np.diff(plan_xy, axis=0).T))]
            tgt = plan_xy[min(np.searchsorted(cum, cum[k0] + Ld), len(plan_xy) - 1)]
            al = wrap(np.arctan2(tgt[1] - est[1], tgt[0] - est[0]) - est[2])
            d_cmd = float(np.clip(np.arctan2(2 * L * np.sin(al), Ld), -VEH.delta_max, VEH.delta_max))
            delta += float(np.clip(d_cmd - delta, -0.6 * DT, 0.6 * DT))
            cte.append(float(np.min(np.hypot(*(plan_xy - true[:2]).T))))
        a = float(np.clip((bb["v_ref"] - v) / 1.0, -4.0, 2.0))
        v = max(0.0, v + a * DT)
        true = true + DT * v * np.array([np.cos(true[2]), np.sin(true[2]), np.tan(delta) / L])
        true[2] = wrap(true[2])
        t += DT; n_ctl += 1
        clr = pl.clearance(true[None, :2], np.array([true[2]])) + (1.15 - VEH.width / 2)   # physical body radius = half width
        min_clr = min(min_clr, float(clr))
        s_t, d_t, _ = to_frenet(_W.route, true[:2], true[2], s_est)
        tr.append((s_t, d_t))
        if clr < 0:
            collided = True; break
        if s_t >= S_END or abs(d_t) > 6.0:
            break
    tr = np.array(tr)
    reached = bool(tr[-1, 0] >= S_END)
    out = dict(mode=mode, seed=seed, success=bool(reached and not collided), collided=bool(collided), reached=reached,
               min_clearance=float(min_clr), cte_rms=float(np.sqrt(np.mean(np.square(cte)))) if cte else None,
               cte_max=float(np.max(cte)) if cte else None, loc_rms=float(np.sqrt(np.mean(np.square(e_loc)))),
               loc_max=float(np.max(e_loc)), no_plan=int(no_plan_ct), time=float(t),
               tags={k: tags.count(k) for k in set(tags)})
    return out, (tr, np.array(e_loc), t) if seed == 0 else None


def main():
    hdmap()
    res = pool_map(run, [(m, s) for m in MODES for s in range(10)])
    agg = {}
    for m in MODES:
        r = [x[0] for x in res if x[0]["mode"] == m]
        agg[m] = dict(n=len(r), success=sum(x["success"] for x in r), collisions=sum(x["collided"] for x in r),
                      min_clearance=ms([x["min_clearance"] for x in r]), loc_rms=ms([x["loc_rms"] for x in r]),
                      loc_max=ms([x["loc_max"] for x in r]),
                      cte_rms=ms([x["cte_rms"] for x in r if x["cte_rms"] is not None]),
                      no_plan=ms([x["no_plan"] for x in r]))
    save_json("exp9_closed_loop", dict(agg=agg, runs=[x[0] for x in res]))
    fig, ax = plt.subplots(1, 2, figsize=(11, 3.0), gridspec_kw=dict(width_ratios=[1.5, 1]))
    a = ax[0]
    for sc_, d_, r_ in ((80, -1.75, 1.0), (120, -1.0, 0.3), (150, -1.75, 1.0), (190, -2.6, 0.4), (235, -1.75, 1.0), (300, -1.75, 1.0),
      (318, 1.75, 1.0)):
        a.add_patch(plt.Circle((sc_, d_), r_, color=SHADE, zorder=1))
        if r_ >= 1.0:
            for o in (-1.5, 1.5):
                a.add_patch(plt.Circle((sc_ + o, d_), r_, color=SHADE, zorder=1))
    for m, col, ls in (("gt", INK, "-"), ("ndt", BLUE, "--"), ("dr", ORANGE, "-.")):
        tr = [x[1][0] for x in res if x[0]["mode"] == m and x[0]["seed"] == 0][0]
        a.plot(tr[:, 0], tr[:, 1], color=col, ls=ls, lw=1.3, label=LABEL[m])
    a.set_xlim(S_START, S_END); a.set_ylim(-4.2, 3.2); a.set_xlabel("route position s [m]"); a.set_ylabel("lateral offset d [m]")
    a.set_title("True path, seed 0 (grey = obstacles)"); a.legend(fontsize=7, ncol=3, loc="lower center")
    b = ax[1]
    for m, col, ls in (("ndt", BLUE, "--"), ("dr", ORANGE, "-.")):
```

```
        e = [x[1][1] for x in res if x[0]["mode"] == m and x[0]["seed"] == 0][0]
        b.plot(np.arange(len(e)) * 0.1, np.maximum(e, 1e-3), color=col, ls=ls, label=LABEL[m])
    b.set_yscale("log"); b.set_xlabel("time [s]"); b.set_ylabel("localisation error [m]"); b.set_title("Localisation error in the loop"
      )
    b.legend(fontsize=7.5)
    savefig(fig, "fig_closed_loop")
    print("exp9 done")


if __name__ == "__main__":
    main()
```

tests/test_core.py

```
"""Fast unit tests for the core algorithms (run with `pytest -q`)."""
import numpy as np
import pytest

from sdc.behavior.bt import Action, Condition, Fallback, Sequence, Status
from sdc.geometry import between, compose, inverse, wrap
from sdc.localization.ndt import NDTGrid, ndt_register_multires
from sdc.localization.pose_graph import Edge, optimize
from sdc.planning.dubins import dubins_path, reverse_dubins_path
from sdc.sim.drive import lidar_scan
from sdc.sim.world import World


def test_se2_algebra():
    a, b = np.array([1.0, 2.0, 0.7]), np.array([-3.0, 0.5, -2.0])
    assert np.allclose(compose(a, between(a, b)), b, atol=1e-12)
    assert np.allclose(compose(a, inverse(a)), 0, atol=1e-12)
    assert abs(wrap(3 * np.pi) - np.pi) < 1e-12 or abs(wrap(3 * np.pi) + np.pi) < 1e-12


@pytest.mark.parametrize("fn", [dubins_path, reverse_dubins_path])
def test_dubins_reaches_goal_and_respects_curvature(fn):
    rng, rho = np.random.default_rng(0), 4.7
    for _ in range(100):
        q0 = np.r_[rng.uniform(-20, 20, 2), rng.uniform(-np.pi, np.pi)]
        q1 = np.r_[rng.uniform(-20, 20, 2), rng.uniform(-np.pi, np.pi)]
        pts, L = fn(q0, q1, rho, 0.05)
        assert np.hypot(*(pts[-1, :2] - q1[:2])) < 0.06 and np.hypot(*(pts[0, :2] - q0[:2])) < 0.06
        d = np.diff(np.unwrap(pts[:, 2])); ds = np.hypot(*np.diff(pts[:, :2], axis=0).T)
        assert np.abs(d / np.maximum(ds, 1e-9)).max() < 1.05 / rho


def test_pose_graph_loop_closure_reduces_drift():
    rng, n = np.random.default_rng(0), 80
    gt = np.zeros((n, 3))
    for k in range(1, n):
        gt[k] = compose(gt[k - 1], [1.5, 0, 2 * np.pi / 70])
    om = np.diag([1 / 0.05 ** 2, 1 / 0.05 ** 2, 1 / 0.01 ** 2])
    est, edges = np.zeros((n, 3)), []
    for k in range(1, n):
        z = between(gt[k - 1], gt[k]) + rng.normal(0, [0.03, 0.03, 0.006])
        edges.append(Edge(k - 1, k, z, om)); est[k] = compose(est[k - 1], z)
    loop = Edge(0, n - 1, between(gt[0], gt[n - 1]), np.diag([1 / 0.1 ** 2, 1 / 0.1 ** 2, 1 / 0.02 ** 2]), "loop")
    opt, _ = optimize(est, edges + [loop])
    err = lambda p: np.hypot(*(p[:, :2] - gt[:, :2]).T).max()
    assert err(opt) < 0.5 * err(est)


def test_ndt_recovers_known_offset():
    w, rng = World(seed=7), np.random.default_rng(0)
    A, B = np.array([60.0, 0.0, 0.0]), np.array([61.0, 0.2, 0.02])
    from sdc.geometry import transform_points
    sA, sB = lidar_scan(w, A, rng), lidar_scan(w, B, rng)
    grids = [NDTGrid(transform_points(A, sA), 3.0), NDTGrid(transform_points(A, sA), 1.5)]
    res = ndt_register_multires(sB, grids, B + np.array([0.4, -0.3, 0.05]))
    assert np.hypot(*(res.pose[:2] - B[:2])) < 0.3 and abs(wrap(res.pose[2] - B[2])) < 0.02


def test_bt_priority_and_preemption():
    bb = {"flag": False, "log": []}
    hi = Sequence("hi", [Condition("c", lambda b: b["flag"]), Action("high", lambda b: (b["log"].append("H"), Status.RUNNING)[1])])
    lo = Action("low", lambda b: (b["log"].append("L"), Status.RUNNING)[1])
    root = Fallback("root", [hi, lo])
    root.tick(bb); bb["flag"] = True; root.tick(bb)
    assert bb["log"] == ["L", "H"] and lo.status == Status.IDLE        # low-priority branch was halted
```

make_tables.py

```
"""Generate numbers.tex (headline-number macros) and tables/*.tex (one file per table) from results/*.json.

Re-run after any experiment:  python make_tables.py   -> then recompile main.tex.
"""
import json
```

```
from pathlib import Path

ROOT = Path(__file__).resolve().parent
R = ROOT / "results"
(ROOT / "tables").mkdir(exist_ok=True)
J = lambda n: json.loads((R / f"{n}.json").read_text())
out = []


def v(x, nd=2, scale=1.0):
    m = x["mean"] if isinstance(x, dict) else (x[0] if isinstance(x, (list, tuple)) else x)
    return f"{m * scale:.{nd}f}"


def pm(x, nd=2, scale=1.0):
    if isinstance(x, dict):
        m, s = x["mean"], x["std"]
    else:
        m, s = x[0], x[1]
    return f"{m * scale:.{nd}f}$\\pm${s * scale:.{nd}f}"


def macro(name, val):
    out.append(f"\\newcommand{{\\{name}}}{{{val}}}")


def table(label, caption, cols, rows, spec=None, note=None):
    spec = spec or "l" + "r" * (len(cols) - 1)
    t = ["% AUTO-GENERATED by make_tables.py", "\\begin{table}[H]\\centering\\small",
         f"\\caption{{{caption}}}\\label{{{label}}}", "\\begin{adjustbox}{max width=\\linewidth}", f"\\begin{{tabular}}{{{spec}}}\\
      toprule",
         " & ".join(cols) + " \\\\ \\midrule"]
    t += [" & ".join(r) + " \\\\" for r in rows]
    t.append("\\bottomrule\\end{tabular}\\end{adjustbox}")
    if note:
        t.append(f"\\par\\vspace{{2pt}}{{\\footnotesize {note}}}")
    t.append("\\end{table}")
    (ROOT / "tables" / (label.split(":")[1] + ".tex")).write_text("\n".join(t) + "\n")


# ---------------------------------------------------------------- E1 VIO
e1 = J("exp1_vio"); T = e1["table"]
names = {"imu_only": "IMU only", "vo_only": "Visual odometry only", "gps_imu": "GPS + IMU", "vio": "GPS + IMU + VO (VIO)",
         "vio_nogps": "VIO, GPS only first 10\\,s"}
rows = [[names[m], pm(T[m]["rmse"]), pm(T[m]["max_tunnel"]), pm(T[m]["exit_tunnel"]), pm(T[m]["max_underpass"]), v(T[m]["yaw_rmse_deg"
    ])]
        for m in names]
table("tab:vio", f"E1: position error [m] over a 1.25-lap drive ({e1['n_runs']} Monte-Carlo runs, mean$\\pm$std).",
      ["Estimator", "RMSE", "Max in tunnel", "Tunnel exit", "Max underpass", "Yaw RMSE [deg]"], rows)
sw = e1["vo_noise_sweep"]
table("tab:vio_sweep", "E1: VIO tunnel-exit error vs.\\ VO noise scale (6 runs each).", ["VO noise scale"] + [f"$\\times${k}" for k in
    sw],
      [["Tunnel-exit error [m]"] + [v(sw[k]["exit"]) for k in sw], ["Max tunnel error [m]"] + [v(sw[k]["maxt"]) for k in sw]],
      spec="l" + "r" * len(sw))
macro("vioExitVio", v(T["vio"]["exit_tunnel"])); macro("vioExitGps", v(T["gps_imu"]["exit_tunnel"]))
macro("vioExitVo", v(T["vo_only"]["exit_tunnel"])); macro("vioExitImu", v(T["imu_only"]["exit_tunnel"], 0))
macro("vioRmseVio", v(T["vio"]["rmse"])); macro("vioDriftPct", v(T["vio_nogps"]["drift_pct"], 2))
macro("vioSigmaTunnel", v(T["vio"]["tunnel_sigma_mean"])); macro("vioInThreeSigma", v(T["vio"]["within_3sigma_tunnel"], 0, 100)); macro
    ("vioTunnelRmse", v(T["vio"]["rmse_tunnel"]))

# ---------------------------------------------------------------- E2 SLAM
S = J("exp2_slam")["table"]
nm = {"icp": "ICP odometry (scan-to-scan)", "ndt_odom": "NDT scan-to-submap odometry", "ndt_loop": "NDT + loop closure (pose graph)"}
rows = []
for m in nm:
    t = S[m]
    rows.append([nm[m], pm(t["rmse"]), pm(t["max"]), pm(t["final"]),
                 (v(t["map"]["mean"], 3) + " / " + v(t["map"]["p95"], 3)) if "map" in t else "--",
                 v(t["ms_per_scan"], 1) if "ms_per_scan" in t else "--", v(t["loops"], 0) if "loops" in t else "--"])
table("tab:slam", "E2: LiDAR SLAM, 1.25 laps, 3 seeds (mean$\\pm$std). Map error = distance of map points to noise-free scene geometry
    [m].",
      ["Method", "ATE RMSE [m]", "ATE max [m]", "Final [m]", "Map mean / P95 [m]", "ms/scan", "Loops"], rows)
macro("slamIcp", v(S["icp"]["rmse"], 1)); macro("slamNdt", v(S["ndt_odom"]["rmse"])); macro("slamLoop", v(S["ndt_loop"]["rmse"]))
macro("slamIcpYaw", v(S["icp"]["yaw_final_deg"], 0)); macro("slamMapOdom", v(S["ndt_odom"]["map"]["mean"], 3))
macro("slamMapLoop", v(S["ndt_loop"]["map"]["mean"], 3)); macro("slamLoops", v(S["ndt_loop"]["loops"], 0))
macro("slamDegTun", f"{S['degeneracy']['tunnel_median']:.3f}"); macro("slamDegOut", f"{S['degeneracy']['outside_median']:.3f}")
macro("slamDegRatio", f"{S['degeneracy']['outside_median'] / S['degeneracy']['tunnel_median']:.0f}")

# ---------------------------------------------------------------- E3 HD map
H = J("exp3_hdmap"); tb = H["table"]
rows = []
for n in ("0", "15", "40"):
    for m, lab in (("ndt", "NDT"), ("pf", "PF (500)"), ("vo", "VO dead reckoning")):
        x = tb[n][m]
        rows.append([n if m == "ndt" else "", lab, v(x["lon_rms"], 1, 100), v(x["lat_rms"], 1, 100), v(x["yaw_rms_deg"], 3),
                     v(x["p95"], 1, 100), v(x["max"], 1, 100), v(x["ms"], 1) if "ms" in x else "--"])
table("tab:hd", "E3: localisation against the HD map (3 seeds; mean). Errors in the vehicle frame [cm]; ``unmapped'' = vehicles absent
    from the map.",
      ["Unmapped", "Method", "Lon RMS", "Lat RMS", "Yaw RMS [deg]", "P95", "Max", "ms/scan"], rows, spec="llrrrrrr")
sw = H["pf_sweep"]
table("tab:pf", "E3: particle-count sweep (seed 0, 15 unmapped vehicles).", ["Particles", "Lon RMS [cm]", "Lat RMS [cm]", "ms/scan"],
      [[k, v(sw[k]["lon_rms"], 1, 100), v(sw[k]["lat_rms"], 1, 100), v(sw[k]["ms"], 1)] for k in sw])
g = H["global_loc"]
table("tab:glob", "E3: global localisation from a uniform prior (20\\,000 particles, 45\\,s).", ["Seed", "Converged at [s]", "Final
    error [m]"],
```

```
      [[str(x["seed"]), ("%.1f" % x["converge_s"]) if x["converge_s"] is not None else "not converged", "%.2f" % x["final"]] for x in g
      ])
macro("hdNdtLon", v(tb["15"]["ndt"]["lon_rms"], 1, 100)); macro("hdNdtLat", v(tb["15"]["ndt"]["lat_rms"], 1, 100))
macro("hdPfLon", v(tb["15"]["pf"]["lon_rms"], 1, 100)); macro("hdPfLat", v(tb["15"]["pf"]["lat_rms"], 1, 100))
macro("hdNdtMax", v(tb["15"]["ndt"]["max"], 0, 100)); macro("hdGlobOk", str(sum(x["converge_s"] is not None for x in g)))
macro("hdMapPts", f"{H['map_points'],}".replace(",", "\\,"))

# ---------------------------------------------------------------- E4 behaviour
B = J("exp4_behavior")
order = [("BT", "Behaviour tree"), ("FSM-basic", "FSM-basic (nominal transitions)"), ("FSM-full", "FSM-full (all pre-emption edges)"),
         ("FSM-full, no RB re-check", "FSM-full, re-check edge removed")]
rows = []
for k, lab in order:
    b = B[k]; c = b["collisions"]
    rows.append([lab, f"{b['success']}/{b['n']}", f"{b['success_surprise']}/{b['n_surprise']}",
                 ", ".join(f"{a} {n}" for a, n in c.items()) or "--", f"{b['time']:.1f}", f"{b['hard_brakes']:.2f}", str(b["emergency"
      ]), str(b["aborts"])])
table("tab:bt", "E4: 300 randomised scenarios (lead vehicle, oncoming traffic, roundabout, pedestrian). Success = route completed
      without collision.",
      ["Controller", "Success", "of which surprise", "Collisions", "Time [s]", "Hard brakes/run", "Emerg.", "Aborts"], rows, spec="
      lrrlrrrr",
      note="Surprise = an oncoming vehicle outside the 90\\,m perception range appears mid-overtake (150 of 300 scenarios).")
st = B["structure"]
table("tab:struct", "E4: structural size of the decision logic.", ["Quantity", "BT", "FSM (nominal)", "FSM (full)"],
      [["Nodes / states", str(st["bt_nodes"]), str(st["fsm_states"]), str(st["fsm_states"])],
       ["Transitions", "--", str(st["fsm_transitions_basic"]), str(st["fsm_transitions_full"])],
       ["Edits to add one global-priority behaviour", f"{st['bt_add_behaviour']} nodes", "--", f"{st['fsm_add_behaviour']} transitions"
      ]])
macro("btOk", str(B["BT"]["success"])); macro("fsmFullOk", str(B["FSM-full"]["success"])); macro("fsmBasicOk", str(B["FSM-basic"]["
      success"]))
macro("fsmNoRecheckOk", str(B["FSM-full, no RB re-check"]["success"])); macro("btAborts", str(B["BT"]["aborts"]))
macro("btNodes", str(st["bt_nodes"])); macro("fsmTrans", str(st["fsm_transitions_full"]))

# ---------------------------------------------------------------- E5 planning
P = J("exp5_planning"); rows = []
lab = {"hybrid": "Hybrid A*", "grid": "Grid A*", "grid_smooth": "Grid A* + Chaikin"}
for sc, row in P["parking"].items():
    for i, m in enumerate(("hybrid", "grid", "grid_smooth")):
        x = row[m]
        g_ = lambda k, nd=2: (f"{x[k]:.{nd}f}" if k in x else "--")
        rows.append([sc if i == 0 else "", lab[m], f"{x['feasible']}/{x['n']}", g_("time", 1) if m == "hybrid" else "--",
                     g_("cusps", 1) if m == "hybrid" else "--", g_("max_kappa"), g_("collision_frac") if m != "hybrid" else "0",
                     g_("goal_heading_err_deg", 0) if m != "hybrid" else "$<$6"])
table("tab:park", f"E5: tight-space planning, 6 start poses per scenario. Feasible = collision-free (exact footprint), curvature $\\le$
       {P['kappa_limit']:.3f}\\,m$^{{-1}}$, goal pose reached. Means over found paths.",
      ["Scenario", "Planner", "Feasible", "Time [s]", "Cusps", "Max $|\\kappa|$ [1/m]", "Collision frac.", "Goal head. err [deg]"],
      rows, spec="llrrrrrr")
L = P["lattice"]
table("tab:lat", "E5: receding-horizon lateral planning on 20 random obstacle layouts. OK = collision-free and curvature within the
      vehicle limit.",
      ["Profile", "OK", "Collision-free", "Max $|\\kappa|$ [1/m]", "Max $|d\\kappa/dl|$ [1/m$^2$]", "Replans w/o candidate"],
      [[p, f"{L[p]['ok']}/{L[p]['n']}", f"{L[p]['collision_free']}/{L[p]['n']}", pm(L[p]["max_kappa"], 3), pm(L[p]["max_dkappa"], 2),
      str(L[p]["plan_fails"])]
       for p in ("quintic", "linear")], note="Curvature rate depends on the 0.25\\,m sampling of the executed path for the kinked
      baseline.")
hy = {s: P["parking"][s]["hybrid"] for s in P["parking"]}
macro("hyPerp", f"{hy['Perpendicular parking']['feasible']}/{hy['Perpendicular parking']['n']}")
macro("hyPar", f"{hy['Parallel parking']['feasible']}/{hy['Parallel parking']['n']}")
macro("hyK", f"{hy['K-turn in a narrow street']['feasible']}/{hy['K-turn in a narrow street']['n']}")
macro("hyParTime", f"{hy['Parallel parking']['time']:.1f}"); macro("hyParErr", f"{hy['Parallel parking']['end_err'] * 100:.0f}")
macro("latQuinticOk", f"{L['quintic']['ok']}/{L['quintic']['n']}"); macro("latLinearOk", f"{L['linear']['ok']}/{L['linear']['n']}")
macro("latQuinticK", v(L["quintic"]["max_kappa"], 3)); macro("latLinearK", v(L["linear"]["max_kappa"], 2)); macro("kappaMax", f"{P['
      kappa_limit']:.3f}")

# ---------------------------------------------------------------- E6-E9 (extended experiments)
def ci(x, nd=2, scale=1.0):
    return f"{x['mean'] * scale:.{nd}f} [{x['lo'] * scale:.{nd}f}, {x['hi'] * scale:.{nd}f}]"


if (R / "exp6_hybrid.json").exists():
    e6 = J("exp6_hybrid")
    f1 = lambda x, nd=1: "--" if x is None else f"{x:.{nd}f}"
    table("tab:slot", "E6: Hybrid A* parallel parking vs.\\ slot length (car 4.5\\,m, 6 start poses per slot, 40\\,s / 60\\,000-
      expansion budget). Time, expansions, cusps and length are means over feasible runs.",
          ["Slot [m]", "Feasible", "Median time [s]", "Expansions", "Cusps", "Length [m]"],
          [[k, f"{a['feasible']}/{a['n']}", f1(a["time_med"]), f1(a["exp_mean"], 0), f1(a["cusps_mean"]), f1(a["length_mean"])]
           for k, a in e6["sweep"].items()], spec="rrrrrr")
    nm = {"baseline": "Baseline ($w=1$)", "w = 1.5": "Weighted A*, $w=1.5$", "w = 2.0": "Weighted A*, $w=2$",
          "Dubins h <= 10 m": "Dubins term only within 10\\,m"}
    table("tab:speed", "E6: search-speed variants on the nominal 6.2\\,m slot (same 6 start poses, run back to back on the same machine
      ).",
          ["Variant", "Feasible", "Median time [s]", "Expansions", "Length [m]"],
          [[nm[k], f"{a['feasible']}/{a['n']}", f1(a["time_med"]), f1(a["exp_mean"], 0), f1(a["length_mean"])] for k, a in e6["speed"].
      items()],
          spec="lrrrr")
    for i, k in enumerate(e6["sweep"]):
        macro("slotOk" + "ABCDE"[i], f"{e6['sweep'][k]['feasible']}/6")
    for suf, k in (("Base", "baseline"), ("Wa", "w = 1.5"), ("Wb", "w = 2.0"), ("Dub", "Dubins h <= 10 m")):
        a = e6["speed"][k]
        macro(f"spd{suf}Ok", f"{a['feasible']}/{a['n']}"); macro(f"spd{suf}Time", f1(a["time_med"])); macro(f"spd{suf}Exp", f1(a["
      exp_mean"], 0))

if (R / "exp7_lattice.json").exists():
    e7 = J("exp7_lattice"); S7 = e7["summary"]
    yn = lambda b: "yes" if b else "no"
    bad = [r for r in e7["rows"] if not r["nominal"]["ok"]]
```

```
    table("tab:latfail", "E7: the layouts in which the quintic lattice fails. ‘‘Corridor path’’ = a Frenet-grid search finds any path from the start to $s=360$\\,m inside the 6.2\\,m corridor "
        "(lateral slope $\\le0.4$, 3-disc body at road heading), with zero clearance and with the planner’s 0.35\\,m margin; the last three columns re-run the planner with a denser lattice, 2\\,m replanning, or both.",
        ["Layout", "Obstacles", "Corridor path, 0\\,m", "Corridor path, 0.35\\,m", "Dense", "2\\,m replan", "Both"],
        [[str(r["seed"]), str(r["n_obs"]), yn(r["solvable_collision_free"]), yn(r["solvable_margin"]), yn(r["dense"]["ok"]), yn(r["frequent"]["ok"]),
          yn(r["dense_frequent"]["ok"])] for r in bad], spec="rrccccccc"[:7])
    macro("lfFail", str(S7["failing"])); macro("lfSolvZero", str(S7["failing_solvable_collision_free"]))
    macro("lfSolvMargin", str(S7["failing_solvable_margin"])); macro("lfRecovered", str(max(S7["dense_ok"], S7["frequent_ok"], S7["dense_frequent_ok"])))
    macro("lfOkUnsolv", str(S7["ok_layouts_unsolvable"]))

if (R / "exp8_sensitivity.json").exists():
    e8 = J("exp8_sensitivity"); N8 = e8["n"]
    sl = e8["slam"]
    table("tab:sens_slam", f"E8: SLAM ATE RMSE [m] over {N8} seeds: mean with 95\\% bootstrap CI of the mean, median and standard deviation.",
        ["Method", "Mean [95\\% CI]", "Median", "Std"],
        [[lab, ci(sl[m], 2 if sl[m]['mean'] < 5 else 1), f"{sl[m]['median']:.2f}", f"{sl[m]['std']:.2f}"]
         for m, lab in (("icp", "ICP odometry"), ("ndt_odom", "NDT odometry"), ("ndt_loop", "NDT + loop closure"))], spec="lrrr")
    lc = e8["loc"]
    table("tab:sens_loc", f"E8: HD-map localization under LiDAR degradation ({N8} seeds, 15 unmapped vehicles). Position RMS [cm], mean with 95\\% bootstrap CI; "
        "‘‘fail’’ = maximum error above 1\\,m.", ["LiDAR", "NDT RMS [cm]", "NDT fail", "PF RMS [cm]", "PF fail"],
        [[c.replace("%", "\\%").replace("noise x4", "noise $\\times$4"), ci(lc[c]["ndt"], 1, 100), f"{lc[c]['ndt_fail']}/{N8}", ci(lc[c]["pf"], 1, 100), f"{lc[c]['pf_fail']}/{N8}"] for c in lc], spec="lrrrr")
    macro("sensSlamLoop", ci(sl["ndt_loop"], 2)); macro("sensSlamOdom", ci(sl["ndt_odom"], 2)); macro("sensSlamIcp", ci(sl["icp"], 1))
    nom = lc["nominal"]
    macro("sensNdtNom", ci(nom["ndt"], 1, 100)); macro("sensPfNom", ci(nom["pf"], 1, 100)); macro("sensSeeds", str(N8))
    for suf, c in (("Noise", "noise x4"), ("Drop", "dropout 40%")):
        macro(f"sensNdt{suf}", ci(lc[c]["ndt"], 1, 100)); macro(f"sensPf{suf}", ci(lc[c]["pf"], 1, 100))
        macro(f"sensNdt{suf}Fail", f"{lc[c]['ndt_fail']}/{N8}")

if (R / "exp9_closed_loop.json").exists():
    e9 = J("exp9_closed_loop"); A9 = e9["agg"]
    lab9 = {"gt": "Ground-truth pose", "ndt": "NDT on HD map", "dr": "VO dead reckoning"}
    table("tab:cl", "E9: closed loop (pose source $\\to$ behaviour tree $\\to$ lattice $\\to$ pure pursuit $\\to$ bicycle model), 10 runs per pose source, 7 obstacles on a 330\\,m route. "
        "Clearance is the minimum body-to-obstacle gap; cross-track is the distance of the true position to the current plan.",
        ["Pose source", "Success", "Collisions", "Min clearance [m]", "Loc. RMS [cm]", "Cross-track RMS [m]", "Replans w/o candidate"],
        [[lab9[m], f"{a['successes']}/{a['n']}", str(a["collisions"]), pm(a["min_clearance"], 2), pm(a["loc_rms"], 1, 100), pm(a["cte_rms"], 2), pm(a["no_plan"], 1)]
         for m, a in A9.items()], spec="lrrrrrr")
    macro("clGt", f"{A9['gt']['successes']}/10"); macro("clNdt", f"{A9['ndt']['successes']}/10"); macro("clDr", f"{A9['dr']['successes']}/10")
    macro("clNdtLoc", v(A9["ndt"]["loc_rms"], 1, 100)); macro("clDrLoc", v(A9["dr"]["loc_rms"], 2)); macro("clCte", v(A9["ndt"]["cte_rms"], 2))
    macro("clClrNdt", v(A9["ndt"]["min_clearance"], 2)); macro("clClrGt", v(A9["gt"]["min_clearance"], 2))

(ROOT / "numbers.tex").write_text("% AUTO-GENERATED by make_tables.py -- do not edit\n" + "\n".join(out) + "\n")
print("numbers.tex +", len(list((ROOT / "tables").glob("*.tex"))), "table files written")
```

run_all.py

```
"""Run every experiment, regenerate tables.tex and the unit tests.   Usage: python run_all.py [--quick-tests-only]"""
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent
STEPS = ["exp1_vio", "exp2_slam", "exp3_hdmap", "exp4_behavior", "exp5_planning",
         "exp6_hybrid_sweep", "exp7_lattice_failures", "exp8_sensitivity", "exp9_closed_loop"]


def run(cmd):
    print(">>", " ".join(cmd), flush=True)
    subprocess.run(cmd, cwd=ROOT, check=True)


if __name__ == "__main__":
    run([sys.executable, "-m", "pytest", "-q", "tests"])
    if "--quick-tests-only" not in sys.argv:
        for s in STEPS:                       # long: E8 (20 seeds) and E6 dominate; use experiments/expN_*.py to run one
            run([sys.executable, f"experiments/{s}.py"])
        run([sys.executable, "make_tables.py"])
    print("done -> compile main.tex (pdflatex x2)")
```